SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

CSA37 – Software Testing

ASSIGNMENT 4 – CO4

Construct, Compare and Justify Test Planning, Test Documentation and Test Organisation for an Electricity Bill Payment Portal

PowerPay Electricity Bill Payment and Consumer Service Portal

CO4: Construct and organize software testing processes, including test planning, to perform test documentation activities.

Student Name	SAJAN S	**Register Number**	1911260014
Department / Section	CSE	**Academic Year / Semester**	2026-27
Faculty Name	Dr.Gnanajeyaraman	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission		**Date of Evaluation**	

Individual assignment report – Tasks 1 to 9 (50 marks)

Evidence statement: figures marked MEASURED were produced by running the scripts listed in Appendix B; figures marked MODELLED or ILLUSTRATIVE are assumptions or constructed examples and are labelled as such wherever they appear.

Table of Contents

Executive Summary iv

Scope, Conventions and Evidence Status v

Source documents and how they were used v
Evidence labels v
Results at a glance v
Mandatory evidence check vi

1. Testing and Debugging Goals: Compare Fault Localisation and Repair Approaches [4 Marks] 1

1.1 Context and objective 1
1.2 Prediction recorded before the exercise 1
1.3 Experimental set-up 2
1.4 Locating each fault with approaches A, B and C 4
1.5 Fixing the faults and writing regression tests 7
1.6 Testing versus debugging in the records of the exercise 9
1.7 Comparison of the three approaches 10
1.8 Evaluation: the approach developers should be required to follow 11
1.9 Critique of an AI assistant for locating and repairing the bill fault 11
1.10 Observation against prediction, summary and recommendation 12

2. Test Planning and Test Plan Components: Compare Test Plan Structures [7 Marks] 13

2.1 Context and objective 13
2.2 Prediction recorded before the exercise 13
2.3 Assumptions on which the plans rest 14
2.4 The three plans for PowerPay 14
2.5 Effort estimate and feasibility 18
2.6 Scoring the plans against the IEEE 829 checklist 19
2.7 Risk analysis 21
2.8 Handling a mid-project change: the tariff revised in week 6 22
2.9 Evaluation and justification of the plan for approval 23
2.10 Critique of an LLM-generated test plan for PowerPay 24
2.11 Observation against prediction, summary and recommendation 25

3. Test Items Identification and Test Plan Attachments: Compare Identification Approaches [5 Marks] 26

3.1 Context and objective 26
3.2 Prediction recorded before the exercise 26
3.3 Test items identified by the three approaches 26
3.4 Plan attachments for the bill-calculation feature 28
3.5 Simulation of the pilot problem: a wrong tariff file in the test environment 29
3.6 Traceability coverage of the test cases 30
3.7 Can an LLM build the traceability matrix? 32
3.8 Comparison of the three approaches 33
3.9 Evaluation: which approach and which attachments are compulsory 34
3.10 Observation against prediction, summary and recommendation 34

4. Test Reporting: Compare Test Reporting Approaches [6 Marks] 35

4.1 Context and objective 35
4.2 Prediction recorded before the exercise 35
4.3 Test execution and the test log 35
4.4 Three test incident reports 37
4.5 Data and reports for the three approaches 38

4.6 What each report tells the Chief Engineer and what it hides 39
4.7 Quality of the incident reports: reproducibility 40
4.8 Effort of producing each report 41
4.9 Evaluation: the reporting set the project should use 41
4.10 Critique of a test summary report written by an LLM 42
4.11 Observation against prediction, summary and recommendation 42
5. Test Policy Development: Compare Test Policy Approaches [7 Marks] 44
5.1 Context and objective 44
5.2 Prediction recorded before the exercise 44
5.3 The three draft policies 44
5.4 Measurable policy statements 46
5.5 Policies against the pilot-year problems 47
5.6 Application to a small and to a large project 47
5.7 Cost, effort and likelihood of being followed 48
5.8 Evaluation: the policy the Corporation should adopt 48
5.9 Critique of an LLM-generated test policy 49
5.10 Observation against prediction, summary and recommendation 49
6. Test Team Organisation: Compare Team Structures [4 Marks] 51
6.1 Context and objective 51
6.2 Prediction recorded before the exercise 51
6.3 Organisation charts and assignment of the thirteen people 51
6.4 Simulation of three situations 53
6.5 Analysis of the structures 54
6.6 Evaluation: the recommended structure 55
6.7 Critique of giving the vendor the right to approve the release 55
6.8 Observation against prediction, summary and recommendation 55
7. Skills of a Test Specialist: Compare Skill-Building Strategies [5 Marks] 57
7.1 Context and objective 57
7.2 Prediction recorded before the exercise 57
7.3 The skills matrix and the scores of the four testers 57
7.4 Four-week plans for the two fresh graduates 58
7.5 Skills built, skills left undeveloped, and the link to pilot defects 60
7.6 Cost and time against the budget and the team's effort 60
7.7 Evaluation: the strategy the test manager should use 61
7.8 How AI tools change the skills of a test specialist 61
7.9 Observation against prediction, summary and recommendation 62
8. Integrated Test Process Evaluation: Compare Test Documentation Sets [4 Marks] 63
8.1 Context and objective 63
8.2 Prediction recorded before the exercise 63
8.3 The integrated test process and its documents 64
8.4 Documents of the three sets for the bill-generation feature and their effort 64
8.5 Analysis of the sets and of the pilot-year problems they would expose 66
8.6 Evaluation: the documentation set to adopt and the rule for skipping a document 67
8.7 Can AI-generated documents be accepted for audit without review? 67
8.8 Observation against prediction, summary and recommendation 68
9. Re-evaluate and Redesign the Test Process [8 Marks] 69
9.1 Impact analysis 69
9.2 Evaluation of the original process 72
9.3 Compare and adapt 73
9.4 Evidence-based defence 77

9.5 Observation against prediction, summary and recommendation 79
10. Conclusion 80
References 81
Appendix A. Assumptions Register and SRS Excerpt 82
Appendix B. Scripts and Evidence Index 84
Appendix C. Test Cases and Results 85
Appendix D. Templates for Evidence to be Inserted 87
D.1 Record sheet for an AI-assistant session (Tasks 1, 2, 4, 5) 87
D.2 Incident report template 87
D.3 Skills assessment sheet (Task 7) 87

Executive Summary

This report answers the nine tasks of Software Testing Assignment 4 (CSA37, CO4) for the PowerPay electricity bill payment and consumer service portal. PowerPay serves 3 lakh consumers through six modules, has a test budget of Rs 1,50,000 and a ten-week schedule with four testers. The pilot showed four problems: a bill calculated wrongly at exactly 500 units, a tested build that was not the released build, no way of saying how much testing had been done, and 52 defects found in testing against 21 found by consumers and staff (a Defect Detection Effectiveness of 71.2%), with 18% of defects reopened. Each task compares three approaches, tests them with an exercise, chooses one, and states when it fails and what happens if it is left out. Task 9 then re-plans the process for a changed requirement: 12 lakh consumers, smart-meter upload every 15 minutes, four weeks, three testers in two offices, masked data and auditable results.

What was done. The slab bill calculation was implemented with four seeded faults and the three localisation approaches were run on it, including a real `git bisect` on a ten-commit history. A wrong tariff file was deployed to a simulated test environment. Twenty-three test cases were executed against a simulated build to produce a test log and three incident reports. A traceability matrix of 15 requirements and 33 test cases was analysed by script. Effort, risk, cost and coverage were calculated from the unit times given in the assignment. All execution outputs shown as console images are real; hours of human effort, costs and probabilities of being followed are modelled assumptions and are labelled as such.

What was found. In Task 1, systematic localisation took a modelled 19 minutes for four faults against 34.3 for the debugger and 53 for print statements, but an absolute-value probe made `git bisect` name the wrong commit twice, a real lesson on masked faults. In Task 2 the effort is 269.1 person-hours (22.4% of the 1,200-hour capacity), and a risk-based plan with an IEEE 829 annex was chosen. Task 3 showed that a baseline check detects a wrong tariff file with no test, while a suite that stops at 500 units would not detect a wrong top-slab rate. Task 4 found that 7 failed tests came from 3 defects and a 68.2% pass rate. Task 5 recommends a signed policy with eight measurable statements. Task 6 recommends module-aligned teams with safeguards. Task 7 recommends mentoring, measured by a skills matrix. Task 8 recommends a risk-tailored documentation set at 110 hours against 228 for the full set.

The redesign. For the changed requirement the effort rises to 598 person-hours against 360 available, a shortfall of 238 hours (39.8%). No strategy can test everything. The process is modified (strategy B): High-risk conditions are executed three times and Medium-risk once, covering 269 of 400 conditions and 87.3% of the risk exposure, with exit criteria that include

an independent reconciliation sample of 3,000 bills, load targets from 115.2 million readings a day, masking, and an audit trail. The 131 conditions left untested are listed for the Chief Engineer's written acceptance, and the process fails if the tariff regulation changes late, for which a reserve and a fifth-week option are recommended.

Scope, Conventions and Evidence Status

Source documents and how they were used

The question document (Assignment 4, PowerPay) defines the scenario, the nine tasks and the marks (4, 7, 5, 6, 7, 4, 5, 4 and 8 = 50). The Assignment 1 answer key (a different scenario, the university student portal) was used only for the style and depth of the answers: the structure of each task, the combination of executed evidence and modelled figures, and the form of the comparison tables. No content of that key was copied into this report. The SRS excerpt, the schedule and the numbers not given in the question are the author's assumptions and are listed in Appendix A.

Evidence labels

Table S.1: Labels used for the status of numbers and examples

Label	Meaning	Examples
MEASURED	Produced by running a script in the container used to prepare this report; the script is described in Appendix B	Pass or fail of tests, bisection steps, debugger steps, tariff detection, test log, checksums, masking output
MODELLED	Calculated from stated unit times or assumptions; must be replaced by observed data in a real project	Minutes of analyst effort, hours of documentation, costs, hop delays
ILLUSTRATIVE	Constructed to show a pattern; not a record of a real event	The 100-test pass-rate example, the AI-style candidate fixes, the LLM statements checked in Tasks 2, 4, 5
ASSUMED	A value not given in the question; stated in Appendix A	Tariff reading, 2,000 concurrent users, 40 tariff-dependent conditions
TO BE COMPLETED	Evidence that needs a real person or tool and that has not been produced	Classmate reproducibility test, real AI-assistant transcripts, real skill assessment

> **Honesty statement:** No screenshot, tool output or test result in this report was invented. The console images are renderings of real script output. No actual PowerPay software or hospital, utility or vendor system was tested; the portal was *simulated* by small programs written for this report. Where the assignment asks for evidence that needs a real person (the classmate session of Task 4, the real LLM transcripts, the real skills assessment), the report gives the method and a record sheet and marks the evidence as to be completed.

Results at a glance

Table S.2: Summary of the nine tasks

Task	Marks	Approaches compared	Choice	Key figure
1 Debugging	4	A print; B debugger; C systematic	C as process, A and B as tools	19 / 34.3 / 53 min (modelled); 67 automated runs
2 Test plan	7	A agile; B IEEE 829; C risk-based	C with the annex of B	269.1 h; 22.4% of capacity; score 24/30
3 Identification	5	A build no.; B baseline; C traceability	B always, C for billing and payment	Wrong tariff file found at deployment by B; by test 5 by C
4 Reporting	6	A log; B dashboard; C summary	All three, layered	22 executed, 15 passed, 7 failed = 3 defects
5 Policy	7	A informal; B formal; C with strategy	B now, C as the path	8 measurable statements; DDE 71.2% to 90%
6 Team	4	A levels; B modules; C vendor	B with safeguards	Dispute 4 h vs 16 and 24 h (modelled)
7 Skills	5	A classroom; B mentoring; C rotation	B base, C measure, A targeted	Domain 2.25 and planning 2.00 the weakest
8 Documents	4	A full; B lean; C hybrid	C	110 h vs 228 h vs 75 h
9 Redesign	8	A retain; B modify; C redesign	B (modify)	598 h vs 360 h; 87.3% of risk exposure covered

Mandatory evidence check

Table S.3: Evidence expected for each task and its status (P = partly present, content given; real evidence to be inserted)

Task	Evidence expected	Status	Where
1	Time and runs table, comparison, justified choice, regression tests; AI-assistant critique	P: AI transcripts illustrative	Sections 1.3 to 1.9; Appendix D
2	Three plans, effort calculation, checklist, risk analysis, contingency, change handling, LLM critique	Present (LLM statements illustrative)	Sections 2.4 to 2.10
3	Item lists, attachments, simulation, traceability matrix, LLM judgement	Present (LLM stand-in is a keyword baseline)	Sections 3.3 to 3.7
4	Test log, three incident reports, reports of A, B, C, reproducibility result, justified set	P: classmate session to be completed	Sections 4.3 to 4.10
5	Three policies, measurable statements, problem table, cost, choice, LLM critique	Present	Sections 5.3 to 5.10
6	Three charts, situation table, justified structure, vendor release critique	Present	Sections 6.3 to 6.7
7	Skills matrix with scores, three plans, cost and gap table, strategy, AI skills	P: scores are the author's judgement	Sections 7.3 to 7.8
8	Documents of three sets, flowchart with owners, effort table, rule, AI audit judgement	Present	Sections 8.3 to 8.7
9	Impact analysis, effort calculation, masking, checklist, comparison, flowchart, exit criteria, defence	Present	Sections 9.1 to 9.4

1. Testing and Debugging Goals: Compare Fault Localisation and Repair Approaches [4 Marks]

1.1 Context and objective

The pilot of PowerPay produced a bill that was calculated wrongly for a consumption of exactly 500 units. The developers' response was that they would "find and fix it quickly". That sentence hides three separate questions. How is the fault located? How is it repaired? And what remains afterwards to show that the repair is correct and will stay correct? In the pilot year 18% of the reported defects were reopened, which means that in roughly one case in five the answer to the third question was "nothing". This task therefore compares three ways of locating and repairing a fault after a test has failed, using an implementation of the PowerPay slab calculation with four seeded faults, and it uses the records of the exercise to separate the goals of testing from the goals of debugging.

The vocabulary used throughout follows the standard definitions used in this course. A *failure* is the observable deviation of the program from its required behaviour (a bill of Rs 2,702 where Rs 2,700 is required). A *fault* (defect, bug) is the incorrect piece of code or data that causes the failure. An *error* is the human mistake that put the fault there. *Testing* is the activity that reveals failures by executing the software against expected results; its goal is to show that the software does what it should and to expose where it does not. *Debugging* starts after a test has failed; its goal is to locate the fault that caused the failure, to repair it, and to confirm that the repair works. Testing therefore ends at the failing result and starts again at the retest, while debugging occupies the interval between them. The distinction matters organisationally because a tester can run a test without knowing any code, whereas debugging needs access to the code, the history of changes and the design intent.

1.2 Prediction recorded before the exercise

The instructions require a prediction to be recorded before the exercise is run and compared with the observation afterwards. The predictions below were derived from the design of the exercise (four faults, ten commits, one failing report each). They were not independently time-stamped, so they are presented as a reasoning device and as the baseline for the comparison in Section 1.10.

Table 1.1: Prediction for Task 1

Technique	Predicted time to locate four faults	Predicted repeatability	Predicted evidence left behind
A. Print and log statements	Slowest; about 4 to 5 minutes per rerun, 2 to 3 reruns per fault	Low: depends on where the developer chooses to print	Almost none once the print statements are deleted
B. Interactive debugger	Medium; fastest for faults near the start of execution, slower for deep faults	Medium: a session is not recorded unless notes are made	Session notes only if the developer writes them
C. Systematic localisation	Fastest overall after a one-time set-up; cost grows only with the logarithm of the history	High: probe script and minimal input can be rerun by anyone	Minimal failing input, first bad commit, regression test

1.3 Experimental set-up

1.3.1 Business rule and its interpretation

The question states the slabs as 0 to 100 units at Rs 3.00, 101 to 300 at Rs 5.00, 301 to 500 at Rs 7.00 and above 500 at Rs 9.00 per unit. Two readings are possible. In the *whole-consumption* reading all units are charged at the rate of the slab in which the total falls. In the *telescopic* (incremental) reading each unit is charged at the rate of the slab in which that unit lies. This report adopts the telescopic reading, which is the usual form of a slab tariff, and records it as Assumption A1 in Appendix A. Under this reading the bill for 500 units is 100 x 3.00 + 200 x 5.00 + 200 x 7.00 = Rs 2,700, and the bill for 501 units is Rs 2,709. The reading must be confirmed with the billing-section coordinator, because under the whole-consumption reading the same 500 units would cost Rs 3,500 and the whole test oracle would change. The ambiguity is itself a defect-prevention finding: a test oracle can be no better than the rule it encodes.

1.3.2 Reference implementation

The reference implementation below was written for this exercise and is not PowerPay code. It was committed as the first version (c01) of a Git repository.

```
SLABS = [(100, 3.00), (300, 5.00), (500, 7.00), (None, 9.00)]   # (upper limit,
Rs per unit)

def calculate_bill(units):
    if units < 0:
        raise ValueError("units must be >= 0")
    bill = 0.0
    lower = 0
    for upper, rate in SLABS:
        if upper is None or units <= upper:
            bill += (units - lower) * rate
            break
        bill += (upper - lower) * rate
        lower = upper
    return round(bill, 2)
```

1.3.3 The four seeded faults

Four faults of different types were introduced in four different commits, separated by harmless commits, so that the history resembles a real repository. The commit messages are deliberately innocent, as they usually are in practice.

Table 1.2: Seeded faults and the commit that introduced each

ID	Fault	Type	Introduced in	Smallest failing input
F1	Upper limit of slab 3 coded as 499 instead of 500, so unit 500 is charged Rs 9 (off-by-one at 500 units)	Boundary / off-by-one	c09 Tidy slab limits	500 units
F2	Rate of slab 2 typed as 5.50 instead of 5.00	Data / constant	c05 Update tariff constants	101 units
F3	Validation of negative consumption removed during a refactoring	Missing check	c03 Refactor input handling	-1 unit
F4	Units above 500 are all charged at Rs 9 (the rate is applied to the whole consumption)	Logic / computation	c07 Optimise top-slab calculation	501 units

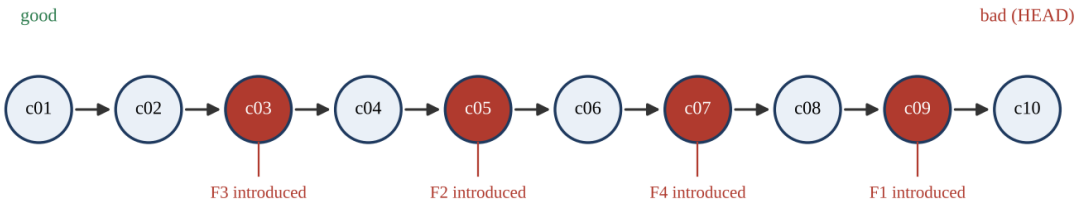


Figure 1.1: Commit history of the exercise repository (ten commits; red circles introduce a fault)

Figure 1.2 shows what each fault does to the bill. F1 changes the bill only by Rs 2 at 500 units and by Rs 2 for every consumption above 500, which is why a human checking one bill can miss it. F2 grows from the first unit of slab 2 and stays constant above 300 units. F4 produces the largest error, Rs 1,800 at 600 units. F3 is not visible on the chart because it concerns invalid input.

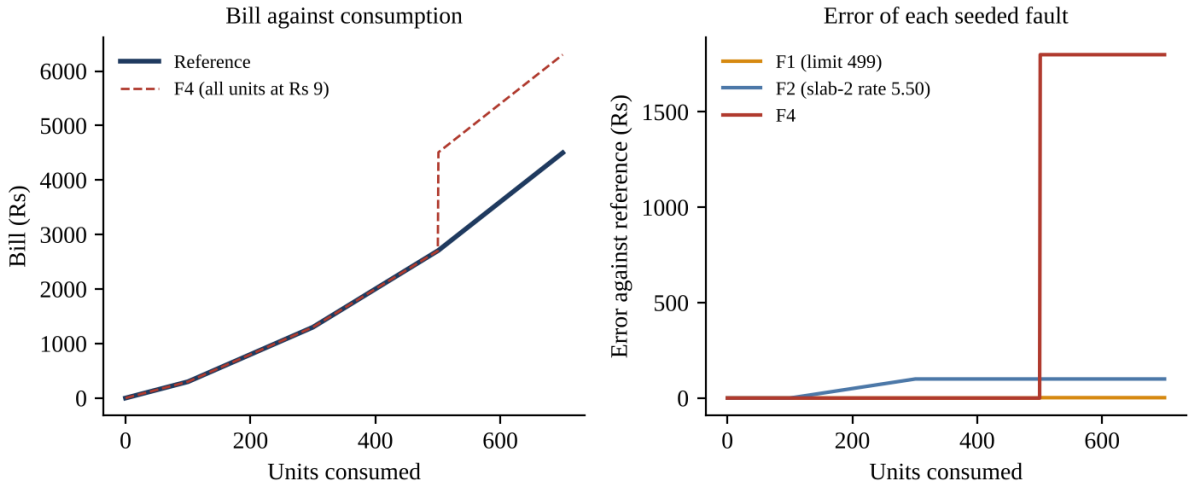


Figure 1.2: Bill against consumption and the error of each seeded fault against the reference

1.3.4 What was measured and what was modelled

> **Evidence status:** The numbers of program executions, the numbers of debugger steps, the Git bisection steps and the pass or fail status of every test in this task were produced by running the scripts in Appendix B in the container used to prepare this report . The minutes of human effort are **modelled** from unit times stated in Section 1.4 because no stopwatch observations of a human developer are available; the student must replace them with logged times if the faculty expects observed values.

1.4 Locating each fault with approaches A, B and C

1.4.1 A. Print and log statements

Protocol. Round 1 prints the input and the result of the failing case and confirms the failure. Round 2 prints, for every slab visited, the slab limit, the rate, the lower bound and the running bill. Round 3 (only if needed) prints the condition of the suspect branch. Each round is one edit and one rerun. For F1 the round-2 output is:

```
Approach A: round-2 print output for F1 (units=500) and F4 (units=600)   (real output)
[F1] result=2702.0
units=500
  slab upper=100 rate=3.0 lower=0 bill_so_far=0.0
  slab upper=300 rate=5.0 lower=100 bill_so_far=300.0
  slab upper=499 rate=7.0 lower=300 bill_so_far=1300.0
  slab upper=None rate=9.0 lower=499 bill_so_far=2693.0
[F4] result=5400.0
units=600
  slab upper=100 rate=3.0 lower=0 bill_so_far=0.0
  slab upper=300 rate=5.0 lower=100 bill_so_far=300.0
  slab upper=500 rate=7.0 lower=300 bill_so_far=1300.0
  slab upper=None rate=9.0 lower=500 bill_so_far=2700.0
```

Figure 1.3: Console output of the print rounds for F1 and F4 (real run)

A developer who knows that slab 3 ends at 500 sees "upper=499" and "lower=499" in the last line and has located the fault. A developer who does not remember the specification must look

it up, which is why the SRS number must be quoted in the incident report. For F4 two rounds are not enough: the round-2 output shows the running bill jumping from Rs 2,700 to Rs 5,400, and a third print inside the top-slab branch is needed to see that the whole consumption is multiplied by 9. The reruns needed were therefore 2, 2, 2 and 3 for F1 to F4, nine in total. The time model assumes 5 minutes per round (2 minutes to edit, 1 minute to run, 2 minutes to read) and 2 minutes at the start to calculate the expected bill by hand.

1.4.2 B. Interactive debugger

A debugger session sets a breakpoint at the first line of the function and steps line by line while watching `units`, `bill`, `lower`, `upper` and `rate`. The number of steps that a developer must take before a wrong value becomes visible can be measured exactly, by tracing the failing mutant and the reference side by side and finding the first line event at which the watched variables differ. This is the minimum number of steps for a developer who notices the difference at once, and a lower bound for a real session. The measured values are 13 steps for F1 (the fault shows only when the third slab is visited), 9 for F2, 2 for F3 (the reference stops at the validation, the mutant continues) and 7 for F4, 31 steps in all, in four sessions. The time model assumes 4 minutes to set up the session, 2 minutes to calculate the expected values by hand and 20 seconds per step.

```
Approach B: single-steps until the locals first differ from the reference trace   (real output)
F1: input=500  reference trace length=16  mutant trace length=20  first visible divergence at step 13
F2: input=150  reference trace length=12  mutant trace length=12  first visible divergence at step 9
F3: input=-1  reference trace length=2  mutant trace length=7  first visible divergence at step 2
F4: input=600  reference trace length=20  mutant trace length=23  first visible divergence at step 7
```

Figure 1.4: Console output: debugger steps until the state first differs from the reference trace (real run)

1.4.3 C. Systematic fault localisation

The systematic approach has three mechanical parts. First, the failing input is reduced to a minimal case. A batch of 40 consumption values (random seed 2026) fails for F1, F2 and F4, and a batch with one negative value fails for F3. Delta debugging (the *ddmin* algorithm) halves the batch repeatedly and keeps the part that still fails, and a binary search then finds the smallest failing value. Second, a probe script is written that exits with status 1 when the specific fault is present, and `git bisect run` applies it to the ten-commit history to find the first bad commit. Third, a regression test is written and committed with the fix. The reductions and bisections were run for real:

Table 1.3: Input reduction and bisection results (measured)

Fault	ddmin runs	Reduced batch	Smallest failing value	Value-search runs	Bisect runs	First bad commit found
F1	6	[514]	500	9	4	c09 Tidy slab limits

Fault	ddmin runs	Reduced batch	Smallest failing value	Value-search runs	Bisect runs	First bad commit found
F2	5	[121]	101	7	3	c05 Update tariff constants
F3	6	[-7]	-1	6	3	c03 Refactor input handling
F4	6	[514]	501	9	3	c07 Optimise top-slab calculation

```
Approach C: ddmin input reduction and value search   (real output)
F1: ddmin runs=6, reduced batch=[514], smallest failing value=500 after 9 extra runs
F2: ddmin runs=5, reduced batch=[121], smallest failing value=101 after 7 extra runs
F3: ddmin runs=6, reduced batch=[-7], smallest failing value=-1 after 6 extra runs
F4: ddmin runs=6, reduced batch=[514], smallest failing value=501 after 9 extra runs
```

Figure 1.5: Console output: ddmin input reduction and value search (real run)

```
$ git bisect run python3 probes/p1.py   (real output, differential probe for F1)
running 'python3' '/home/claude/work/probes/p1.py'
Bisecting: 2 revisions left to test after this (roughly 1 step)
[ab70dfd504cacb6f503593834aaadfa6a0b345ab] c07: Optimise top-slab calculation
running 'python3' '/home/claude/work/probes/p1.py'
Bisecting: 0 revisions left to test after this (roughly 1 step)
[1b84806f8459daa061f2df12d9668aa219c7b656] c09: Tidy slab limits
running 'python3' '/home/claude/work/probes/p1.py'
Bisecting: 0 revisions left to test after this (roughly 0 steps)
[ccbbe37f7c83e5dba73d548d32797d3f4787d4f0] c08: Add receipt formatter for SMS
running 'python3' '/home/claude/work/probes/p1.py'
1b84806f8459daa061f2df12d9668aa219c7b656 is the first bad commit
commit 1b84806f8459daa061f2df12d9668aa219c7b656
Author: tester <a@b.c>
Date:   Wed Nov 4 07:46:42 2026 +0000
    c09: Tidy slab limits
 bill.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
bisect found first bad commit
```

Figure 1.6: Console output: git bisect run with the differential probe for F1 (real run)

In every case the bisection returned exactly the commit that had introduced the fault, and the diff of that commit is one line long, so the fault is located to the line. The wall-clock time of each bisection was below 0.1 second; the analyst's attention is needed only for writing the probe (modelled as 2 minutes), reading the reduction (1 minute) and reading the diff (1 minute), 4 minutes per fault plus 3 minutes once to prepare the harness.

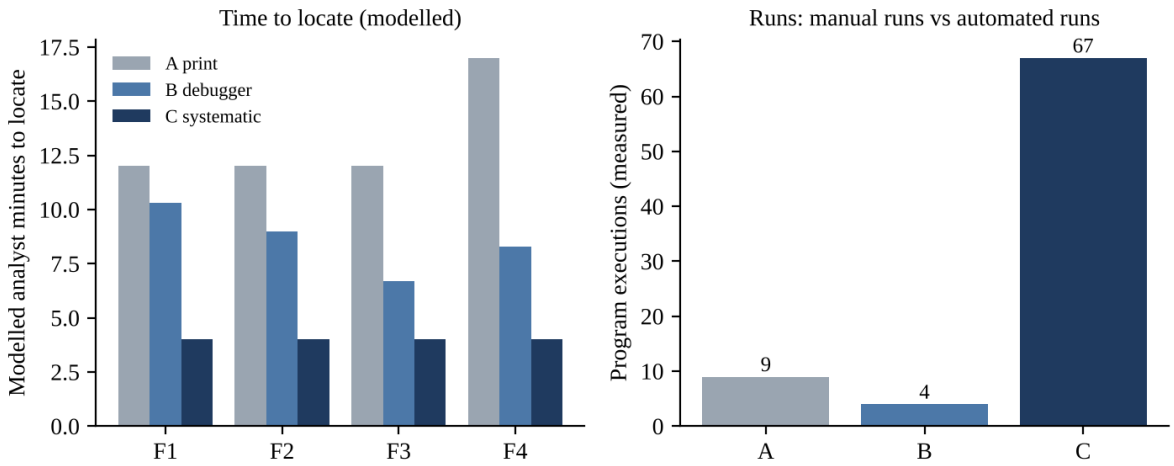


Figure 1.7: Modelled analyst time to locate each fault (left) and measured program executions (right)

1.4.4 A caution found while measuring: bisection with an unreliable probe

The first version of the probe scripts compared the bill for the *reported* input with the absolute expected value, for example `calculate_bill(500) == 2700`. When that probe was used for F1, `git bisect` named commit c05 (Update tariff constants), not c09. The reason is that F2, introduced earlier, also makes the bill at 500 units wrong, so the absolute probe starts failing at c05 and the bisection correctly reports the first commit at which *the probe* fails, which is not the commit that introduced *the fault under investigation*. The same happened for F4 (c05 instead of c07). A differential probe, which checks only the property that the fault violates (for F1, that the 500th unit costs Rs 7: `bill(500) - bill(499) == 7`), returned the correct commits. This is a real finding of the exercise: systematic localisation is reliable only when each probe isolates one fault, and it also explains one of the mechanisms behind reopened defects, discussed in Section 1.5.

Table 1.4: Absolute probe versus differential probe for bisection (measured)

Fault	Absolute probe (input from the report)	Differential probe	Correct commit
F1	c05 (wrong: masked by F2)	c09	c09
F2	c05	c05	c05
F3	not applicable	c03	c03
F4	c05 (wrong: masked by F2)	c07	c07

1.4.5 Summary of time, runs and tools

Table 1.5: Time, runs and tools to locate the four faults

Approach	Tool	Runs (measured)	Steps / probes (measured)	Time to locate, four faults (modelled)	Evidence left behind
A. Print and log	Editor, `print`, terminal	9 manual reruns	none	53 min (12 + 12 + 12 + 17)	None after the prints are removed
B. Interactive debugger	Python debugger / IDE	4 sessions	31 single steps	34.3 min (10.3 + 9.0 + 6.7 + 8.3)	Notes only if written
C. Systematic	`ddmin` script, `git bisect run`, probe scripts	67 automated runs (15 to 19 per fault)	4 probes, 4 first-bad commits	19 min (4 x 4 + 3 set-up)	Minimal input, probe, commit id, diff

The numbers favour C, but the result must be read with care. With only ten commits and a function of fifteen lines, approach B is not far behind. The advantage of approach C grows with the size of the problem, because bisection needs at most the ceiling of log2(n) test runs for a history of n commits: 4 runs for n = 10, 9 runs for n = 500 and 10 runs for n = 1,000, while the number of print rounds and debugger steps grows with the size of the code and the depth of the call chain. PowerPay is a system of six modules with tariff data, and a failing bill may involve code, configuration and a tariff file written by different people, which is the case in which a history search is the only method that does not depend on the developer's memory.

1.5 Fixing the faults and writing regression tests

Each fault was fixed in its own commit (c11 to c14) and one regression test was written for each. The tests use absolute expected values taken from the SRS rule, and each fails on the faulty code and passes after its fix, with the one exception explained below.

Table 1.6: Fixes, changes made and regression tests (measured)

Fault	Change made	Regression test	Before fix	After its own fix	After all four fixes
F1	`(499, 7.00)` changed to `(500, 7.00)`	RT1: bill(500) = 2700.00 and bill(501) = 2709.00	FAIL	**FAIL** (masked by F2: bill(500) = 2800)	PASS
F2	`(300, 5.50)` changed to `(300, 5.00)`	RT2: bill(101) = 305.00 and bill(300) = 1300.00	FAIL	PASS	PASS
F3	Check `units < 0` restored at entry	RT3: bill(-1) raises ValueError	FAIL	PASS	PASS
F4	Special branch removed; only units above 500 charged at Rs 9	RT4: bill(600) = 3600.00, bill(1000) = 7200.00	FAIL	PASS	PASS

The first fix, for the reported 500-unit failure, did not make the reported symptom disappear. After F1 was repaired, bill(500) was Rs 2,800 instead of Rs 2,700, because F2 adds Rs 100 to every bill above 300 units. A developer who stops at this point concludes that the fix "did not work", reverts it or adds another patch. Only a regression test that is *kept* and a search that goes on until the oracle agrees can resolve the situation. After the four fixes the complete suite (the four regression tests and a boundary table of eight values: 0, 1, 100, 101, 300, 301, 500, 501) ran as follows:

```
$ python3 -m unittest -v tests.test_bill_regression   (real output at commit c14)
test_boundaries ... ok
test_RT1_F1_exactly_500_units ... ok
test_RT2_F2_slab2_rate ... ok
test_RT3_F3_negative_units_rejected ... ok
test_RT4_F4_top_slab_marginal_only ... ok

----------------------------------------------------------------------
Ran 5 tests in 0.000s

OK
```

Figure 1.8: Console output: complete regression suite at commit c14 (real run)

1.5.1 Fixes that introduce new faults, and the 18% reopen rate

None of the four fixes made in this exercise introduced a new fault, but two realistic ways to produce one were examined and executed. The first is the **symptom fix**: the developer special-cases the reported input (`if units == 500: return 2700`). It makes the reported case pass, and bill(499) is still correct, but bill(501) is Rs 2,711 instead of Rs 2,709, bill(600) is Rs 3,602 instead of Rs 3,600 and bill(1000) is Rs 7,202 instead of Rs 7,200. The defect is closed and then reopened when a consumer above 500 units complains. The second is the **over-correcting fix** of the validation fault: if F3 is repaired as `if units <= 0` instead of `if units < 0`, the function rejects zero consumption, and a consumer whose meter did not move in the month cannot be billed. Both are new faults caused by a repair that was not checked against the specification and against neighbouring values.

The pilot data give 52 defects found in testing and 21 found by consumers and staff, 73 in all. An 18% reopen rate on 73 reported defects is about 13 defects (0.18 x 73 = 13.1) that were declared fixed and failed again. Taking the average fix time of 4 days, and assuming that a reopened defect costs one more fix cycle, this is of the order of 13 x 4 = 52 further developer-days (modelled). The records of this exercise suggest four causes, all of which approach A leaves open and approach C closes: (1) symptom fixes of the reported input; (2) masked faults, where fixing one fault leaves the symptom alive because another fault is present, so the defect looks unfixed; (3) no regression test, so that a later commit re-introduces the fault (c09 in this history re-broke the slab limits under the title "Tidy slab limits"); and (4) no check of the fix

on the neighbours of the boundary. The requirement in approach C that each fix carries a regression test that fails before and passes after, plus the boundary table, addresses all four.

1.6 Testing versus debugging in the records of the exercise

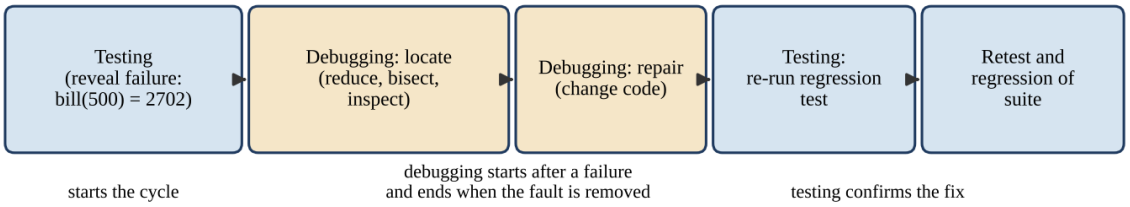


Figure 1.9: Where testing and debugging start and end in the exercise for the 500-unit failure

Table 1.7: Testing activity and debugging activity compared, using the records of this exercise

Question	Testing	Debugging
Goal	Reveal that bill(500) = 2,702 instead of 2,700 (a failure)	Find that slab 3 ends at 499 (the fault) and repair it
Starts when	A test case is executed with an expected result	A test has failed and the failure has been recorded
Ends when	The result is recorded as pass or fail; later, when the regression test passes	The fault is removed and the regression test passes
Input from the other activity	Receives the fixed build for retest	Receives the failing case, the expected value and the build identifier
Needs the code?	No	Yes, and the history of changes
Evidence produced in this exercise	Failing probe results, 5-test suite result	Minimal input, first bad commit, diff, fix commit
Who should do it	An independent tester (Task 6 of this report discusses the organisation)	The developer who owns the module

The comparison shows where the activities touch. The test result is the *input* of debugging and the regression test is the *output* of debugging that returns to testing. A team that runs the two as one activity ("the developer clicks around and fixes") has no point at which an independent judgement is made that the product is correct, and the figure of 52 defects found in testing against 21 found in the field suggests that the pilot suffered from exactly that.

1.7 Comparison of the three approaches

Table 1.8: Feature comparison of approaches A, B and C

Criterion	A. Print and log	B. Interactive debugger	C. Systematic localisation
Time to locate (four faults, modelled)	53 min	34.3 min	19 min
Repeatability by another person	None: depends on where the prints were put	Low to medium: a session is not recorded	High: probe, input and commit id can be rerun
Works when the fault was introduced by a change	Only if the developer suspects the change	Only if the developer suspects the change	Designed for this case (bisection)
Works for a fault in data (tariff file)	Yes, if the file is printed	Yes, if the data are watched	Yes, if the data file is under version control (Task 3)
Effect of code size and history	Grows with code size	Grows with call depth	Grows with log2 of history length
Evidence left behind	None	Notes	Minimal case, probe, commit, regression test
Skill required	Low	Medium	Medium (version control, scripting)
Risk of a masked fault	High	High	Low if probes are differential; high otherwise
Suited to production incidents (no debugger)	Yes (logging)	No	Yes, on the test copy

1.7.1 When each approach fails and what happens if it is left out

Table 1.9: Failure conditions and consequences of omission

Approach	Fails when	If it is left out
A. Print and log	The code path is long or concurrent (payment callbacks), or the fault is in timing; output is not read against the specification	Production incidents cannot be examined at all, because logging is the only window into a live system; keep structured logging as an evidence source, not as the method
B. Interactive debugger	There is no access to the running system (production) or the fault depends on timing or load (the duplicate payment of the pilot)	Developers lose the fastest way to inspect the state of a single failing run; they would have to rely on guesses or on bisection alone
C. Systematic localisation	The fault is not in the history (it was in the original design), the history has few or very large commits, or the probe covers more than one fault	No minimal reproducible case, no commit evidence and no regression test is left; the same fault returns, which is what an 18% reopen rate looks like

1.8 Evaluation: the approach developers should be required to follow

The developers should be required to follow approach C as the *process*, and are free to use A or B as the *tools* inside it. The justification rests on the three criteria requested. **Time to locate.** C was the fastest in this exercise (19 minutes against 34.3 and 53) and its cost grows slowly. **Repeatability.** Only C produces a case that another person can rerun: a reduced input, a probe

and a commit. A and B depend on the person. **Evidence left behind.** For a system that bills lakhs of households and will be audited, the evidence of what was wrong, where it was found and what test now protects it is as valuable as the repair. C leaves four artefacts; A leaves none.

The required procedure is stated as a rule that can be checked: *a defect may be set to Fixed only when the incident report names the minimal failing input, the commit that introduced the fault (or states that none exists), the change made, and a regression test that failed before and passes after the change.* The rule costs about 6 minutes per defect for writing the test (modelled) and removes the main cause of reopened defects. The residual risk is that the rule is satisfied formally, with a weak test. The mitigation is that the test manager reviews a sample of regression tests, as Task 8 proposes for documents, and the boundary table of Section 1.5 is part of the review.

1.9 Critique of an AI assistant for locating and repairing the bill fault

An AI assistant can read the function, notice the suspicious constant and propose a repair within seconds. The question is whether the explanation is correct, whether the fix is complete, and whether it passes the regression tests. The check below was applied to four candidate repairs of the faulty version c10. **The candidates are illustrative**: they were written for this report to represent the kinds of repair that automated assistants commonly propose, and are not a transcript of a particular assistant. The student should repeat the exercise with a real assistant, paste the transcript in Appendix D and fill the same table.

Table 1.10: Four illustrative candidate repairs applied to the faulty version (measured by running the tests)

Candidate repair	bill(500)	RT1	RT2	RT3	RT4	Boundary suite	Verdict
1. Special-case `units == 500`	2700	FAIL	FAIL	FAIL	FAIL	FAIL	Symptom hidden; incomplete
2. Change <= to < in the slab test	2802	FAIL	FAIL	FAIL	FAIL	FAIL	Wrong explanation; breaks other boundaries
3. Correct slab-3 limit only (F1)	2800	FAIL	FAIL	FAIL	FAIL	FAIL	Correct for one fault; reported case still wrong
4. All four root causes repaired	2700	PASS	PASS	PASS	PASS	PASS	Complete

Three observations follow. First, candidate 1 produces exactly the number that the developer wanted to see, and therefore passes the check that a hurried reviewer would make, yet fails every regression test: an explanation can be plausible and a fix can satisfy the reported case while being wrong. Second, candidate 3 is a correct and well-explained repair of one fault, and

it still leaves the reported failure alive, because the assistant (like the developer) was told about one symptom while the code held four faults. Third, only a suite written from the specification, with boundary values, can judge the repair. The checks to apply to any assistant output are therefore: (1) does the explanation match the specification and the trace of the failing run; (2) does the change remove the *cause* rather than the *case*; (3) does the change leave the neighbours of the boundary (499, 501, 100, 101, 300, 301) correct; (4) does the full regression suite pass; (5) does the change introduce validation that rejects legal input such as zero units. An assistant is a useful generator of hypotheses and of tests, but it does not replace the oracle.

1.10 Observation against prediction, summary and recommendation

Table 1.11: Techniques as rows and measures as columns

Technique	Faults located (of 4)	Manual runs	Automated runs / steps	Time (modelled)	Repeatability	Regression test enforced	Prediction confirmed?
A. Print and log	4	9	none	53 min	Low	No	Yes: slowest, no evidence
B. Interactive debugger	4	4 sessions	31 steps	34.3 min	Medium	No	Yes: medium time; very fast for F3
C. Systematic	4 (after differential probes)	0	67 runs	19 min	High	Yes	Partly: fastest, but absolute probes bisected to the wrong commit for F1 and F4

The observation agrees with the prediction on the ranking and disagrees on one point: the prediction did not foresee that the systematic method could mislead when the probe is not isolated. That correction strengthens rather than weakens the recommendation, because the cause of the mistake is visible in the records of approach C, whereas a wrong conclusion reached by print statements leaves no trace. **Recommendation:** adopt approach C as the required procedure for every failed test in billing and payment, use the debugger and logging as tools within it, and write the differential probe and the regression test before the fix is merged. The statement "find and fix it quickly" is replaced by the checkable rule given in Section 1.8.

2. Test Planning and Test Plan Components: Compare Test Plan Structures [7 Marks]

2.1 Context and objective

The Chief Engineer (IT) wants a test plan that he can approve in one meeting. The audit section expects a complete plan. These two wishes pull in opposite directions, and the pilot shows what happens when neither is satisfied: nobody could say how much testing had been done, the wrong build was released, and a billing rule failed at a boundary that a plan with explicit exit criteria would have forced someone to test. A test plan is the document in which the test manager records *what* will be tested, *how*, *by whom*, *when*, with *what resources* and *under what risks*, and *how everyone will know the testing is finished*. This task prepares three plans of different structure for PowerPay, estimates the effort, scores each plan against the components of an IEEE 829-style test plan, analyses risk and change, and justifies one plan for approval.

The planning data given in the assignment are: ten weeks; thirteen people (one test manager, four testers, five developers, one build engineer, one business analyst, one billing-section coordinator); one test server; a tool budget of Rs 1,50,000; six modules; about 180 functional test conditions; and the pilot figures (52 defects found in testing, 21 found by consumers and staff of which 6 were critical, an average fix time of 4 days and an 18% reopen rate). The test levels used in all three plans are unit and integration testing by the developers with the build engineer, system testing (functional, performance and security) by the testers, and acceptance testing by the business analyst and the billing-section coordinator with a tester.

2.2 Prediction recorded before the exercise

Table 2.1: Prediction for Task 2

Plan	Predicted pages / preparation effort	Predicted completeness against IEEE 829 (16 components)	Predicted behaviour under the week-6 tariff change
A. Iteration-level agile plans	Shortest, about 8 to 10 pages in all; quickest to approve	About half the components present; weak on suspension criteria, approvals and staffing	Cheapest to change: only the remaining iteration plans are rewritten
B. Complete IEEE 829-style plan	Longest, about 20 pages; slowest to approve	Full coverage	Most expensive: sections and approvals must be reissued
C. Risk-based plan with test-level breakdown	Medium, about 10 to 12 pages	Between A and B; strong on approach and exit criteria, weak on administration	Cheap to change: the tariff risk is re-scored and its row updated

2.3 Assumptions on which the plans rest

The question permits the SRS excerpt to be written by the student, and requires every assumption to be stated. The main assumptions are listed here; the full register is in Appendix A.

- **Requirements.** Fifteen numbered requirements (SRS-01 to SRS-15, Appendix A) cover registration, login, meter reading, tariff slabs, bill generation, payment, duplicate prevention, receipt, SMS, complaints, outages, load, audit log and tariff control.
- **Schedule.** Ten calendar weeks; three builds from the developers at the start of weeks 3, 5 and 8; three test cycles; a release decision in week 10. Week 1 is planning and week 2 is test design and environment preparation.
- **Productive hours.** Each tester gives 30 productive hours per week, as the question states, i.e. 4 x 10 x 30 = 1,200 hours of tester capacity.
- **Environment.** One shared test server that is not available during deployment windows (assumed 4 hours per build) and is the single point of failure of the plan.
- **Tariff.** Telescopic slabs (Task 1, assumption A1). The regulator revises the tariff slabs in week 6 (given in the question).
- **Concurrency.** The SRS excerpt requires 2,000 concurrent users and a 3-second page response for 3 lakh consumers (assumption A4); the value is not given in the question and must be confirmed.
- **Tool budget.** Rs 1,50,000 is allocated as proposed in Table 2.2.

Table 2.2: Proposed allocation of the tool budget (assumption)

Item	Amount (Rs)	Purpose
Load-generation capacity (cloud or licensed tool)	60,000	SRS-13 and the smart-meter load later in Task 9
Test environment top-up (second environment for load runs)	40,000	Reduces risk R1 (single test server)
Test-data generation and masking utilities	20,000	Realistic but non-personal data
Test-management tool hosting, training and scripts	20,000	Test cases, logs, incident reports, dashboard (Task 4)
Contingency	10,000	Unplanned licences
Total	**1,50,000**	

2.4 The three plans for PowerPay

2.4.1 Plan A: release strategy and iteration-level plans

Plan A consists of a short release test strategy and a one-page plan for each two-week iteration. The strategy states the test levels, the three-cycle approach, the tools and the rule for releasing; the iteration plans state scope, risks and exit conditions for the iteration only. The strategy and the plan of the first iteration are given in full below; the other four iterations follow the same one-page format.

Table 2.3: Plan A: release test strategy (summary)

Element	Content
Objective	Show that PowerPay computes bills correctly, records each payment exactly once, can be released to 3 lakh consumers, and that the evidence of testing is available to auditors
Levels	Unit and integration (developers, build engineer); system (4 testers); acceptance (analyst, billing coordinator, one tester)
Approach	Requirement-based test design with equivalence partitioning and boundary-value analysis for billing, state-based tests for payment, load tests for SRS-13; three cycles on three builds
Release rule	No release with an open Critical defect; all billing and payment conditions executed in cycle 3
Reporting	Weekly status; summary report at the end of the system and acceptance levels

Table 2.4: Plan A: one-page plan for iteration 1 (weeks 1 and 2)

Element	Content
Scope	Plan review; test design for SRS-01 to SRS-04 (registration, login, meter reading, consumption); environment and masked test data prepared
Risks	R1 environment unavailable; R5 ambiguity of requirements (slab interpretation)
Exit conditions	Test design reviewed by the analyst; 100% of SRS-01 to SRS-04 conditions designed; environment passes a smoke check
People	Test manager (plan), 2 testers (design), build engineer (environment)

Table 2.5: Plan A: scope of the five iterations

Iteration	Weeks	Scope	Exit conditions (summary)
1	1-2	Plan, design for registration, login, meter reading; environment	Design reviewed; smoke check passed
2	3-4	Build 1: execution cycle 1 for registration, login, meter reading, bill generation	All planned cases executed; no open Critical in these features
3	5-6	Build 2: payment, receipt, SMS; tariff change in week 6	Payment cases executed; duplicate-payment test passed
4	7-8	Complaints, outages, load and security tests; Build 3	Load target met; no open Major in billing or payment
5	9-10	Cycle 3, regression, acceptance, summary report, release decision	Exit criteria of the strategy met

2.4.2 Plan B: complete IEEE 829-style test plan

Plan B contains all the components that the question lists. Table 2.6 gives the content for PowerPay in the order of the standard.

Table 2.6: Plan B: components of the IEEE 829-style test plan for PowerPay

No.	Component	Content for PowerPay
1	Test plan identifier	PP-TP-001, version 1.0, baseline BL-0.0 (plan baselined before test design); changes recorded in a change log
2	Introduction	Purpose, scope (six modules, 15 requirements), references (SRS excerpt, pilot report, policy of Task 5), and the problems of the pilot year that the plan must prevent
3	Test items	Items under test: build 1042, 1043 and later builds as numbered by the build engineer; tariff file `tariff_v2.2.csv`; configuration files; database scripts; user documents. Identified by baseline and release note (Task 3)
4	Features to be tested	SRS-01 to SRS-15 (registration and login; meter reading and consumption; slab tariff and bill; payment, duplicate prevention, receipt, SMS; complaints and outages; load; audit; tariff control)
5	Features not to be tested	Internals of the bank, UPI switch and SMS operator networks (vendor-certified; only the interface contract is tested); back-office reports outside the SRS excerpt; data migration of historical bills; native mobile applications (the portal is web only)
6	Approach	Requirement-based design with equivalence partitioning and boundary-value analysis for billing and meter reading; state-transition tests for payment status; decision tables for payment amount rules; load tests; role-based security tests; three cycles with regression; risk-based ordering inside each cycle
7	Item pass/fail criteria	An item passes when all High-risk conditions are executed in the latest cycle and pass, no Critical or Major defect is open in billing or payment, at least 95% of all conditions pass, and load target SRS-13 is met
8	Suspension criteria and resumption requirements	Suspend a feature when a blocking defect prevents 30% or more of its cases, when the environment is unavailable for more than 4 hours, when the smoke test of a build fails 2 or more of 10 cases, or when the deployed tariff version differs from the baseline. Resume when the smoke test passes 10 of 10 on a verified baseline and the blocking defect is retested
9	Test deliverables	Test plan; test design specifications; test case specifications; test procedures; test item transmittal reports; test logs; test incident reports; test summary reports; traceability matrix
10	Testing tasks	Plan review; design; environment and data set-up; smoke test of each build; execution of cycles 1 to 3; regression; load and security tests; incident reporting and retest; summary report
11	Environmental needs	One test server (hardware, OS, database), tariff file under version control, payment-gateway sandbox, SMS gateway stub, load generator, masked test data, test-management tool
12	Responsibilities	Test manager: plan, report, release recommendation. Testers: design, execution, incident reports. Developers: unit tests, fixes, debugging. Build engineer: builds, baselines, deployment. Analyst: requirement clarification. Billing coordinator: tariff rules and acceptance
13	Staffing and training needs	4 testers at 30 h per week; 2 days of training on the SRS and tariff rules; 1 day on the test-management tool; boundary-value workshop for the two fresh graduates (Task 7)

No.	Component	Content for PowerPay
14	Schedule	Weeks 1-2 plan, design, environment; week 3 build 1; week 5 build 2; week 8 build 3; weeks 9-10 regression, acceptance, report; release decision at the end of week 10
15	Risks and contingencies	Ten risks scored by probability and impact (Section 2.7) with contingencies for the highest
16	Approvals	Test manager (author), Chief Engineer (IT), billing-section coordinator, audit section; re-approval required for changes in scope, criteria or schedule

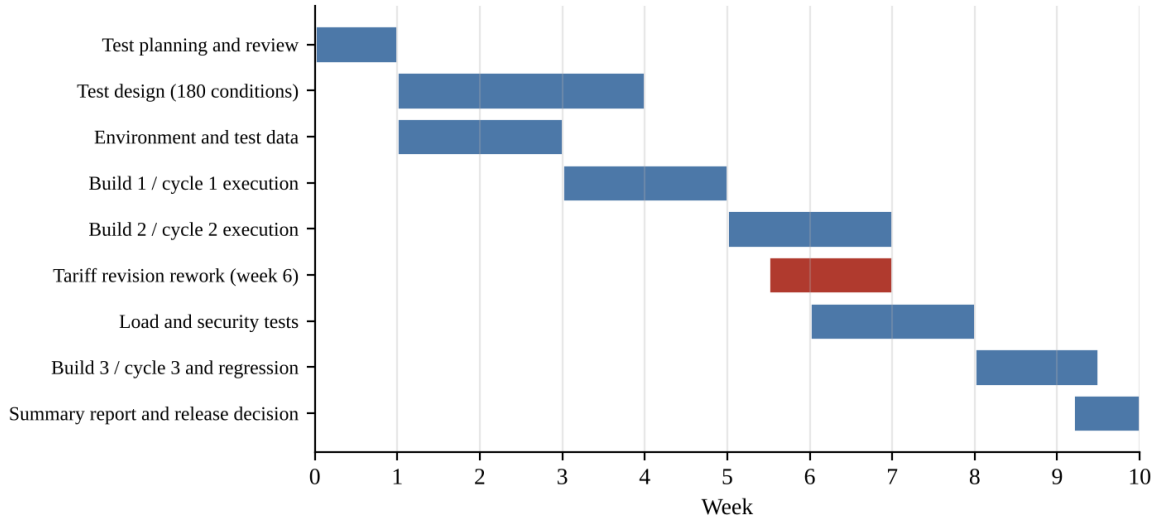


Figure 2.1: Ten-week schedule of the PowerPay test effort (the red bar is the tariff-revision rework)

2.4.3 Plan C: risk-based plan with test-level breakdown

Plan C is organised by product risk. Each risk is mapped to a test level, a technique, an effort and an exit criterion, so that the plan reads as the answer to the question "what could go wrong, and how much testing is spent on it?". The efforts total 269.1 hours, the same total as the formula in Section 2.5, allocated in proportion to the risk.

Table 2.7: Plan C: product risks mapped to level, technique, effort and exit criterion

Product risk	Level	Technique	Effort (h)	Exit criterion
Wrong bill at slab boundaries (the pilot's 500-unit failure)	System	Boundary-value analysis, equivalence partitioning, differential oracle	70	All boundary cases pass; 0 open Critical/Major in billing
Payment recorded twice or lost	Integration, system	State-transition and concurrency tests with repeated callbacks	55	Duplicate-callback test passes; reconciliation of paid amounts is exact
Wrong build or tariff file tested	Process	Baseline check, manifest, smoke test	15	100% of test logs carry a baseline identifier
Receipt or SMS not delivered	Integration	Interface tests with gateway stub	25	SMS within 60 s in 95% of cases (stub)
Slow or failing portal at peak	System	Load test, stress test	35	2,000 users, 3 s response (SRS-13)

Product risk	Level	Technique	Effort (h)	Exit criterion
Unauthorised access or tariff change	System	Role-based and negative security tests	20	No unauthorised role can change a tariff or view another account
Complaint and outage handling errors	System	Equivalence classes, workflow tests	25	Ticket created and updated for every category
Test management and reporting overhead	Management	Plan, report, review	24.1	Reports issued on schedule

2.5 Effort estimate and feasibility

The estimate follows the unit times given in the question. For about 180 test conditions: design takes 0.4 hour per condition; execution takes 0.25 hour per condition per cycle for 3 cycles; retest takes 20% of the execution effort; and management overhead is 15%. The ambiguity of "20% retest effort" is resolved as 20% of the *execution* effort, which is the only effort that is repeated when a defect is fixed (Assumption A6); the alternative reading is shown as a sensitivity check.

Table 2.8: Test effort estimate for about 180 test conditions

Item	Calculation	Person-hours
Test design	180 x 0.4	72.0
Test execution	180 x 0.25 x 3 cycles	135.0
Retest	20% of 135.0	27.0
Subtotal	72.0 + 135.0 + 27.0	234.0
Management overhead	15% of 234.0	35.1
Total	234.0 + 35.1	**269.1**

Feasibility. The capacity of the team is 4 testers x 10 weeks x 30 hours = 1,200 hours. The required effort of 269.1 hours is 269.1 / 1,200 = 22.4% of the capacity, so the plan is feasible with a margin of 930.9 hours. The sensitivity reading, in which the 20% retest is applied to design and execution together, gives (72 + 135 + 0.2 x 207) x 1.15 = 285.7 hours, which is 23.8% of the capacity and does not change the conclusion.

A margin of three quarters is too comfortable to be believed, and it is a warning that the formula leaves out work that the team will certainly have to do. Four further items, estimated by assumption, are shown in Table 2.9. With them the baseline rises to 489.1 hours, which is 40.8% of capacity. The remaining 59% is the contingency that pays for the risks of Section 2.7: a late build costs a week of idle testing, a tariff revision costs rework, and absence of a tester removes 30 hours per week.

Table 2.9: Full effort baseline including activities not in the formula (assumptions)

Item	Hours	Basis
Formula estimate (Table 2.8)	269.1	Question's unit times
Test environment and data preparation	50	Masked data for 3 lakh consumers, tariff file, gateway sandbox
Load and performance test design and runs	70	Scripts, three runs, analysis
Defect reporting, triage, verification	60	About 80 defects x 0.75 h
Plan writing, reviews and walkthroughs	40	Plan review, design review
Total	**489.1**	40.8% of 1,200 h

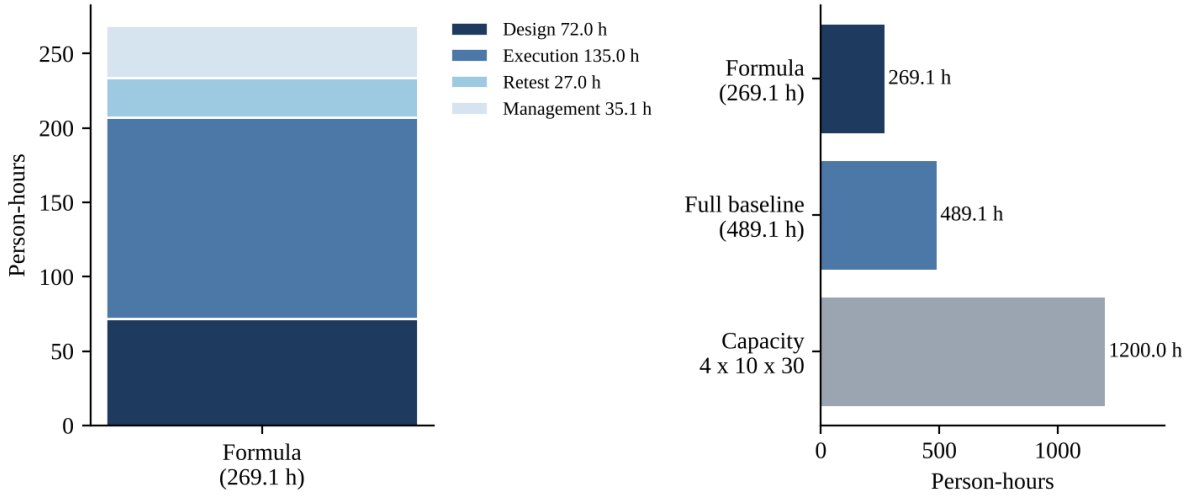


Figure 2.2: Effort by activity (left) and against the capacity of four testers over ten weeks (right)

2.6 Scoring the plans against the IEEE 829 checklist

Each of the 16 components was scored for each plan as present (1), partial (0.5) or missing (0). *Partial* means that the information exists but is incomplete or implicit, for example exit conditions that apply to one iteration only.

Table 2.10: Checklist of IEEE 829 test plan components (P = present, ~ = partial, M = missing)

No.	Component	A. Agile	B. IEEE 829	C. Risk-based
1	Test plan identifier	~	P	~
2	Introduction	P	P	P
3	Test items	~	P	~
4	Features to be tested	P	P	P
5	Features not to be tested	~	P	P
6	Approach	P	P	P
7	Item pass/fail criteria	~	P	P
8	Suspension and resumption criteria	M	P	M
9	Test deliverables	M	P	~

No.	Component	A. Agile	B. IEEE 829	C. Risk-based
10	Testing tasks	~	P	~
11	Environmental needs	~	P	~
12	Responsibilities	~	P	~
13	Staffing and training	M	P	M
14	Schedule	P	P	~
15	Risks and contingencies	~	P	P
16	Approvals	M	P	M
	Score (of 16)	**8.0 (50.0%)**	**16.0 (100%)**	**9.5 (59.4%)**

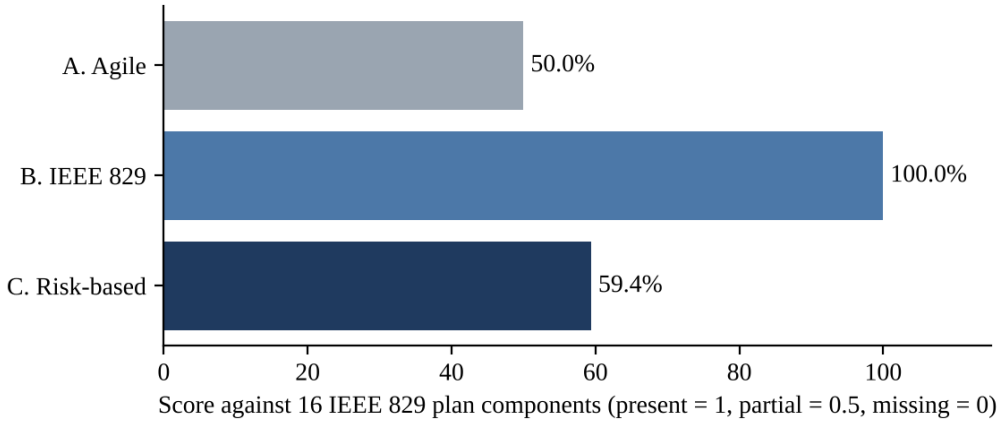


Figure 2.3: Checklist scores of the three plans

2.6.1 Consequences of missing components for the pilot-year problems

Table 2.11: Missing components of Plans A and C and the pilot-year problem each would have allowed

Missing or partial component	Plans	Consequence for the pilot-year problem
Test items (partial)	A, C	The tested build differs from the released build: without an identified item and baseline no one can detect it
Item pass/fail criteria (partial in A)	A	A wrong bill at 500 units can be accepted because the criterion applies to a single iteration, not to the release
Suspension and resumption criteria (missing)	A, C	Testing continues on a broken build, wasting effort and producing results that are later discarded; no rule says when to stop
Deliverables (missing in A)	A	No one can say how much testing was done, because the logs and reports are not committed deliverables
Staffing and training (missing)	A, C	The two fresh graduates are not trained in boundary analysis; a boundary fault in a slab can be missed (Task 7)
Approvals (missing)	A, C	The plan is not binding; scope and criteria can change without anyone accepting the risk, which the audit section cannot accept
Features not to be tested (partial in A)	A	Untested areas are not declared, so the Chief Engineer assumes they were tested

2.7 Risk analysis

Ten risks were identified. Probability and impact were scored from 1 (low) to 5 (high) and the exposure is their product, a standard simple measure of risk. Probabilities are the judgement of the author informed by the pilot (for example, an average fix time of 4 days makes slow fixes likely); they are not statistics and should be reviewed by the test manager.

Table 2.12: Risk register with probability, impact and exposure

ID	Risk	Probability	Impact	Exposure
R1	Test environment unavailable (one shared test server)	4	4	16
R2	Builds delivered late by the developers	4	4	16
R3	Billing rule changed by the regulator during the project	3	5	15
R6	Wrong build or tariff file tested (as in the pilot)	3	5	15
R5	Requirements ambiguous (for example the slab interpretation)	3	4	12
R7	Payment-gateway sandbox unavailable or unstable	3	4	12
R10	Defect fixes slow (4-day average) and reopened (18%)	4	3	12
R4	Absence of a tester	3	3	9
R8	Test data not realistic or not masked	3	3	9
R9	Load infrastructure insufficient for the target	2	4	8

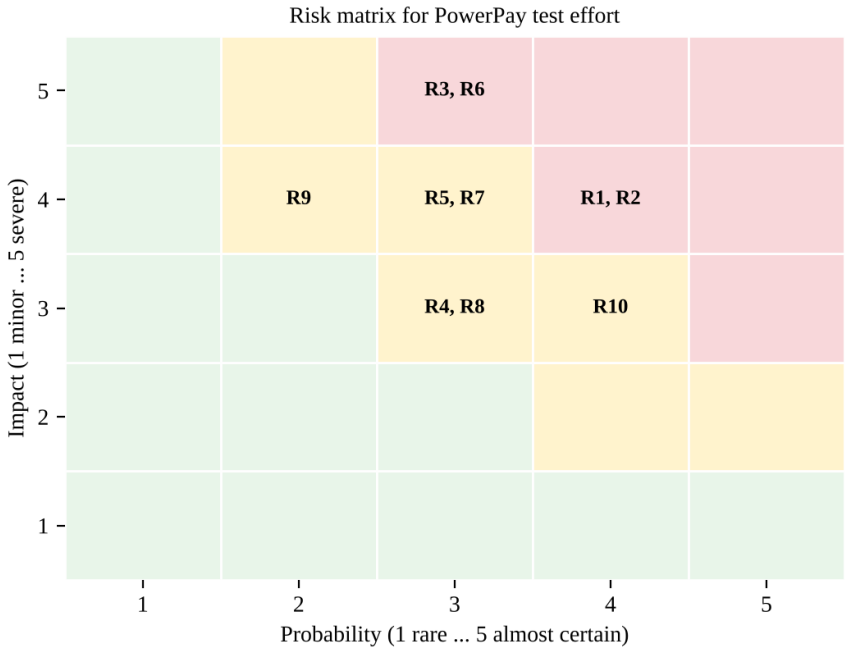


Figure 2.4: Probability and impact of the ten risks

2.7.1 Contingencies for the highest risks

Table 2.13: Contingency plans for the three highest risks and for R6

Risk	Trigger	Contingency	Owner
R1 Environment unavailable (16)	Environment down for more than 4 hours	Use the Rs 40,000 budget line for a second environment; schedule load runs on it; keep smoke test automated; run manual design reviews during outages	Build engineer
R2 Late builds (16)	Build not delivered 1 working day after the plan date	Shift test design and review into the gap; compress cycle 1 by executing High-risk conditions first; escalate to the Chief Engineer after 2 days; shorten cycle 3 only for Low-risk conditions	Test manager
R3 Tariff change (15)	Regulator's notice received	Freeze the tariff-related conditions; revise the tariff file under version control; re-score risks; re-run all billing conditions (Section 2.8)	Billing coordinator and test manager
R6 Wrong build tested (15)	Checksum of deployed items differs from the manifest	Stop testing, redeploy from the baseline, repeat the smoke test, discard results obtained on the wrong build	Build engineer

2.8 Handling a mid-project change: the tariff revised in week 6

The regulator revises the tariff slabs in week 6, when build 2 is being tested. About 40 of the 180 conditions concern the tariff (assumption). They must be redesigned (0.2 hour each for expected values) and re-executed in the remaining cycles: 40 x 0.2 = 8 hours of redesign and 40 x 0.25 x 2 = 20 hours of re-execution, 28 hours in all (modelled) irrespective of the plan. What differs between the plans is what has to be rewritten and re-approved.

Table 2.14: Effect of the week-6 tariff change on each plan

Plan	What must be revised	What must be re-approved	Replanning effort (modelled)
A. Agile	Iteration plans 3 to 5 (scope, exit conditions); strategy unchanged	Revised iteration plans by the Chief Engineer at the next iteration review	About 2 h
B. IEEE 829	Items (new tariff file version), features, approach, criteria, schedule, risks; change log	The whole plan by all four approvers	About 10 h and one approval cycle
C. Risk-based	Re-score the tariff risk; update the billing row (effort, exit criterion) and the effort total	The risk table and the billing row	About 5 h

Plan B is the safest but the slowest to change; plan A is the fastest but does not record what the change did to the release criteria; plan C changes in a controlled way. The ideal is a plan B

whose volatile sections (schedule, risks and the tariff-dependent features) are kept short and separately versioned, which is the structure recommended below.

2.9 Evaluation and justification of the plan for approval

The criteria are those of the Chief Engineer (approval in one meeting), the audit section (completeness), the developers (workable detail) and the test manager (ability to change). Table 2.15 scores the plans from 1 to 5 using the measures of this task.

Table 2.15: Evaluation of the three plans (1 = poor, 5 = good)

Criterion	A. Agile	B. IEEE 829	C. Risk-based
Completeness against IEEE 829 (Table 2.10)	2	5	3
Approvable in one meeting	5	2	4
Audit evidence of what is covered	2	5	4
Ease of change (week-6 tariff revision)	5	2	4
Visibility of risk and effort allocation	3	3	5
Preparation effort (modelled: A 16 h, B 52 h, C 28 h)	5	2	4
Total (of 30)	**22**	**19**	**24**

Decision. The plan submitted for approval is a **hybrid with Plan C as the core**: the risk-based body (Table 2.7) is the plan that the Chief Engineer reads in the meeting, and the five administrative components that Plan C lacks (suspension and resumption criteria, deliverables, staffing and training, approvals, identifier) are copied from Plan B as a two-page annex. The result scores 13.5 of 16 on the checklist, or 84.4% (the five components that remain partial are test items, tasks, environment, responsibilities and schedule, which are given by reference to the identification of Task 3, the schedule of Figure 2.1 and the budget of Table 2.2) and remains short enough for one meeting. Plan A is rejected as the main plan because it fails the audit test; it remains in use *inside* the plan as the iteration reviews, which cost one page each. Plan B in full is rejected because its cost of change (10 hours and four signatures) is out of proportion for a project in which a tariff change is a known event.

2.9.1 Defence of the features not to be tested

The list of features not to be tested is a risk decision and must be defended. (1) *Internals of the bank, UPI and SMS networks* are not tested because the Corporation does not own them and cannot fix them; the interface contract (request, response, callback, timeout) is tested, which covers the pilot failure of a duplicated payment. The risk accepted is a defect inside the gateway; it is mitigated by the reconciliation of paid amounts. (2) *Back-office reports outside the SRS excerpt* are not tested because they have no requirement against which expected results

can be written. (3) *Data migration of historical bills* is outside the scope of the 10-week project as stated and needs its own plan. (4) *Native mobile applications* do not exist in the scope. Each exclusion appears in the plan with the sentence "not tested, risk accepted by the Chief Engineer", so that it cannot be mistaken for a feature that was tested and passed.

2.9.2 Defence of the suspension and resumption criteria

The criteria of component 8 are chosen to stop wasted effort without stopping too easily. The threshold of 30% blocked cases for a feature means that a defect which blocks one case in three justifies stopping that feature, while a single failing case does not. The environment threshold of 4 hours equals the assumed deployment window, so a normal deployment does not trigger it. The smoke-test threshold (2 of 10 cases failing) rejects a build that is not worth testing and is cheap to apply. The tariff-version criterion directly encodes the pilot lesson. Resumption requires a clean smoke test on a verified baseline, which prevents the cycle of resuming on a build whose condition nobody has checked.

2.10 Critique of an LLM-generated test plan for PowerPay

A language model can produce a test plan with correct headings in seconds. The risk is that it is plausible and generic. The statements in Table 2.16 are **illustrative**: they are typical of unreviewed output and were written for this report; they are not a transcript. Each is checked against the project data given in the assignment.

Table 2.16: Illustrative LLM plan statements and their check against the project data

Illustrative statement	Check against the data	Finding
"Testing will be completed in 8 weeks by a team of 6 testers"	The project has 4 testers and 10 weeks	Wrong assumption
"Use Selenium and JMeter; budget as required"	The budget is Rs 1,50,000 and must be allocated	Generic, no allocation
"Test the payment gateway thoroughly"	The gateway belongs to a third party; only the interface can be tested	Wrong scope
"Exit when all test cases pass"	Boundary at 500 units, severity and unexecuted cases are not addressed	Incomplete criterion
"Risks: schedule delays, resource constraints"	No probability, impact, contingency; no tariff change, no single environment	Generic risks
"Features: login, billing, payment, complaints"	Outage reporting, load, audit log and tariff control are missing; no SRS numbers	Incomplete list
"Test cycles: 3"	Consistent with the data	Acceptable

Validation before approval. (1) Check every number against the data: people, weeks, budget, modules, conditions. (2) Recompute the effort with the stated formula (Table 2.8). (3) Trace the features to SRS-01 to SRS-15 and look for omissions and inventions. (4) Look for

statements that would be true of any project, delete them, and see what is left. (5) Check that every risk has a probability, impact, contingency and owner. (6) Have the billing-section coordinator confirm the business rules and the analyst confirm the scope. (7) Record who reviewed the plan and when, and only then place it for approval. An LLM draft that survives these checks has been *edited into* a plan; it has not been accepted as one.

2.11 Observation against prediction, summary and recommendation

Table 2.17: Techniques as rows and measures as columns

Plan	Preparation effort (modelled)	IEEE 829 score (of 16)	Week-6 change effort	Evaluation score (of 30)	Prediction confirmed?
A. Agile	16 h	8.0 (50.0%)	2 h	22	Yes
B. IEEE 829	52 h	16.0 (100%)	10 h + approval cycle	19	Yes
C. Risk-based	28 h	9.5 (59.4%)	5 h	24	Yes; score slightly above prediction because exit criteria are explicit

The predictions were confirmed in direction. Plan C scored highest on the evaluation because it is the only structure that shows where the effort goes in proportion to the risk. The recommendation is to approve the risk-based plan with the administrative annex, to keep the iteration reviews of Plan A as the working tool, and to record the effort estimate of 269.1 hours (489.1 hours with the activities outside the formula) and the capacity margin in the plan itself so that the Chief Engineer can see how much contingency he is approving.

3. Test Items Identification and Test Plan Attachments: Compare Identification Approaches [5 Marks]

3.1 Context and objective

The pilot failed partly because the build that was tested was not the build that was released. This is a failure of *identification*: the plan said "test the portal", but nothing said which portal. A **test item** is a software item that is the object of testing: a build, a module, a configuration file, a data table or a document. **Identification** gives each item an unambiguous name, a version and a place in a baseline, so that a test result can always be tied to the exact thing that produced it. The **plan attachments** are the documents that sit under the plan and make it executable: the test design specification (which features, by which approach, with what pass criteria), the test case specifications, the test procedure (how to run the cases) and the item transmittal report (the handover of the item from the developers to the testers, with its identity and known state).

In a billing system the problem is wider than the program code. The bill depends on the program, on the tariff table, on configuration and on database scripts. A correct build with last year's tariff file produces wrong bills, and no amount of correct testing of the code finds it unless the tariff file is itself an identified item. This task prepares the list of test items by three approaches, prepares the plan attachments for the bill-calculation feature, simulates the pilot problem with a wrong tariff file, analyses traceability and judges what an LLM can do for the traceability matrix.

3.2 Prediction recorded before the exercise

Table 3.1: Prediction for Task 3

Approach	Predicted detection of a wrong tariff file	Predicted effort	Predicted weakness
A. Build-number identification	Not detected: the build number does not change	Lowest (minutes per build)	Identifies only the build, not the data and configuration around it
B. Baseline and configuration items	Detected at deployment, before any test is run	Medium (one-time set-up plus one release note per build)	Does not say which requirement a test covers
C. Requirement traceability	Detected only when a test that covers the wrong value is executed, so only after some test cases	Highest (matrix creation and upkeep)	Detects by failure, not by identity; coverage gaps hide faults

3.3 Test items identified by the three approaches

3.3.1 Approach A: build-number identification

Under approach A the test item is the build number and date supplied by the build engineer. For PowerPay the plan lists three entries: build 1042 (week 3), build 1043 (week 5) and build 1044 (week 8). Nothing else is named. The tariff file, the configuration and the database scripts are implied to be part of "the build".

3.3.2 Approach B: baseline and configuration-item identification

Under approach B every item that influences behaviour is a configuration item under version control, a baseline is tagged when the build is released to test, and a release note lists every item and its version. The versions shown are illustrative; the checksums in Table 3.2 for the two items used in the simulation of Section 3.5 are real values produced by that simulation.

Table 3.2: Approach B: configuration items of baseline BL-1.0-RC1 (versions illustrative)

Item ID	Item	Version	Identification	Tested?
M1	Registration and login module	1.0.3	Tag BL-1.0-RC1/M1	Yes
M2	Meter-reading upload module	1.0.2	Tag BL-1.0-RC1/M2	Yes
M3	Bill-generation module	1.1.0	Tag BL-1.0-RC1/M3	Yes
M4	Online-payment module	1.0.5	Tag BL-1.0-RC1/M4	Yes
M5	Complaint and outage module	1.0.1	Tag BL-1.0-RC1/M5	Yes
M6	Receipt and SMS module	1.0.2	Tag BL-1.0-RC1/M6	Yes
CFG-01	Tariff file `tariff_v2.2.csv`	2.2	SHA-256 prefix 62832616942d (measured in the simulation)	Yes
CFG-02	Application configuration	1.0.0	Tag	Yes
DB-01	Database creation and migration scripts	1.0.4	Tag	Yes
BLD-01	Build artefact `app_build_1042.bin`	1042	SHA-256 prefix 87f0e9c4da04 (measured in the simulation)	Yes
DOC-01	SRS excerpt	1.0	Document register	Used as basis
DOC-02	User guide	0.5	Document register	Review only
EXT-01	Payment-gateway sandbox (third party)	vendor	Contract reference	No (interface only)
EXT-02	SMS operator gateway (third party)	vendor	Contract reference	No (interface only)

3.3.3 Approach C: requirement-traceability identification

Under approach C the item is identified by what it is meant to satisfy: each requirement is traced to the features to be tested, the test design specification, the test cases and the build in which they were executed. The identification chain for the billing requirement is SRS-05 (slab tariff) to feature F-BILL (bill generation) to TDS-BILL-01 to TC-BILL-01 to TC-BILL-11 to "executed on build 1042, cycle 1". The full matrix is in Section 3.6.

3.3.4 Items not to be tested

The plan must also say what is *not* tested. For PowerPay: the third-party gateways EXT-01 and EXT-02 (interface only); the operating system and database server software (assumed certified by the infrastructure team); the user guide beyond a review for accuracy of the tariff description; and historical data not loaded by the project. Declaring these items protects the testers from an implicit promise and the Chief Engineer from a false assumption.

3.4 Plan attachments for the bill-calculation feature

3.4.1 Test design specification TDS-BILL-01

Table 3.3: Test design specification for the bill-calculation feature

Field	Content
Identifier	TDS-BILL-01, version 1.0; baseline BL-0.0
Features to be tested	F-BILL: slab-based bill computation from consumption (SRS-04, SRS-05, SRS-06)
Approach refinements	Equivalence partitioning of consumption into the four slabs and the invalid class (negative); boundary-value analysis at 0, 1, 100, 101, 300, 301, 500, 501; one large value (1,000) and one interior value per slab; a differential oracle (an independent reference calculation)
Test case identification	TC-BILL-01 to TC-BILL-11 (listed in Appendix C)
Feature pass/fail criteria	Every case returns the expected bill to the paisa; invalid input is rejected with a message; no case is skipped; a failure of a boundary case is severity Critical
Risks addressed	Pilot failure at 500 units; unclear slab reading (A1); tariff file mismatch (Task 3 simulation)

3.4.2 Four test case specifications

The four cases below are the boundary cases on both sides of the slab limits that matter most for the pilot failure. Each specification follows the IEEE 829 fields: test items, input specification, output specification, environmental needs, special procedural requirements and intercase dependencies.

Table 3.4: Test case specifications TC-BILL-03, TC-BILL-04, TC-BILL-07 and TC-BILL-08

Field	TC-BILL-03	TC-BILL-04	TC-BILL-07	TC-BILL-08
Test item	M3 bill module, tariff CFG-01	M3, CFG-01	M3, CFG-01	M3, CFG-01
Requirement	SRS-05	SRS-05	SRS-05, SRS-06	SRS-05, SRS-06
Input	units = 100	units = 101	units = 500	units = 501
Expected output	Bill Rs 300.00 (100 x 3.00)	Bill Rs 305.00 (300 + 1 x 5.00)	Bill Rs 2,700.00 (300 + 1,000 + 1,400)	Bill Rs 2,709.00 (2,700 + 1 x 9.00)
Purpose	Upper boundary of slab 1	Lower boundary of slab 2	Upper boundary of slab 3 (pilot failure)	Lower boundary of slab 4
Environment	Test server, baseline verified, masked data	same	same	same

Field	TC-BILL-03	TC-BILL-04	TC-BILL-07	TC-BILL-08
Special procedure	Tariff version printed in the log	same	same	same
Dependencies	None	None	None	None

3.4.3 Test procedure TP-BILL-01

1. Verify that the deployed items match the release note of the build under test (item transmittal report ITR-1042-01); stop if any checksum differs.
2. Log in as the test consumer and open the bill-generation page; record the tariff version shown.
3. For each of TC-BILL-01 to TC-BILL-11, enter the consumption stated in the test case.
4. Record the displayed bill, compare with the expected value to the paisa, and record pass or fail with the build and tariff identifiers.
5. For any failure, stop the case, save the evidence (screen and log extract) and raise a test incident report.
6. At the end, record the number of cases executed, passed, failed and blocked in the test log.

3.4.4 Item transmittal report ITR-1042-01

Table 3.5: Item transmittal report for build 1042

Field	Content
Transmitted items	Build 1042 with baseline BL-1.0-RC1, items M1 to M6, CFG-01, CFG-02, DB-01 (Table 3.2)
Location	Release repository, tag BL-1.0-RC1; checksums in `manifest.json`
Status of items	Unit tests passed by the developers (to be evidenced); known issues: none reported
Transmitted by / to	Build engineer / test manager
Approvals	Developers' lead, test manager
Receipt	Verified by recomputing the checksums at deployment (Table 3.6)

3.5 Simulation of the pilot problem: a wrong tariff file in the test environment

The pilot problem was simulated with a script (Appendix B). A release folder holds build 1042 and the correct tariff file `tariff_v2.2.csv`, together with a manifest of SHA-256 checksums. The test environment is then deployed with the *correct build* but **last year's tariff file** under the same file name (rates 2.50, 4.50, 6.50 and 8.50 instead of 3.00, 5.00, 7.00 and 9.00). The

three approaches were then applied to the environment. The script also ran a second scenario in which only the top-slab rate is wrong (8.50 instead of 9.00).

Table 3.6: Detection of the wrong tariff file by each approach (measured)

Approach	Scenario 1: all four slab rates wrong	Scenario 2: only slab 4 wrong (Rs 8.50)
A. Build number	Not detected: build number 1042 matches the plan	Not detected
B. Baseline check at deployment	Detected before any test: checksum of `tariff_v2.2.csv` differs from the manifest	Detected before any test
C. Traceability with execution	Detected by the first test (TC-BILL-01: expected 150.00, actual 125.00)	Detected only by test 5 (TC-BILL-05, 501 units: expected 2,709.00, actual 2,708.50); tests 1 to 4 all pass

```
Task 3 simulation: wrong tariff file (scenario 1, then scenario 2)   (real output)
Released manifest: {'app_build_1042.bin': '87f0e9c4da04', 'tariff_v2.2.csv': '62832616942d'}
A. Build number in test environment: 1042 -> matches plan (1042): True => mismatch NOT detected
B. Baseline check of deployed files against manifest at deployment time: mismatched items -> ['tariff_v2.2.csv']
C. TC-BILL-01 units=50 expected=150.0 actual=125.0 FAIL
C. TC-BILL-02 units=100 expected=300.0 actual=250.0 FAIL
C. TC-BILL-03 units=101 expected=305.0 actual=254.5 FAIL
C. TC-BILL-04 units=500 expected=2700.0 actual=2450.0 FAIL
C. first detecting test is number 1 after executing 1 test cases (the defect is found during execution, with item/vers
-- Scenario 2: only slab 4 wrong (8.50)
A. not detected (build number unchanged)
B. baseline check mismatch -> ['tariff_v2.2.csv']
C. TC-BILL-01 units=50 expected=150.0 actual=150.0 PASS
C. TC-BILL-02 units=100 expected=300.0 actual=300.0 PASS
C. TC-BILL-03 units=101 expected=305.0 actual=305.0 PASS
C. TC-BILL-04 units=500 expected=2700.0 actual=2700.0 PASS
C. TC-BILL-05 units=501 expected=2709.0 actual=2708.5 FAIL
C. TC-BILL-06 units=600 expected=3600.0 actual=3550.0 FAIL
C. first detecting test number: 5
```

Figure 3.1: Console output of the tariff-file simulation, scenarios 1 and 2 (real run)

The result shows the character of each approach. Approach A is blind to anything that is not the build. Approach B finds the problem at the earliest possible moment and with no test at all, because the item is identified by content. Approach C finds the problem only by executing a test whose expected value is sensitive to the wrong rate. In scenario 2 the error lies in the slab that begins at 501 units, and a test suite that stopped at 500 units (the pilot's failing boundary) would have passed every case on a wrong tariff file. Identification by content (B) and detection by traceable tests (C) are therefore complements, not alternatives.

3.6 Traceability coverage of the test cases

The matrix relates the 15 requirements of the SRS excerpt to the 33 test cases prepared for PowerPay (Appendix C). The matrix was built and analysed by script, and Figure 3.2 shows it.

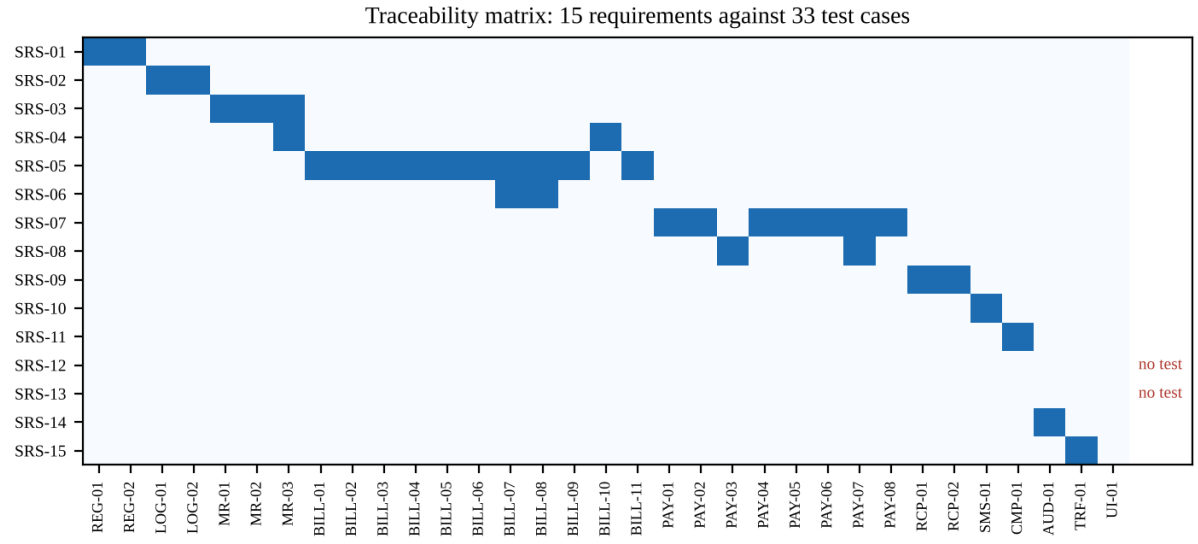


Figure 3.2: Traceability matrix: 15 requirements against 33 test cases

Table 3.7: Requirements and the test cases that cover them

Requirement	Description	Test cases	Number
SRS-01	Consumer registration with 12-digit consumer number and mobile OTP verification	TC-REG-01, TC-REG-02	2
SRS-02	Login with password policy and lock-out after 5 failed attempts	TC-LOG-01, TC-LOG-02	2
SRS-03	Meter-reading upload: integer reading not lower than previous reading	TC-MR-01, TC-MR-02, TC-MR-03	3
SRS-04	Consumption equals current reading minus previous reading; negative consumption rejected	TC-MR-03, TC-BILL-10	2
SRS-05	Slab tariff: 0-100 units Rs 3.00, 101-300 Rs 5.00, 301-500 Rs 7.00, above 500 Rs 9.00 per unit	TC-BILL-01, TC-BILL-02, TC-BILL-03, TC-BILL-04, TC-BILL-05, TC-BILL-06, TC-BILL-07, TC-BILL-08, TC-BILL-09, TC-BILL-11	10
SRS-06	Bill generation with slab-wise breakup, unique bill number and due date	TC-BILL-07, TC-BILL-08	2
SRS-07	Online payment by UPI, card or net banking; amount must equal the outstanding bill	TC-PAY-01, TC-PAY-02, TC-PAY-04, TC-PAY-05, TC-PAY-06, TC-PAY-07, TC-PAY-08	7
SRS-08	A transaction identifier is recorded once; duplicate payments are prevented	TC-PAY-03, TC-PAY-07	2
SRS-09	Payment receipt with unique receipt number, amount and date	TC-RCP-01, TC-RCP-02	2
SRS-10	SMS receipt sent within 60 seconds of a successful payment	TC-SMS-01	1
SRS-11	Complaint registration with category and ticket number	TC-CMP-01	1
SRS-12	Outage reporting by area or feeder with status updates	**none**	0
SRS-13	Load: 2,000 concurrent users with page response within 3 seconds	**none**	0
SRS-14	Audit log of all billing and payment transactions, including failed attempts	TC-AUD-01	1

Requirement	Description	Test cases	Number
SRS-15	Tariff table changes by authorised user only, versioned with effective date	TC-TRF-01	1

Of the 15 requirements, 13 are covered by at least one test case (86.7%) and two have no test: **SRS-12** (outage reporting) and **SRS-13** (load of 2,000 users). Of the 33 test cases, one has no requirement: **TC-UI-01** (download of the bill as a PDF in a dark theme), which traces to no statement in the SRS. There are 36 requirement-to-test links in all. The two uncovered requirements are both in the part of the system that matters at scale: the load requirement is the one that Task 9 will multiply by four. The orphan test is either gold-plating (effort spent on something nobody asked for) or evidence of a missing requirement; the analyst must decide which. The actions are to design tests for SRS-12 and SRS-13 before cycle 1 and to either delete TC-UI-01 or ask the analyst to add the requirement.

3.7 Can an LLM build the traceability matrix?

An LLM can propose links between requirement text and test-case text quickly, and the proposal can be reviewed far faster than a matrix can be built by hand. The danger is that it proposes links by resemblance of words, which gives both **invented links** (a test linked to a requirement it does not verify) and **missing links** (a real link not seen). To measure the effect without claiming the output of a particular model, the matrix was also proposed by a simple keyword baseline: a test is linked to a requirement when their descriptions share two or more content words. **This baseline stands in for an unreviewed LLM and is not an LLM**; the student should repeat the check with a real model's output.

Table 3.8: Keyword baseline against the authoritative matrix (measured)

Measure	Value
Links proposed by the baseline	46
Links in the authoritative matrix	36
Correct links	35
Missing link	1 (TC-PAY-07 to SRS-08)
Links not in the authoritative matrix	11
Precision (correct / proposed)	35 / 46 = 76.1%
Recall (correct / authoritative)	35 / 36 = 97.2%

The 11 extra links include cases that a reviewer would reject, for example TC-CMP-01 (complaint registration) linked to SRS-01 (registration) because both contain the word "registration", and TC-SMS-01 linked to SRS-09 (receipt). Some are defensible (TC-MR-01, meter-reading upload, linked to SRS-04, consumption), which shows why the decision belongs

to a person. The missing link, TC-PAY-07 to SRS-08, is the more dangerous: it is the test that proves the duplicate payment requirement, and without it the matrix would show SRS-08 covered by a single test. The control is a review in two directions: for every proposed link a person confirms that the test would fail if the requirement were not met; for every requirement, a person asks whether the covering tests are enough. The tool is acceptable to *propose* the matrix; the matrix is *accepted* only after this review.

3.8 Comparison of the three approaches

Table 3.9: Comparison of identification approaches

Criterion	A. Build number	B. Baseline and configuration items	C. Requirement traceability
What is identified	The build	All items that affect behaviour, with versions	What each test is for and where it ran
Detects wrong data file at deployment	No	Yes (checksum)	No (only by failing tests)
Detects tested build different from released	Only if numbers are compared	Yes (baseline and manifest)	Only if the build is recorded in the log
Shows coverage gaps	No	No	Yes (two requirements with no test, one orphan test)
One-time effort (modelled)	About 0.25 h per build	8 h set-up, 1 h per build	14 h to build, 2 h per cycle
Total over three builds (modelled)	0.75 h	11 h	20 h
Auditability	Low	High for configuration	High for coverage
Fails when	Any item other than the build changes	Items are not under version control or checksums are not compared	The matrix is out of date or built by resemblance of words

Table 3.10: What happens if each approach is left out

Approach	If it is left out
A. Build number	Nothing is lost if B is in place, because the baseline contains the build number; but with no identifier at all the test log cannot be tied to a version
B. Baseline and configuration items	The pilot failure repeats: a correct build with a wrong tariff file or configuration is tested or released and no one can show which version was tested
C. Traceability	Requirements without tests go unnoticed (SRS-12 and SRS-13 here) and effort is spent on tests that cover nothing (TC-UI-01); the Chief Engineer cannot be told how much of the SRS was tested

3.9 Evaluation: which approach and which attachments are compulsory

The cost of the three approaches is small compared with the cost of one wrong tariff file reaching 3 lakh consumers, and the pilot showed that the tested build differed from the released one. Approach **B is compulsory** for every release: baseline, manifest and release note, with the

checksum comparison done at deployment as the first step of every test procedure. Approach **C is compulsory for billing and payment** (requirements SRS-04 to SRS-09 and SRS-14, SRS-15), where a missed requirement costs money, and is a lighter tool-generated view for the other modules. Approach **A is retained only as a field** of the baseline (the build number).

Among the attachments, the item transmittal report is compulsory for every build (about 1 hour), the test design specification and the test case specifications are compulsory for billing and payment, and the test procedure is compulsory where the steps are not obvious from the case (billing and payment). For the remaining modules a test case in the tool with the requirement ID is sufficient. This keeps the documentation effort at about 11 + 20 = 31 hours for identification plus about 11 hours for the bill-feature attachments (TDS 5 h, four case specifications 2 h, procedure 3 h, transmittal 1 h), which is under 10% of the 489.1-hour baseline of Task 2 and answers the concern that excessive paperwork will make the team ignore the process.

3.10 Observation against prediction, summary and recommendation

Table 3.11: Techniques as rows and measures as columns

Approach	Scenario 1 detection	Scenario 2 detection	Effort over 3 builds (modelled)	Coverage gaps shown	Prediction confirmed?
A. Build number	Not detected	Not detected	0.75 h	None	Yes
B. Baseline and CIs	At deployment (0 tests)	At deployment (0 tests)	11 h	None	Yes
C. Traceability	After 1 test	After 5 tests	20 h	2 requirements without test; 1 orphan test	Yes

All three predictions were confirmed. The unexpected value of approach C was not the detection of the wrong file, which is weaker than B, but the discovery of the two untested requirements and one orphan test by simple counting. **Recommendation:** use B for identity at every deployment, C for the traceability of billing and payment, the transmittal report on every build, and the full attachment set only where money is computed or moved.

4. Test Reporting: Compare Test Reporting Approaches [6 Marks]

4.1 Context and objective

During the pilot nobody could say how much testing had been done. A report is the means by which the results of testing reach the people who must decide. Three groups need different things from it: the Chief Engineer needs to know whether the portal is ready, the auditors need to see evidence that can be checked, and the developers need enough detail to reproduce and fix each failure. The vocabulary is that of the IEEE 829 documents: the **test log** is the chronological record of what was executed and what happened; the **test incident report** describes one event that needs investigation; the **test summary report** summarises the results of a level of testing and evaluates the items against the criteria in the plan.

This task executes 23 test cases of the bill and payment features against a simulated build, prepares the test log and three incident reports, and derives from them the reports of the three approaches: A (log and incident reports only), B (daily dashboard and weekly status report) and C (test summary report with evaluation).

> **Evidence status:** The test cases were executed by script (Appendix B) against a **simulated** PowerPay build whose bill module carries fault F1 of Task 1 and whose payment and audit modules carry two further seeded faults. The results (pass, fail, blocked, expected and actual values) are **measured**. The clock times in the log and the assignment of cases to days are **simulated**, because the script runs in a fraction of a second. No real PowerPay system was tested, and the reproducibility test with a classmate has not yet been carried out (Section 4.7).

4.2 Prediction recorded before the exercise

Table 4.1: Prediction for Task 4

Approach	What it will tell the Chief Engineer	What it will hide	Predicted effort
A. Log and incident reports	Every detail of every failure	Overall progress and readiness; the reader must count	Lowest
B. Dashboard and weekly status	Progress and trend in one view	Meaning of the numbers (which tests failed, how serious)	Medium
C. Summary report with evaluation	A judgement against the criteria, with variances	Day-to-day movement; it arrives too late to steer	Highest per issue, low per week

4.3 Test execution and the test log

Twenty-three test cases were designed for the bill, payment, receipt, SMS and audit requirements (TC-BILL-01 to TC-BILL-11, TC-PAY-01 to TC-PAY-08, TC-RCP-01 and 02, TC-SMS-01 and TC-AUD-01). Eighteen were designed before execution; five (TC-PAY-07, TC-PAY-08, TC-BILL-09, TC-BILL-10 and TC-BILL-11) were added during execution to examine the consequences of failures that had appeared. One case, TC-SMS-01, could not be run because the SMS gateway stub was not provisioned in the exercise and was recorded as **blocked**, not as passed or failed. Figure 4.1 shows the log.

```
Test log of the pilot-build exercise (23 test cases; simulated clock)   (real execution results)
09:00:00 TC-BILL-01 SRS-05/06 PASS    expected=0.0                       actual=0.0
09:00:37 TC-BILL-02 SRS-05/06 PASS    expected=3.0                       actual=3.0
09:01:14 TC-BILL-03 SRS-05/06 PASS    expected=300.0                     actual=300.0
09:01:51 TC-BILL-04 SRS-05/06 PASS    expected=305.0                     actual=305.0
09:02:28 TC-BILL-05 SRS-05/06 PASS    expected=1300.0                    actual=1300.0
09:03:05 TC-BILL-06 SRS-05/06 PASS    expected=1307.0                    actual=1307.0
09:03:42 TC-BILL-07 SRS-05/06 FAIL    expected=2700.0                    actual=2702.0
09:04:19 TC-BILL-08 SRS-05/06 FAIL    expected=2709.0                    actual=2711.0
09:04:56 TC-PAY-01  SRS-07    PASS    expected=OK:RCPT-0001              actual=OK:RCPT-0001
09:05:33 TC-PAY-02  SRS-07    PASS    expected=REJECTED:AMOUNT_MISMATCH  actual=REJECTED:AMOUNT_MISMATCH
09:06:10 TC-PAY-03  SRS-08    FAIL    expected=1                         actual=2
09:06:47 TC-PAY-04  SRS-07    PASS    expected=0.0                       actual=0.0
09:07:24 TC-PAY-05  SRS-07    PASS    expected=REJECTED:AMOUNT_MISMATCH  actual=REJECTED:AMOUNT_MISMATCH
09:08:01 TC-PAY-06  SRS-07    PASS    expected=REJECTED:UNKNOWN_CONSUMER actual=REJECTED:UNKNOWN_CONSUMER
09:08:38 TC-RCP-01  SRS-09    PASS    expected=unique                    actual=unique
09:09:15 TC-RCP-02  SRS-09    PASS    expected=2700.0                    actual=2700.0
09:09:52 TC-AUD-01  SRS-14    FAIL    expected=True                      actual=False
09:11:06 TC-SMS-01  SRS-10    BLOCKED expected=SMS sent                  actual=NOT RUN: gateway stub unavailable
09:11:06 TC-PAY-07  SRS-07    FAIL    expected=1307.0                    actual=2614.0
09:11:43 TC-BILL-09 SRS-05    FAIL    expected=7200.0                    actual=7202.0
09:12:20 TC-BILL-10 SRS-04    PASS    expected=EXC:ValueError            actual=EXC:ValueError
09:12:57 TC-PAY-08  SRS-07    PASS    expected=REJECTED:AMOUNT_MISMATCH  actual=REJECTED:AMOUNT_MISMATCH
09:13:34 TC-BILL-11 SRS-05    FAIL    expected=3600.0                    actual=3602.0
```

Figure 4.1: The test log of the exercise (real results; simulated clock)

Table 4.2: Result counts of the exercise (measured)

Measure	Value
Test cases in the log	23
Executed	22
Passed	15
Failed	7
Blocked	1
Pass rate of executed cases	15 / 22 = 68.2%
Pass rate of all cases (blocked counted as not passed)	15 / 23 = 65.2%
Distinct defects behind the 7 failures	3

The seven failures have only three causes. The slab-limit fault explains four failures (TC-BILL-07, 08, 09 and 11, because every bill from 500 units upward is wrong by Rs 2); the double-recording of a repeated gateway callback explains two (TC-PAY-03 and TC-PAY-07, the second being the effect of the first on the total paid); and the missing audit entries for rejected

payments explain one (TC-AUD-01). A report that counts failed *tests* overstates the number of *defects* by a factor of 7/3, and a report that counts defects hides the number of cases that reveal them. Both figures are needed.

4.4 Three test incident reports

Table 4.3: Test incident report TIR-001

Field	Content
Identifier / date	TIR-001 / day 2, 09:03:42 (simulated clock)
Summary	Bill for exactly 500 units is Rs 2,702.00 instead of Rs 2,700.00
Item / environment	Bill module M3, build 1043, test server, tariff `tariff_v2.2.csv`
Steps to reproduce	(1) Open bill generation for a test consumer. (2) Enter consumption 500. (3) Generate the bill.
Expected result	Rs 2,700.00 (100 x 3.00 + 200 x 5.00 + 200 x 7.00), SRS-05
Actual result	Rs 2,702.00; the 500th unit is charged at Rs 9.00. TC-BILL-08 (501 units) shows 2,711.00 instead of 2,709.00
Impact	Every consumer above 499 units is overcharged by Rs 2 per bill; with the pilot's complaints this is a regulatory risk
Severity / priority	Critical / P1 (money, all consumers above 499 units)
Evidence	Test log lines TC-BILL-07, 08, 09, 11; screenshot to be attached in a real project
Status / owner	Open / developer of module M3

Table 4.4: Test incident report TIR-002

Field	Content
Identifier / date	TIR-002 / day 3, 09:06:10 (simulated clock)
Summary	The same payment callback, received twice, is recorded twice
Item / environment	Payment module M4, build 1043, gateway sandbox, consumer C003 (outstanding Rs 1,307.00)
Steps to reproduce	(1) Make a payment of Rs 1,307.00 for C003. (2) Have the sandbox send the success callback twice at the same moment (a retry). (3) Count the records for the transaction.
Expected result	One record for transaction TXN-3 (SRS-08)
Actual result	Two records; the total paid for C003 is Rs 2,614.00 (TC-PAY-07)
Impact	Consumers are shown as having paid twice; refunds and reconciliation errors
Severity / priority	Critical / P1 (money recorded wrongly)
Evidence	Test log lines TC-PAY-03 and TC-PAY-07
Status / owner	Open / developer of module M4

Table 4.5: Test incident report TIR-003

Field	Content
Identifier / date	TIR-003 / day 4, 09:09:52 (simulated clock)

Field	Content
Summary	A rejected payment attempt leaves no audit-log entry
Item / environment	Payment module M4 and audit log, build 1043
Steps to reproduce	(1) Attempt to pay Rs 1,000.00 for C002 whose outstanding amount is Rs 1,300.00. (2) Observe the rejection. (3) Inspect the audit log.
Expected result	An entry for the failed attempt (SRS-14: all transactions, including failed attempts)
Actual result	No entry; only successful payments are logged
Impact	Failed or fraudulent attempts cannot be traced; the audit requirement is not met
Severity / priority	Major / P2 (no wrong money, but a legal and audit gap)
Evidence	Test log line TC-AUD-01
Status / owner	Open / developer of module M4

4.5 Data and reports for the three approaches

4.5.1 Approach A: log and incident reports only

The report set of approach A is exactly what Figure 4.1 and the incident reports Table 4.3, Table 4.4 and Table 4.5 contain. It is complete and checkable, and for a reader who has the time it answers every factual question. It contains no summary: the reader must count the lines to learn that 15 of 22 executed cases passed.

4.5.2 Approach B: one week of dashboard figures and a weekly status report

The 23 results were distributed over five days (5, 5, 5, 4 and 4 cases, in log order; the allocation of cases to days is illustrative). The dashboard of Figure 4.2 shows the cumulative status and the open defects by severity.

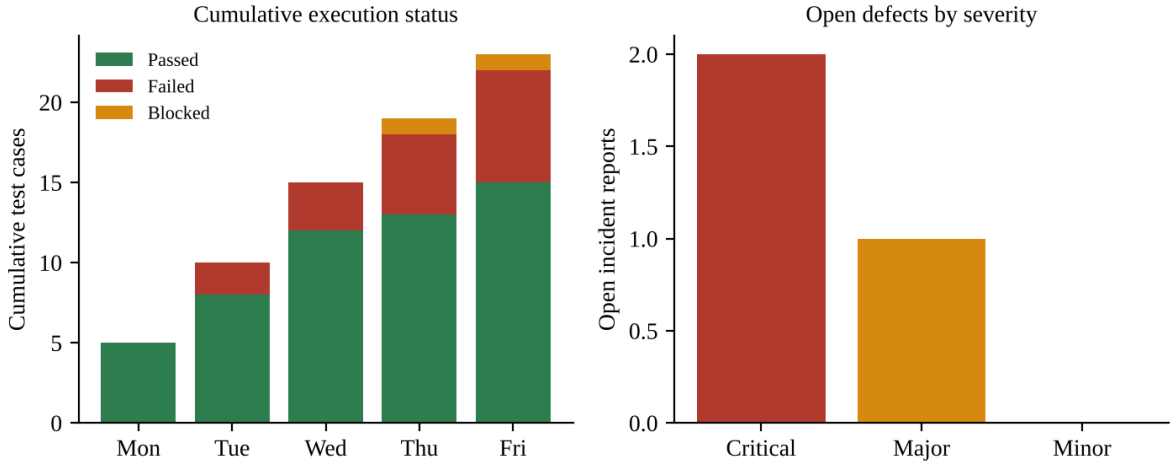


Figure 4.2: Dashboard for the week: cumulative execution status (left) and open defects by severity (right)

Table 4.6: One-page weekly status report (week 3 of the plan)

Item	Content
Period / build	Week 3, build 1043, cycle 1 of the billing and payment features
Progress	23 cases planned for the week; 22 executed, 1 blocked; 15 passed, 7 failed (pass rate of executed cases 68.2%)
Defects	3 open: 2 Critical (TIR-001 wrong bill at 500 units; TIR-002 duplicate payment record), 1 Major (TIR-003 audit gap)
Blockers	SMS gateway stub not available; TC-SMS-01 not executed (risk R7)
Risks	Billing and payment are not stable; the release rule (no open Critical) cannot be met in the current build
Next week	Retest after fixes of TIR-001 to TIR-003; execute TC-SMS-01 when the stub is ready; add boundary cases above 500 units
Decision needed	Chief Engineer: confirm priority of the SMS stub; developers: fix dates for the two Critical defects

4.5.3 Approach C: test summary report for one level (system test of billing and payment)

Table 4.7: Test summary report TSR-SYS-BP-01 (IEEE 829 style)

Section	Content
Identifier	TSR-SYS-BP-01
Summary	System testing of the bill-generation, payment, receipt, SMS and audit features of build 1043 against SRS-04 to SRS-10 and SRS-14
Variances	Eighteen cases planned; five added during execution (TC-PAY-07, TC-PAY-08, TC-BILL-09, 10, 11) to examine failures; TC-SMS-01 not executed (gateway stub unavailable)
Comprehensiveness assessment	Of the 8 requirements in scope, 7 have executed cases (SRS-10 none); boundary values of the slabs were covered up to 501 units; no load, security or concurrency-at-scale tests were run at this level; SRS-12 and SRS-13 have no test cases (Task 3)
Summary of results	22 executed: 15 passed, 7 failed; 3 distinct defects (2 Critical, 1 Major); 1 case blocked
Evaluation	The items do **not** meet the item pass/fail criteria of the plan: two Critical defects are open and billing conditions fail at the 500-unit boundary. The result is valid only for build 1043 and its tariff file
Summary of activities	Execution on the test server using the log of Figure 4.1; incident reports TIR-001 to TIR-003; effort about 9 person-hours (modelled)
Approvals	Test manager (author); Chief Engineer; billing-section coordinator (to be signed)

4.6 What each report tells the Chief Engineer and what it hides

Table 4.8: The question "is the portal ready?" and what each approach can and cannot say

Approach	What it can answer	What it cannot answer	Risk of misreading
A. Log and incident reports	"What exactly failed, and how do I reproduce it?"	"How much is done?" and "Are we ready?" without counting	The reader under-reads: three important failures are buried among 23 lines
B. Dashboard and weekly status	"How far have we got and which way is it moving?"	"Is it correct?" The dashboard shows counts, not meaning	Over-reading of the pass rate
C. Summary report with evaluation	"Does the item meet the criteria? What was not tested?"	"What happened yesterday?" It arrives at the end of the level	Late: a decision by the time it is written may already be forced

4.6.1 The risk of reporting figures without context

The Chief Engineer is likely to read one number: the pass rate. In this exercise it is 68.2%, a number that looks alarming and is. Consider instead an illustrative run of 100 executed cases in which 95 pass (Figure 4.3). A 95% pass rate looks healthy, yet if the five failures include a wrong bill for every consumer above 500 units, the product is not releasable. The pass rate counts tests equally, but defects are not equal: a test of the colour of a button and a test of the bill at 500 units each count one. The pass rate also depends on the test set. A set that has no boundary test above 500 units would report a 100% pass rate on a faulty tariff file (Task 3, scenario 2). A report that gives the rate without the severity of the failures, the coverage of the requirements and the number of blocked and unexecuted cases is a source of false confidence.

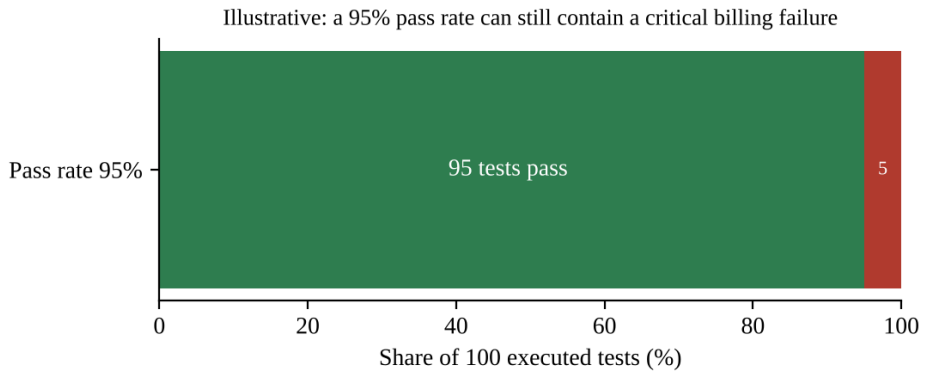


Figure 4.3: Illustrative: a high pass rate can hide a critical failure (constructed example, not measured)

4.7 Quality of the incident reports: reproducibility

The question asks for a classmate to reproduce each failure from the report alone and for the success rate to be recorded. **This session has not been carried out**, because it needs a real person and a real build, and no result is invented here. Table 4.9 is the record sheet to be

completed. The completeness of the three reports against a checklist of what a reader needs was reviewed by the author (the checklist is not a substitute for the classmate session).

Table 4.9: Reproducibility record sheet (to be completed with a classmate) and author's checklist

Report	Steps given	Input values given	Expected and actual given	Build and item given	Evidence cited	Reproduced by classmate? (record Y/N and time)
TIR-001	Yes	Yes (500)	Yes	Yes	Log lines	_____
TIR-002	Yes	Yes (C003, 1,307.00)	Yes	Yes	Log lines	_____
TIR-003	Yes	Yes (C002, 1,000.00)	Yes	Yes	Log lines	_____
Success rate						**___ of 3 = ___%**

The checklist shows that the three reports are complete in the form. The weakness that a classmate is most likely to find is in TIR-002, which describes a concurrent callback: the step "have the sandbox send the callback twice at the same moment" assumes a tool and a procedure that a classmate may not have. If the failure cannot be reproduced from the report, the report must be improved (a script for the repeated callback attached), and a failure that cannot be reproduced cannot be fixed or retested reliably.

4.8 Effort of producing each report

Table 4.10: Modelled effort of reporting (assumptions)

Item	Calculation	Hours
A. Test log	Tool-generated; 0.1 h per day x 50 days	5
A. Incident reports	About 80 defects x 0.5 h	40
Approach A total		**45**
B. Dashboard and weekly status	3 h set-up + 10 weeks x 1.5 h	18
C. Test summary reports	3 levels x 6 h	18
Approaches A + B + C	45 + 18 + 18	**81**

The 81 hours are 16.6% of the 489.1-hour baseline of Task 2. The log and incident reports are produced in any case, so the marginal cost of B and C is 36 hours. The effort to produce a report is not the main cost; the main cost is the decision taken on a wrong or misleading report.

4.9 Evaluation: the reporting set the project should use

The project should use **all three, layered, with different readers**. The test log and incident reports (A) are the evidence for auditors and the working material of developers, and are

generated by the tool as work is done. The dashboard and weekly status report (B) are for the Chief Engineer and the test manager during the project, since they show progress and trend while there is still time to act. The test summary report (C) is the formal evaluation at the end of each level and the document on which the release decision rests and the auditors rely. The dashboard must always show, next to the pass rate, the open defects by severity, the number of blocked and unexecuted cases and the requirement coverage, so that the number is never read alone. Only the combination answers the three audiences: the auditor gets evidence, the Chief Engineer gets a judgement, the developer gets reproducible detail.

4.10 Critique of a test summary report written by an LLM

A language model given the log will write a smooth report. The statements in Table 4.11 are **illustrative** of what unreviewed output can contain and were written for this report; each is compared with the facts of Figure 4.1 and Table 4.2. The student should repeat the exercise with a real model and paste the actual report in Appendix D.

Table 4.11: Illustrative LLM summary statements checked against the log

Illustrative statement	Fact from the log	Finding
"All 23 test cases were executed."	22 executed; TC-SMS-01 blocked	False
"The pass rate was 91%."	15 of 22 = 68.2%	False figure
"All critical functionality passed."	Two Critical defects open (TIR-001, TIR-002)	False and dangerous
"Duplicate payments are prevented."	TC-PAY-03 and TC-PAY-07 failed	False
"SMS receipts are delivered within 60 seconds."	TC-SMS-01 not run	Unsupported
"The test environment was stable throughout."	No stability data in the log	Unsupported
"Three defects were found."	Three distinct defects	Correct
"The portal is ready for release with minor issues."	Criteria not met	Unsupported conclusion

Of the eight statements one is correct, five are false or unsupported claims of fact, and the last is a conclusion that contradicts the evaluation criteria. The review procedure is to check every number against the log, to mark every statement that has no line in the log behind it, and to require that the evaluation sentence be written or confirmed by the test manager, who signs the report.

4.11 Observation against prediction, summary and recommendation

Table 4.12: Techniques as rows and measures as columns

Approach	Tells the Chief Engineer	Hides	Effort (modelled)	Prediction confirmed?
A. Log and incident reports	Every failure and how to reproduce it	Progress and readiness	45 h over the project	Yes
B. Dashboard and weekly status	68.2% pass, 3 open defects, 1 blocked	Which defects matter, unless severity is shown	+18 h	Yes
C. Summary report with evaluation	Criteria not met; variances; coverage	Day-to-day movement	+18 h	Yes

The predictions were confirmed. One result was not foreseen: only 3 defects stood behind 7 failed tests, so each report must say which of the two is counted. **Recommendation:** A as the generated evidence base, B daily with severity and blocked counts, C at the end of each level, and no report issued without the baseline identifier of the build.

5. Test Policy Development: Compare Test Policy Approaches [7 Marks]

5.1 Context and objective

The Corporation has no written testing policy; each project does as it likes. A **test policy** is a short, high-level document, approved by senior management, that states why the organisation tests, what it expects of testing and who is responsible. It sits above the **test strategy** (the general approach used across projects) and the **test plan** (the approach for one project). Without a policy every project invents its own rules, and the pilot year shows the result: the tested build was not the released build, nobody could say how much testing had been done, six critical defects were found by consumers, and 18% of the defects reopened. This task drafts three policies of increasing strength, makes the policy statements measurable, tests the policies against the pilot-year problems and a small and a large project, and compares cost and likelihood of being followed.

5.2 Prediction recorded before the exercise

Table 5.1: Prediction for Task 5

Policy	Predicted prevention of pilot problems	Predicted cost	Predicted likelihood of being followed
A. Informal guidelines	Few: advice without enforcement	Lowest (a few hours)	Low, because nothing checks it
B. Formal organisational policy	Most of them, where a rule can be checked	Medium	Medium to high, because it is signed and measured
C. Policy with strategy and improvement path	Most, and the improvement of the process over time	Highest	Highest if resourced; at risk if the improvement path is not funded

5.3 The three draft policies

5.3.1 Policy A: informal guidelines (one page)

Policy A is a one-page list circulated by e-mail. **Scope:** all software projects of the Corporation. **Objective:** better testing. **Roles:** the project manager decides. **Release rule:** "release when the testing is done and the project manager is satisfied". The ten guidelines are: test against requirements; test boundaries; keep a record of tests; have someone other than the author test critical features; test every build you release; fix defects found; retest after fixing; report defects with steps; keep test data safe; tell the Chief Engineer about known problems. There is no owner, no measure and no consequence.

5.3.2 Policy B: formal organisational test policy

Table 5.2: Policy B: contents of the signed organisational test policy

Section	Content for the Corporation
1. Scope and purpose	All software systems that bill, collect or record money or serve consumers (PowerPay and later systems); a lighter tailoring for internal tools (Section 5.6)
2. Mission of testing	Testing provides the Chief Engineer, the auditors and the consumers with evidence that a system does what the regulations and requirements say, and exposes where it does not, before release
3. Definitions	Failure, fault, error, defect; test item, baseline; severity (Critical, Major, Minor) and priority; test incident; the definitions used in this report
4. Principles	Testing shows the presence of defects, not their absence; early testing; exhaustive testing is impossible, so risk decides; testing is context dependent; defects cluster; independent testing of billing and payment
5. Test levels	Unit and integration (developers), system (testers), acceptance (analyst and billing coordinator), with entry and exit criteria for each
6. Standards and documents	IEEE 829-style plan, test cases, log, incident report and summary report, in the tailoring of Task 8; baseline identification of every tested item
7. Metrics and quality targets	The eight measurable statements of Table 5.4
8. Roles and responsibilities	Chief Engineer owns the policy; test manager enforces it and signs the release recommendation; project managers apply it; testers and developers follow it; audit section reviews compliance yearly
9. Release rule	A system is released to consumers only when the exit criteria of the plan are met, no Critical defect is open, and the Chief Engineer has signed on the test summary report
10. Review	The policy is reviewed every year and after any release that causes a Critical failure in production

5.3.3 Policy C: policy with test strategy and improvement path

Policy C contains Policy B and adds two documents. The **test strategy** states, for every project, the standard approach: the risk-based selection of conditions, the use of boundary-value analysis and equivalence partitioning for numeric rules, the three-cycle execution, the automation of regression tests for billing and payment, and the standard reports (Task 4). The **improvement path** sets out how the Corporation will raise the maturity of its testing process over two years in the steps of the Test Maturity Model integration (TMMi). Level 2 ("Managed") covers test policy and strategy, test planning, test monitoring and control, test design and execution, and the test environment. Level 3 ("Defined") adds the test organisation, a training programme, integration of testing into the life cycle, non-functional testing and peer reviews. The path assigns owners and dates to the Level 2 goals first (months 1 to 12) and to Level 3 later (months 13 to 24).

Table 5.3: Policy C: improvement path (summary)

Period	Goal	Evidence of achievement
Months 1-3	Policy B in force; strategy issued; baseline measurement of the eight metrics	Signed policy; first metric report
Months 4-12	TMMi level 2 areas in use on PowerPay: planning, monitoring, design and execution, test environment	Self-assessment against level 2; audit review
Months 13-24	Level 3 areas: test organisation, training programme, peer reviews, non-functional testing	Self-assessment against level 3; training records

5.4 Measurable policy statements

Table 5.4: Eight measurable policy statements

ID	Statement	How measured	Target	Baseline (pilot)
P1	No release with an open Critical defect	Defect-tool query at the release gate	0 open	6 critical found by users
P2	Every billing and payment requirement is traced to at least one test case before execution starts	Traceability matrix coverage	100%	86.7% in the matrix of Task 3 (13 of 15)
P3	Every payment and billing feature has automated regression tests, run on every build	Automated share of billing and payment conditions; share of builds on which the suite ran	At least 90%; 100% of builds	Not measured
P4	Every test log and report states the baseline identifier and tariff version of the item tested	Audit of 10 logs per release	100%	Tested build differed from released build
P5	Defect Detection Effectiveness of each release is at least 90%	DDE = defects found before release / (before + found in the first 90 days)	At least 90%	52 / 73 = 71.2%
P6	Reopen rate of fixed defects is at most 8%	Reopened / fixed, from the defect tool	At most 8%	18%
P7	Every fixed billing or payment defect has a linked regression test that failed before and passed after the fix	Link check in the defect tool	100%	Not required
P8	No unmasked personal data in any test environment or test report	Automated scan of test data and reports	0 findings	Not measured

The Defect Detection Effectiveness of the pilot is 52 / 73 = 71.2%, so 28.8% of the defects reached consumers. A target of 90% means that of 73 defects at most 7 may escape. The targets are deliberately a few measures that a tool can produce without extra work; policies with thirty indicators are not followed.

5.5 Policies against the pilot-year problems

Table 5.5: Which pilot-year problem each policy would have prevented (Y = yes, P = partly, N = no)

Pilot-year problem	A. Informal	B. Formal	C. Strategy and path
Wrong bill at 500 units reached the pilot	N: "test boundaries" is advice	P: principle and P2 push boundary design; no technique mandated	Y: strategy mandates boundary-value analysis and P7 regression tests
Tested build differed from released build	N	Y: P4 requires baseline and tariff version in every log	Y: same, and the strategy makes the baseline check the first step of every test procedure
Nobody could say how much testing had been done	N	Y: documents and P2, P3 give measures	Y: plus monitoring and control at TMMi level 2
Six critical defects found by consumers	N	Y: P1 stops the release; P5 measures escapes	Y: plus the improvement path reduces escapes over time
18% of defects reopened	N	P: P6 measures it, P7 prevents it for billing and payment	Y: P6, P7 and the strategy's differential probes (Task 1)
Duplicate payment recorded twice	N	P: not named, but P3 requires regression tests	Y: strategy lists concurrency and repeated-callback tests for payment

5.6 Application to a small and to a large project

Table 5.6: Applying the policy to a feedback form and to PowerPay

Policy element	Feedback form (small, no money)	PowerPay (large, money, audit)
Release rule	No open Critical or Major defect; project manager signs	Full rule of the policy; Chief Engineer signs on the summary report
Test levels	Developer testing and one acceptance check	Unit, integration, system, acceptance
Documents	Tool-generated test cases and log; one-page summary	Task 8 hybrid set: full documents for billing and payment
P2 traceability	Not required	Required for billing and payment
P3 automated regression	Optional	Required
P5 DDE target	Reported, no target	At least 90%
P8 masking	Required if personal data are used	Required
Where tailoring is not allowed	Release rule for Critical defects and P8	Release rule, P1, P4, P7, P8

A policy that cannot be tailored will be ignored by small projects and a policy that can be tailored without limit is not a policy. The last row states the limit: the release rule for Critical defects and the protection of personal data apply to every project.

5.7 Cost, effort and likelihood of being followed

Table 5.7: Introduction effort (modelled) and likelihood of being followed (judgement)

Policy	Effort components	Hours	Likelihood followed
A. Informal	One person writes one page and sends it	8	About 25%: no owner, no check
B. Formal	Drafting 24 h, review 16 h, approval 8 h, training 12 h	60	About 65%: signed, measured, audited
C. Strategy and path	B (60 h) + strategy 40 h + improvement path 30 h + maturity gap assessment 24 h	154	About 75% if the path is resourced

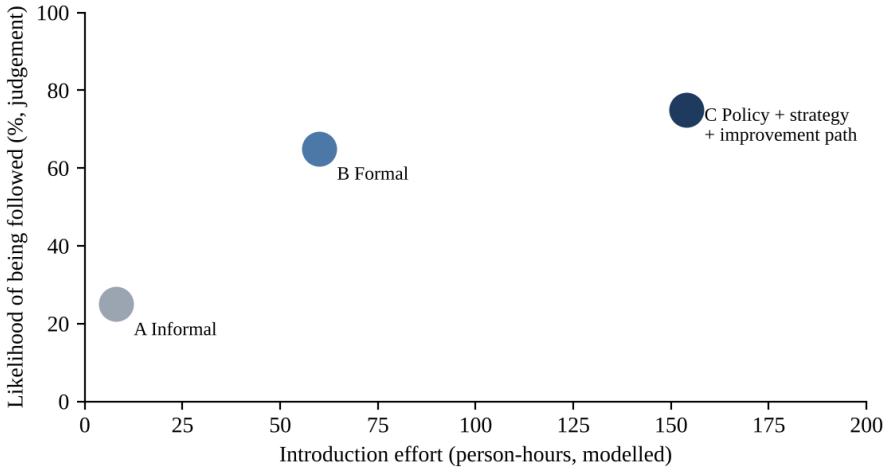


Figure 5.1: Introduction effort and likelihood of being followed (effort modelled; likelihood a judgement)

The likelihoods are the author's judgement from three properties that determine whether a rule is followed: whether somebody owns it, whether it is measured, and whether a consequence follows from it. They are not statistics. Policy A has none of the three. Policy B has all three but depends on the Chief Engineer's attention. Policy C adds the means of improvement but costs 154 hours, which is 2.6 times the cost of B for the further 10 points of likelihood.

5.8 Evaluation: the policy the Corporation should adopt

The Corporation should adopt **Policy B immediately and Policy C as the path from it**: sign Policy B in month 1 (60 hours), issue the strategy for PowerPay in months 2 and 3, and start the improvement path as the eight measures begin to produce data. The reasons follow the three criteria asked for. **Cost:** B costs 60 hours, a small fraction of the 489.1-hour test effort of Task 2, while A, though cheaper, would have prevented none of the problems of Table 5.5. **Measurability:** only B and C give statements that a tool can check (Table 5.4); a policy that cannot be measured cannot be audited. **Audit:** the auditors need a document that says what the organisation requires and a record of compliance; B provides both, and C provides in addition a record of improvement. The full cost of C is not incurred before the Corporation shows with

the baseline measurement that B is being followed, which is the condition under which an improvement path is credible.

5.9 Critique of an LLM-generated test policy

A language model asked to write a test policy for an electricity corporation produces a document that is fluent and generic. The checks below are applied to **illustrative** sentences of the kind such output contains (written for this report, not a transcript); the student should repeat them with a real model's output.

Table 5.8: Illustrative LLM policy statements checked for specificity, measurability and consistency

Illustrative statement	Specific to the Corporation?	Measurable?	Consistent with the definitions of this report?
"Testing will be performed to ensure high quality."	No	No	Not applicable
"All critical defects must be resolved before release."	Partly	Yes, if Critical is defined	Needs the definition of Critical
"Test coverage should be sufficient."	No	No: "sufficient" has no value	No
"Defects will be tracked in a defect-management tool."	No	Partly	Yes
"PowerPay billing will be tested at every slab boundary."	Yes	Yes (boundary list)	Yes
"Testers will report to the project manager."	No	Yes	Conflicts with independence of billing and payment testing

Only one of the six statements is specific, measurable and consistent. The policy is acceptable only after each statement has been rewritten as one of the measurable forms of Table 5.4, the terms have been tied to the definitions of Section 3 of the policy, and the Chief Engineer has signed it. A model is a useful drafter of structure and wording; it cannot know that this Corporation had 6 critical escapes and a DDE of 71.2%, and a policy that does not use these data is not the Corporation's.

5.10 Observation against prediction, summary and recommendation

Table 5.9: Techniques as rows and measures as columns

Policy	Pilot problems prevented (of 6, Y counts 1, P counts 0.5)	Measurable statements	Effort (modelled)	Likelihood followed	Prediction confirmed?
A. Informal	0	0	8 h	About 25%	Yes
B. Formal	4.5 (3 Y, 3 P)	8	60 h	About 65%	Yes

Policy	Pilot problems prevented (of 6, Y counts 1, P counts 0.5)	Measurable statements	Effort (modelled)	Likelihood followed	Prediction confirmed?
C. Strategy and path	6.0	8	154 h	About 75%	Yes; the gain over B lies in the three problems that B only partly prevents

The predictions were confirmed. The step from A to B is the largest in prevention (0 to 4.5 of 6) and costs 52 more hours; the step from B to C adds the last 1.5 for 94 further hours. **Recommendation:** sign Policy B now, issue the PowerPay strategy, and fund the improvement path from the savings that follow from fewer reopened defects (a reopen rate of 8% instead of 18% saves roughly 7 of the 13 reopened defects of the pilot, about 28 developer-days at the 4-day average).

6. Test Team Organisation: Compare Team Structures [4 Marks]

6.1 Context and objective

PowerPay needs an organisation for its four testers and the people who support them. The organisation decides who may stop a release, how quickly a dispute is settled and who covers when someone is absent. A test group has to balance two forces: *independence* (a tester who reports to the same manager as the developer may be pressed to pass the work) and *integration* (a tester who sits far from the developer and the business sees problems late and does not know the rules). The three structures compared here make this trade-off in different ways. The thirteen people available are one test manager, four testers, five developers, one build engineer, one business analyst and one billing-section coordinator; the vendor staff of structure C are outside the thirteen.

6.2 Prediction recorded before the exercise

Table 6.1: Prediction for Task 6

Structure	Independence	Speed of decisions	Main weakness
A. Test-level functional teams	High	Medium (several leads)	Hand-offs between levels; domain knowledge late
B. Module-aligned teams	Medium (testers sit with developers)	Fastest	One tester per module is a single point of failure
C. Vendor with internal acceptance	High (external)	Slowest (contract path)	Dependence on the vendor; largest management overhead

6.3 Organisation charts and assignment of the thirteen people

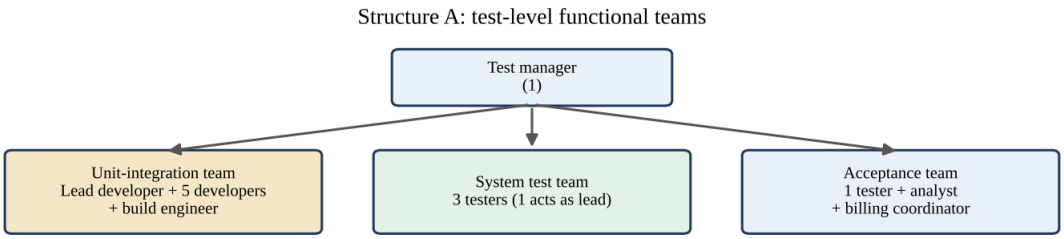


Figure 6.1: Structure A: test-level functional teams

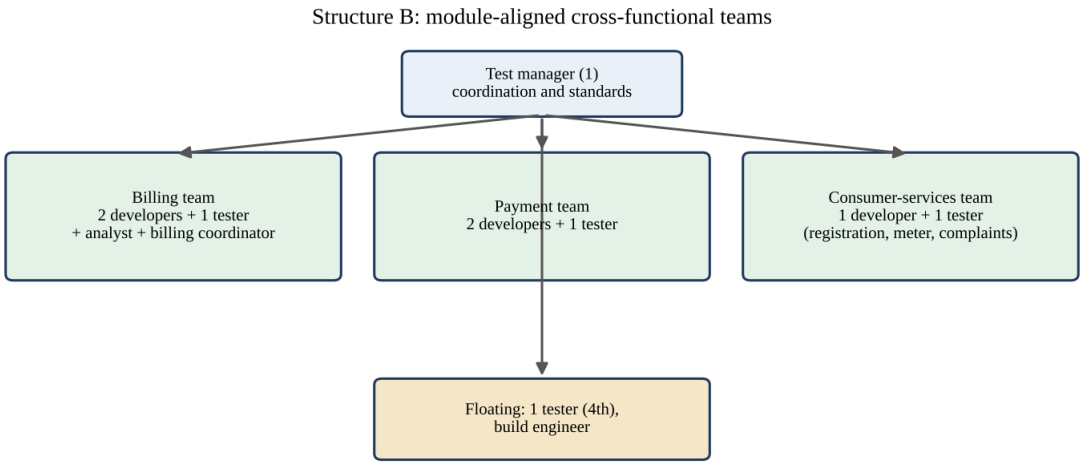


Figure 6.2: Structure B: module-aligned cross-functional teams

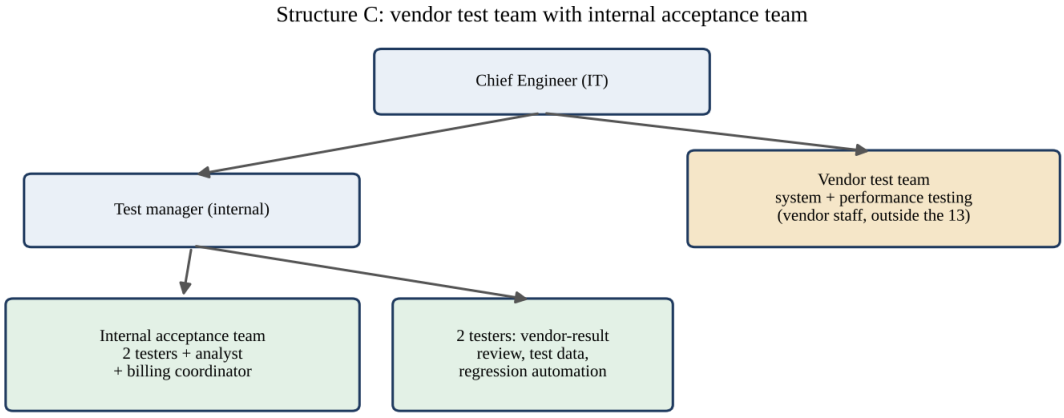


Figure 6.3: Structure C: vendor test team with internal acceptance team

Table 6.2: Assignment of the 13 people in each structure

Role (number)	A. Test levels	B. Modules	C. Vendor
Test manager (1)	Heads all three teams; reports to the Chief Engineer	Coordinates and sets standards; testers keep a line to the test manager	Manages the vendor contract and the internal team
Testers (4)	3 in system test, 1 in acceptance	1 in billing, 1 in payment, 1 in consumer services, 1 floating	2 in acceptance, 1 reviews vendor results, 1 prepares test data and regression automation
Developers (5)	Unit-integration team (a lead and four)	2 billing, 2 payment, 1 consumer services	Remain in the development team; interface to the vendor through the test manager
Build engineer (1)	Unit-integration team	Floating with the 4th tester	Builds and baselines for vendor and internal team
Business analyst (1)	Acceptance team	Billing team (shared with the other two)	Acceptance team

Role (number)	A. Test levels	B. Modules	C. Vendor
Billing coordinator (1)	Acceptance team	Billing team	Acceptance team

Interfaces. In structure A the five developers meet the testers at two hand-offs, from unit-integration to system and from system to acceptance; the billing-section coordinator is met only at the acceptance level. In structure B the developers and the coordinator are members of the team that owns the module, so the interface is daily. In structure C the interface is the contract: the developers deliver builds and the vendor returns incident reports through the test manager, while the coordinator works only with the internal acceptance team.

6.4 Simulation of three situations

Each situation was walked through in each structure by listing the people through whom the decision must pass. A *hop* is one hand-over of the question from one person to another; each hop is assumed to cost 4 working hours (waiting for attention, a short discussion and a reply). The delays are therefore **modelled**, not observed, and the hop counts are the result of the structures as drawn.

Table 6.3: Decision paths and modelled delay in each situation

Situation	A. Test levels	B. Modules	C. Vendor
1. Defect dispute: the tester says a boundary bill is wrong, the developer says it is right	Tester, system-test lead, unit-integration lead, test manager decides (4 hops, 16 h)	Tester and developer in one team; billing coordinator in the team gives the rule; team lead decides (1 hop, 4 h)	Vendor tester, vendor test lead, vendor manager, Corporation liaison, test manager, development lead (6 hops, 24 h)
2. Urgent tariff change	Coordinator informs test manager, who informs system-test lead and acceptance team (3 hops, 12 h)	Coordinator and billing team revise tests in the team; payment and consumer-services teams informed (2 hops, 8 h)	Contract change note to the vendor, quotation, approval (about 5 working days, 40 h)
3. A tester absent for a week	System-test lead reassigns one of the other two testers (2 hops, 8 h; capacity down by one third)	The module has no tester; the floating tester takes over after ramp-up (24 h; module uncovered for 3 days)	Vendor substitutes under its service-level agreement (2 hops, 8 h)

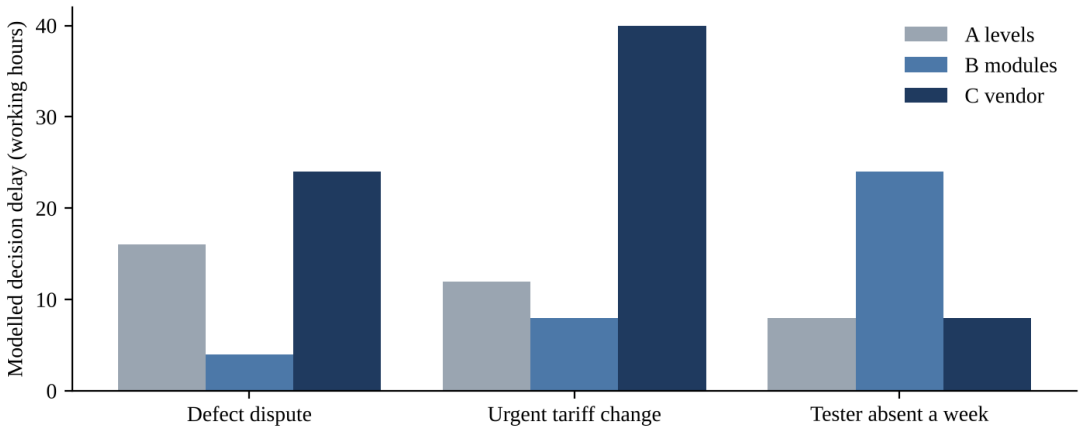


Figure 6.4: Modelled decision delay in the three situations

6.5 Analysis of the structures

Table 6.4: Communication paths, independence, domain knowledge, tools and single points of failure

Criterion	A. Test levels	B. Modules	C. Vendor
Communication paths	Within teams 15 + 3 + 3 = 21 pairs; two formal hand-offs between levels	Within teams 10 + 3 + 1 + 1 = 15 pairs; daily contact with the owner of the rule	Few pairs inside; one formal channel through the test manager to the vendor
Independence of testing	High: testers outside the development team	Medium: testers sit with developers; reporting line to the test manager must be kept	High: the vendor is external
Domain knowledge	Held by acceptance only; system testers learn the rules late	High: coordinator and analyst in the billing team	Low at the start; the vendor must be taught the rules
Tool support	One shared tool owned by the test manager	One shared tool; module teams may drift in use	The vendor's own tools; results must be converted
Single points of failure	Build engineer; test manager	One tester per module; the analyst shared across teams	The vendor; the test manager as liaison

6.5.1 Schedule support and management overhead

The structure that best supports the ten-week schedule is B, because its feedback loops are shortest: the tester raises a defect to a developer who is in the next chair and a coordinator who can say at once what the rule is. This matters because the plan of Task 2 assumes a 4-day average fix time and an 18% reopen rate; shortening the loop attacks both. The structure with the largest management overhead is C, with the contract, the vendor's reporting and the conversion of results, followed by A with its three leads and two formal hand-offs. The vendor structure also starts slowly: onboarding the vendor to the tariff rules takes time that the schedule does not have, unless the vendor team starts before week 3.

6.6 Evaluation: the recommended structure

Recommendation: structure B with three safeguards. (1) *Independence:* each module tester reports to the test manager and not to the module team lead, and the release recommendation is made by the test manager. (2) *Single point of failure:* the floating tester is paired with a module tester each week in rotation, so that every module has two testers who know it; this reduces the absence delay of situation 3 from 24 hours to about 8 hours at a cost of the floating tester's coverage. (3) *Acceptance:* the internal acceptance function of structure C is retained, with the analyst and the billing coordinator defining the acceptance tests for billing, so that the business reads the result independently of the module team. The simulation supports the choice: structure B decides a dispute in 4 hours against 16 and 24, and handles the tariff change in 8 hours against 12 and 40; it loses only on absence, and the safeguard repairs that.

6.7 Critique of giving the vendor the right to approve the release

The proposal should be rejected. First, **responsibility cannot be delegated.** The Corporation answers to consumers, the regulator and the auditors for the bills it issues; a contract can transfer work but not accountability. Second, **incentives conflict.** A vendor paid for completing testing has an interest in a pass, and one whose penalty clauses depend on defects found has an interest in a low count; neither is a neutral judge of release. Third, **the vendor lacks the context.** A release decision balances test evidence against business risk, for example a known minor defect against a regulator's deadline, and the vendor does not hold that information. Fourth, **the vendor's evidence is the one input that must be reviewed.** The internal team in structure C is there to review it; if the vendor also approves, nobody checks the checker. The right arrangement is that the vendor *delivers evidence and a recommendation*, with the right to record a formal objection that goes to the Chief Engineer, and that the *release decision stays with the Chief Engineer on the test manager's recommendation*.

6.8 Observation against prediction, summary and recommendation

Table 6.5: Structures as rows and measures as columns

Structure	Hops: dispute / tariff / absence	Independence	Domain knowledge	Single point of failure	Prediction confirmed?
A. Test levels	4 / 3 / 2	High	Late	Build engineer, test manager	Yes
B. Modules	1 / 2 / 6 (24 h)	Medium, with safeguard	High	One tester per module	Yes
C. Vendor	6 / 10 (40 h) / 2	High	Low at first	Vendor	Yes

The predictions were confirmed. In the absence situation the figures of structure B are in 4-hour hops (6 hops, 24 hours); the tariff change in C is counted by its contract path (40 hours,

equivalent to 10 hops). The recommended organisation is structure B with the safeguards of Section 6.6.

7. Skills of a Test Specialist: Compare Skill-Building Strategies [5 Marks]

7.1 Context and objective

A test specialist needs more than the ability to run test cases. The testers recruited for PowerPay have different backgrounds: two are fresh graduates (T1 and T2), one comes from billing operations (T3) and one is a former developer (T4). Their strengths do not overlap. The billing-operations tester knows the tariff and the consumers' complaints; the former developer can read code and script; the fresh graduates know the theory but have little experience of either. The pilot showed the cost of a gap: a boundary fault in a billing slab was missed, which needs both the technique (boundary-value analysis) and the domain knowledge (that slab 3 ends at 500 units and not at 499). This task builds a skills matrix, plans four weeks of skill building for the fresh graduates under three strategies, and compares skills built, cost and measurable gains.

7.2 Prediction recorded before the exercise

Table 7.1: Prediction for Task 7

Strategy	Skills built best	Skills left undeveloped	Predicted cost
A. Classroom training and certification	Technical vocabulary and techniques	Domain, tools in practice, communication	Highest cash cost
B. Mentoring and pair testing	Domain, practice, report writing	Breadth beyond what the mentor knows	No cash, but senior time
C. Skills matrix with rotation	Breadth: design, automation, analysis, reporting	Depth in any one area in four weeks	Low cash, some lost productivity

7.3 The skills matrix and the scores of the four testers

Table 7.2: Skill categories of a test specialist and the scale used

Category	What it covers	Evidence of a high score
Technical	Test design techniques (equivalence partitioning, boundary-value analysis, decision tables, state transitions), levels and types of testing, scripting	Designs the boundary set for a slab tariff without prompting
Domain	Tariff, billing rules, payment flow, consumer complaints, regulation	Detects that a bill rule is ambiguous before testing
Analytical	Risk assessment, defect analysis, root-cause thinking, reading results	Finds that 7 failures have 3 causes
Communication	Incident reports, reviews, negotiating a dispute with a developer	Reports a defect that a stranger can reproduce
Planning	Estimating effort, scheduling, prioritising by risk	Estimates a feature within 20% of the actual
Tools	Test-management tool, version control, automation, load tools	Writes a regression test and a bisection probe

The scale is 1 (aware), 2 (basic, needs help), 3 (works independently), 4 (strong, helps others) and 5 (expert). The scores in Figure 7.1 are the **author's judgement from the four profiles stated in the assignment** and are not the result of an assessment of real people; the test manager should replace them with a scored interview and a practical exercise.

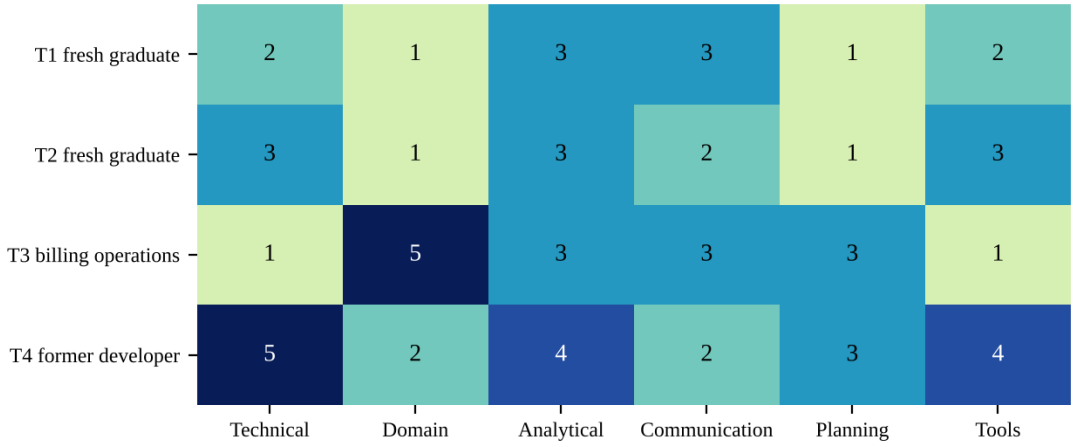


Figure 7.1: Skills matrix of the four testers (author's judgement from the stated profiles)

Table 7.3: Team average by category and the main gap

Category	T1	T2	T3	T4	Team average	Reading
Technical	2	3	1	5	2.75	One strong tester; boundary technique is thin in three of four
Domain	1	1	5	2	2.25	Weakest category; only T3 knows the billing rules
Analytical	3	3	3	4	3.25	Strongest category
Communication	3	2	3	2	2.50	Incident reports will need review
Planning	1	1	3	3	2.00	Weakest with domain; no one has estimated test effort
Tools	2	3	1	4	2.50	Automation depends on T4

The two weakest categories are domain (2.25) and planning (2.00). The domain gap is the one the pilot punished: only one of four testers could have recognised that the slab limit was wrong without being told. The matrix also shows a dependence: technical depth and tools both rest on T4, and domain on T3, so the absence of either (Task 6, situation 3) removes a category from the team.

7.4 Four-week plans for the two fresh graduates

Table 7.4: Strategy A: classroom training and certification (per graduate)

Week	Activities	Hours	Assessment
1	Course module 1: fundamentals, principles, test levels and types; self-study 4 h	14	Short quiz
2	Module 2: test design techniques, defects and incident reports; self-study 4 h	14	Practice questions
3	Module 3: test management, tools and risk; self-study 4 h	14	Mock exam 1
4	Module 4: review; mock exam 2; certification examination; self-study 4 h	14	Examination result
Total		**56**	Pass or fail of the examination

Table 7.5: Strategy B: mentoring and pair testing (per graduate; mentors are T3 and T4)

Week	Activities	Mentee hours	Mentor hours	Assessment
1	Shadow the mentor on billing tests; mentor explains the slab tariff and the 500-unit pilot failure; pair design of boundary cases for TC-BILL	20	5	Review of the 8 boundary cases
2	Pair design and execution on the payment feature; mentee writes two incident reports, mentor reviews	20	5	Incident-report checklist
3	Mentee leads, mentor reviews tests and reports	20	5	Review of 10 test cases
4	Mentee works alone; mentor reviews a sample; seeded-fault exercise (four faults F1 to F4 of Task 1)	20	5	Faults found (target 3 of 4)
Total		**80**	**20**	

Table 7.6: Strategy C: skills matrix with rotation (per graduate)

Week	Rotation	Activities	Hours	Assessment
1	Test design	Design the boundary and partition set for the bill feature; the billing coordinator explains the rules (4 h)	14	Matrix re-score: technical, domain
2	Automation	Write unit tests for the slab function and a regression test (Task 1)	14	Matrix re-score: tools
3	Defect analysis	Classify and analyse the pilot defects; Pareto of causes; root cause of one defect	14	Matrix re-score: analytical
4	Reporting	Produce the dashboard and weekly status report from the log (Task 4)	6	Matrix re-score: communication, planning
	Matrix review	Meeting with the test manager; plan of the next rotation	8	
Total			**56**	

7.5 Skills built, skills left undeveloped, and the link to pilot defects

Table 7.7: Skills each strategy builds (full), partly builds, or leaves undeveloped

Skill	A. Classroom	B. Mentoring	C. Rotation	Link to a pilot-year defect
Technical (boundary analysis)	Full (theory)	Partly (what the mentor shows)	Partly (design rotation)	Boundary fault at 500 units missed
Domain (tariff rules)	None	Full (mentors T3, T4)	Partly (coordinator session)	Wrong bill: the rule was misread
Analytical	Partly	Partly	Full (defect-analysis rotation)	Seven failures from three causes; 18% reopen
Communication	None	Full (report review)	Partly (reporting rotation)	Reports not reproducible (Task 4)
Planning	Partly (theory)	None	Partly	Nobody could say how much testing was done
Tools	Partly	Partly	Full (automation rotation)	No regression tests, so fixes reopened

A leaves the practical skills undeveloped, and a certificate does not make a tester who can recognise a wrong slab limit. B transfers the domain and the habits of reporting that the pilot lacked, but it can pass on the mentor's blind spots, and for the fresh graduates it depends on two people (T3 and T4) whose time the project also needs. C builds the breadth and, importantly, produces the measurement (the matrix), but four weeks of rotation does not make anyone deep in an area.

7.6 Cost and time against the budget and the team's effort

Table 7.8: Cost and time of the three strategies for the two graduates (assumptions marked)

Strategy	Cash cost (Rs)	Trainee hours	Senior or manager hours	Cash as share of the Rs 1,50,000 budget	Effect on the 489.1 h baseline
A. Classroom and certification	44,000 (assumed Rs 22,000 per person for course and examination; check the current price list)	112	0	29.3%	112 h taken from testers' time
B. Mentoring and pair testing	0	160 (productive while paired)	40 (mentors)	0%	40 h of mentors' time, offset by the mentees' output
C. Skills matrix with rotation	0 (spreadsheet)	112	12 (manager)	0%	About 10% productivity loss for four weeks, 48 h

Task 2 set aside Rs 20,000 of the budget for test-management tool hosting, training and scripts. Strategy A at Rs 44,000 would exceed that line by Rs 24,000 and the whole contingency (Rs 10,000) could not cover it, so it would have to be paid for from the Corporation's training budget or its scope reduced to one certification. B and C fit within the budget since they cost mainly time.

7.7 Evaluation: the strategy the test manager should use

The test manager should use **B as the base, C as the measuring instrument, and A only as a targeted module**. The two graduates are paired with T3 and T4 for four weeks (B), which supplies the two things the matrix shows they lack, domain and practice. The skills matrix (C) is scored at week 0 and week 4, so that the gain is measured and the rotation is used in the second month. From A the project takes only the 10-hour module on test design techniques, which gives the vocabulary of the techniques cheaply, and leaves the certification as an option for those who want it, since the pilot did not fail for want of certificates. The expected gains are measurable:

Table 7.9: Measurable skill gains (targets)

Measure	Baseline	Target at week 4
Matrix score, technical (T1, T2)	2 and 3	At least 3 and 4
Matrix score, domain (T1, T2)	1 and 1	At least 3 and 3
Seeded-fault exercise: faults F1 to F4 of Task 1 found with a prepared boundary set	Not measured	At least 3 of 4
Incident-report checklist score (Table 4.9)	Not measured	At least 90%
Reopen rate of defects raised by the two graduates	18% (team, pilot)	At most 8% (policy P6)

7.8 How AI tools change the skills of a test specialist

AI tools lower the cost of producing test cases, scripts and reports. They do not lower the cost of knowing whether those are right. The evidence of this report makes the point: in Task 1 three of four illustrative AI-style repairs failed the regression suite, in Task 3 an unreviewed keyword matcher produced a matrix with 76.1% precision and missed the link to the duplicate-payment requirement, and in Task 4 the draft summary contained five false or unsupported claims in eight statements. The skills that gain value are therefore the **oracle** skills: knowing the business rule well enough to write the expected value (domain), designing the cases that separate right from wrong (technical), reviewing output with suspicion (analytical), stating a requirement so precisely that a machine cannot misread it (communication), and deciding what to automate and what to inspect (planning). The skills that lose value are routine ones: typing out repetitive cases, formatting reports, writing boiler-plate scripts. The skills that remain essential are

domain knowledge, boundary and risk reasoning, independence of judgement and honest reporting. A new entry in the matrix is needed, *working with AI tools*, with sub-skills for prompting, checking and recording what the tool produced.

7.9 Observation against prediction, summary and recommendation

Table 7.10: Strategies as rows and measures as columns

Strategy	Skills built (full)	Cash cost	Senior time	Measurable gain	Prediction confirmed?
A. Classroom	Technical vocabulary	Rs 44,000	0 h	Examination result only	Yes
B. Mentoring	Domain, communication	0	40 h	Seeded-fault exercise, report checklist	Yes
C. Rotation	Analytical, tools	0	12 h	Matrix re-scores	Yes

The predictions were confirmed. **Recommendation:** B for the first four weeks with T3 and T4 as mentors, C to measure the gain and to widen the graduates in month two, and A restricted to the design-techniques module. The weakest categories of the team (domain 2.25, planning 2.00) are the first targets, and the test manager should re-score the matrix at the end of week 4 and publish the result.

8. Integrated Test Process Evaluation: Compare Test Documentation Sets [4 Marks]

8.1 Context and objective

Tasks 1 to 7 each compared three approaches and chose one. This task combines the choices into one test process for PowerPay and compares three *documentation sets* for the whole process: a full IEEE 829-style set (A), a lean set generated by the test-management tool (B) and a hybrid tailored to risk (C). The question is not whether to document but how much, for which feature and for whom, because every document costs effort and an unread document costs more than it saves.

Table 8.1: Choices of Tasks 1 to 7 carried into the integrated process

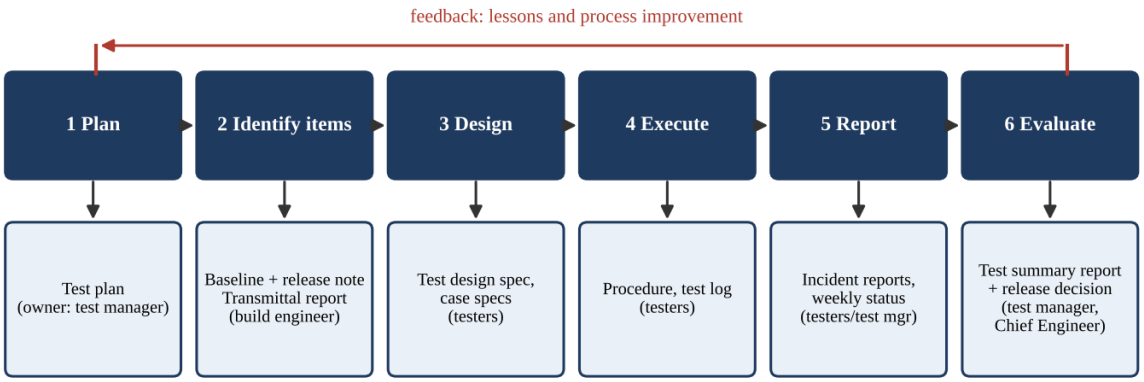
Task	Choice	Effect on the process
1 Fault localisation	Systematic localisation (reduce, bisect, regression test) as the required procedure; print statements and debugger as tools	A fix needs a minimal case, a probe and a regression test
2 Test plan	Risk-based plan with the administrative annex of the IEEE 829 plan	Effort 269.1 h (489.1 h with extras); exit criteria per risk
3 Identification	Baseline and configuration items for every build; traceability for billing and payment; transmittal report	Item identity checked at deployment
4 Reporting	Log and incident reports (generated), dashboard and weekly status, summary report per level	Three layers for three readers
5 Policy	Policy B signed now, strategy and improvement path to follow	Eight measurable statements
6 Team	Module-aligned teams with independent reporting line and rotation	Short feedback loops
7 Skills	Mentoring for the graduates, matrix to measure, targeted classroom module	Domain and practice built early

8.2 Prediction recorded before the exercise

Table 8.2: Prediction for Task 8

Set	Predicted effort for the bill feature	Predicted weakness	Predicted strength
A. Full IEEE 829	Highest (about 35 to 40 h)	Documents nobody reads; slow to change	Complete and auditable
B. Lean tool-generated	Lowest (about 10 to 15 h)	Little narrative; weak where judgement must be explained	Cheap, always current
C. Risk-tailored hybrid	Between (about 30 h)	Needs a clear rule for what is tailored	Effort goes where the risk is

8.3 The integrated test process and its documents



Each stage has one owner and one named output document; a document is skipped only by the rule in Section 8.6

Figure 8.1: The PowerPay test process with the document produced and the owner at each stage

Table 8.3: Stages, documents and owners

Stage	Activity	Document produced	Owner
1. Plan	Rank risks, fix scope, estimate effort, set exit criteria	Test plan (risk-based, with annex)	Test manager
2. Identify items	Baseline, manifest, checksums, release note; deployment check	Baseline record, test item transmittal report	Build engineer
3. Design	Select techniques, derive conditions, specify cases, trace to SRS	Test design specification, test case specifications, traceability matrix	Testers, with the analyst
4. Execute	Run the procedure, record the result, raise incidents	Test procedure, test log, incident reports	Testers
5. Report	Dashboard daily, weekly status, summary per level	Dashboard, weekly status report, test summary report	Test manager
6. Evaluate	Compare with exit criteria, release recommendation, lessons learned	Summary report with evaluation, process improvement notes	Test manager, Chief Engineer

The feedback arrow of Figure 8.1 is part of the process. Lessons from stage 6 (for example the observation of Task 1 that a probe must isolate a single fault) change the plan of the next release, and the improvement path of Policy C gives the arrow an owner.

8.4 Documents of the three sets for the bill-generation feature and their effort

The feature is bill generation (SRS-04 to SRS-06) with 18 designed conditions: the 11 numeric slab cases TC-BILL-01 to TC-BILL-11 of Appendix C and seven further cases (bill number uniqueness, due date, slab-wise breakup, rounding and similar) that are not listed in Appendix

C. The documents of **set A** for this feature are those already prepared in Tasks 3 and 4: the plan section, TDS-BILL-01 (Table 3.3), the case specifications (Table 3.4 shows four of 18), TP-BILL-01, the log (Task 4), the three incident reports and the summary report. In **set B** the test cases live as records in the tool; the log and the incident reports are generated from execution; narrative is limited to a two-page plan and a one-page summary. **Set C** is set A for this feature, except that the log and incident reports are tool-generated.

Table 8.4: Set B: a lean test case record as the tool would store it (illustrative format)

Field	Value
Case ID / title	TC-BILL-07 / Bill for 500 units
Linked requirement	SRS-05, SRS-06
Risk / priority	High / P1
Steps	Enter 500 units; generate bill
Expected	Rs 2,700.00
Baseline / tariff version	BL-1.0-RC1 / v2.2 (filled at execution)
Result history	Cycle 1: FAIL (2,702.00, TIR-001); cycle 2: to be run

Table 8.5: Effort in person-hours to produce the documents for the bill-generation feature (modelled)

Document	A. Full	B. Lean	C. Hybrid	Basis
Plan (including item identification and transmittal)	6	4	6	Plan section; lean plan of 2 pages
Test design specification	5	0	5	Table 3.3
Test case specifications / records	9	4.5	9	18 cases x 0.5 h (full); 18 x 0.25 h (record)
Test procedure	3	0	3	TP-BILL-01
Test log	3	0.5	0.5	Manual log vs export set-up
Incident reports	6	1.5	1.5	3 x 2 h (full) vs 3 x 0.5 h (tool)
Summary report	6	2	5	IEEE-style report vs one page
Total for the feature	**38**	**12.5**	**30**	

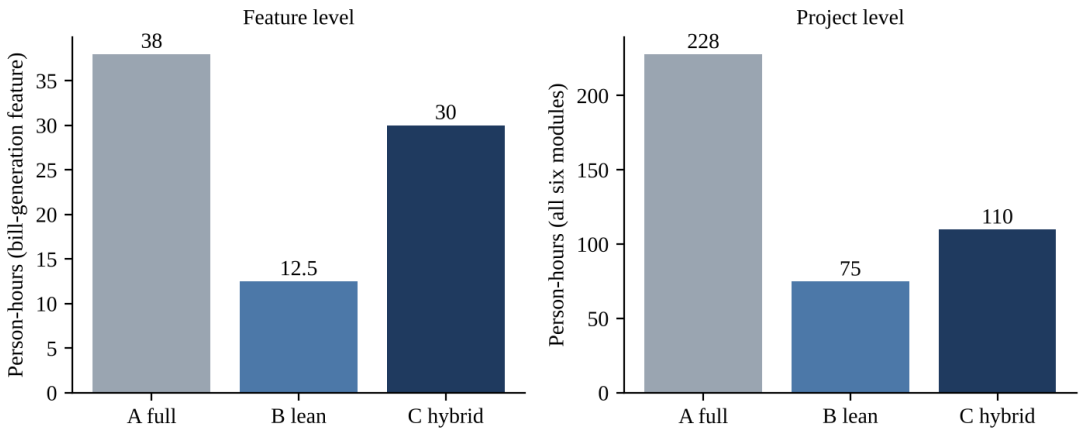


Figure 8.2: Documentation effort for the feature and for all six modules (modelled)

For the whole project the totals are: set A, six modules x 38 h = 228 h; set B, six x 12.5 h = 75 h; set C, two modules (billing and payment) at 30 h plus four modules at 12.5 h = 60 + 50 = 110 h. Compared with the 269.1 hours of test effort of Task 2, the documentation of set A is 84.7% of the test effort and consumes 45.9% of the combined effort (228 / 497.1); set B is 27.9% and 21.8% (75 / 344.1); set C is 40.9% and 29.0% (110 / 379.1). A documentation set that costs almost as much as the testing is a sign that the documents are produced for the document's sake.

8.5 Analysis of the sets and of the pilot-year problems they would expose

Table 8.6: Comparison of documentation sets (1 = poor, 5 = good)

Criterion	A. Full	B. Lean	C. Hybrid	Reason
Completeness	5	3	4	B has no design specification or procedure; C is complete where money moves
Auditability	5	4	5	Tool logs are time-stamped and immutable; C adds full documents for billing and payment
Effort (5 = least)	1	5	3	Hours of Table 8.5
Readability	2	4	4	Long documents are not read; short records and reports are
Ease of change	2	5	4	A tariff change rewrites many documents in A; in B it changes records
Total (of 25)	**15**	**21**	**20**	

Table 8.7: Pilot-year problems each set would have exposed (Y = yes, P = partly)

Pilot-year problem	A. Full	B. Lean	C. Hybrid
Wrong bill at 500 units	Y: the design specification lists boundary values; the case TC-BILL-07 would exist	P: only if the tester thought of the case	Y for billing

Pilot-year problem	A. Full	B. Lean	C. Hybrid
Tested build differed from released build	Y: the transmittal report	Y: baseline in every record	Y
Nobody knew how much testing was done	Y	Y: dashboard	Y
Duplicate payment recorded twice	Y for payment (concurrency in the design)	P	Y for payment
No regression tests, 18% reopened	Y: incident report requires one	Y: tool link	Y
Audit gap on failed payment attempts	Y: audit requirement traced	P	Y

8.6 Evaluation: the documentation set to adopt and the rule for skipping a document

The Corporation should adopt **set C**. It scores 20 against 21 for set B on the five criteria and 15 for set A, but it is the only set that gives the full documents where money is computed and moved and the lean tool-generated ones where it is not, and so it passes the audit test for the part of the system that the auditors will look at. It costs 110 hours, 41% of the test effort, 118 hours less than the full set. Set B is the right choice for a feature with no money and no personal data; set A for every module costs 228 hours and would be abandoned by the team.

> **Rule for skipping a document:** A document may be omitted only when **all three** conditions hold: (1) the feature handles neither money nor personal data (its risk exposure in the plan is below 12); (2) the information the document would contain is held by the tool and can be exported on request; and (3) the test manager has recorded the omission and the reason in the plan, with a date. Billing and payment documents may never be skipped.

8.7 Can AI-generated documents be accepted for audit without review?

No. An AI tool can draft a test case specification, a procedure or a summary from the test cases quickly, and the saving is real. But an audit looks for three things that a draft alone cannot provide. **Authenticity:** the document must be traceable to something that happened (a log line, a baseline, a checksum); a model can write a plausible sentence about an execution that never occurred, as the draft summary of Task 4 did. **Accuracy:** Task 4 found five unsupported claims in eight statements, and Task 3 found that a keyword matcher invented 11 links. **Accountability:** a signature on an audit document is a person's statement that they have checked it; a model cannot sign. A draft is acceptable for audit only under the conditions of Table 8.8.

Table 8.8: Conditions under which AI-drafted documents are acceptable for audit

Condition	Evidence the auditor can check
Every statement of fact is checked against the log or the tool	Reviewer's checklist with the log lines cited
The source data (log, test cases, baseline) are kept with the document	Links to the records in the tool
The tool, version and date of generation are recorded	Generation record
A named reviewer approves and signs	Name, role, date on the document
Documents for billing and payment are reviewed by the test manager	Approval record

8.8 Observation against prediction, summary and recommendation

Table 8.9: Documentation sets as rows and measures as columns

Set	Feature effort	Project effort	Share of combined effort	Score (of 25)	Prediction confirmed?
A. Full IEEE 829	38 h	228 h	45.9%	15	Yes
B. Lean tool-generated	12.5 h	75 h	21.8%	21	Yes
C. Risk-tailored hybrid	30 h	110 h	29.0%	20	Yes

The predictions were confirmed. **Recommendation:** adopt set C; generate everything the tool can generate; write full documents only for billing and payment; apply the skipping rule of Section 8.6; and require the review conditions of Table 8.8 for any AI-drafted document.

9. Re-evaluate and Redesign the Test Process [8 Marks]

Changed requirement and objective. PowerPay is now chosen for the **whole state with 12 lakh consumers**. It must include **smart-meter data upload every 15 minutes**. It must be tested in **4 weeks instead of 10**. The team is **split across two offices**, **one tester is withdrawn** (three remain), and a **data-protection law** requires that consumer data used in testing be masked and that every test result be auditable. The process designed in Tasks 1 to 8 was built for the earlier conditions. This task analyses the impact of each change, evaluates the original process against a checklist, compares three ways of adapting it (A retain and cut scope by risk, B modify, C redesign), decides, and defends the decision with the evidence of the earlier tasks.

Table 9.1: Prediction for Task 9

Strategy	Predicted schedule feasibility	Predicted risk coverage	Predicted weakness
A. Retain and cut scope by risk	Poor: documents and ceremony remain	Good for High risk, nothing below	Overhead consumes the scarce hours
B. Modify the process	Best	Best	Depends on the shared tool and on discipline in two offices
C. Redesign organisation and policy	Fair: set-up costs hours before testing starts	Good, with the best audit evidence	Over budget and slow to start

9.1 Impact analysis

9.1.1 Assumptions of the original plan that are no longer valid

Table 9.2: Assumptions of the original plan against the new conditions

Assumption (original)	Now	Valid?	What happens if the original plan is used unchanged
Duration: 10 weeks, builds in weeks 3, 5 and 8, three cycles	4 weeks	No	Three cycles do not fit; build 3 would arrive after the release date; weekly status arrives too late to act
Team: 4 testers in one office	3 testers in two offices	No	Capacity falls from 1,200 h to 360 h; handovers between offices are not planned; the floating tester of the safeguard is gone
Volume: 3 lakh consumers	12 lakh (4 times)	No	One test server cannot hold the data; a defect hitting 0.1% of bills now affects 1,200 consumers instead of 300
Scope: about 180 conditions	About 400 (smart-meter upload and load)	No	Required effort 598 h against 360 h of capacity
Data: realistic data, assumed available	Masked, non-personal	No	Real consumer data in the test environment would break the law
Reporting: weekly, a summary per level	Daily, auditable	No	The Chief Engineer learns of a critical defect up to a week late; results without a trail are not evidence

9.1.2 Effect of each change on each part of the process

Table 9.3: Effect of each changed condition on each part of the process

Change	Plan	Documents	Policy	Organisation	Reporting
12 lakh consumers	Load and scale tests added; risk re-ranked	Larger data volume in the test data record	P3 regression automation becomes essential	More work per module	Defect counts must be read as consumers affected
Smart-meter upload every 15 minutes	New High-risk area; 15-minute stream tests	New design specification	New measurable statement for data loss	Needs a tester with tool skill (T4)	Throughput and loss metrics added
4 weeks	One cycle per tier; weekly builds	Lean documents except billing and payment	Exit rule keeps P1	Short feedback loops needed	Daily, not weekly
Two offices	Hand-over times planned	Shared tool, not files	Same policy in both offices	Embedded testers in each office	One live view
One tester withdrawn	Capacity 360 h	Less time for documents	Rule for skipping documents applies	No floating tester	Capacity shown daily
Data-protection law	Masking is a stage of the plan	Reports contain no personal data	P8 is now legal, not advisory	Roles for masking and key custody	Reports checked for personal data
Auditable results	Evidence capture planned	Every result carries baseline, tariff version, tester, time	P4 extended to all results	Audit liaison named	Report trail kept

9.1.3 Recalculation of the effort

The unit times of Task 2 are kept: design 0.4 h per condition; execution 0.25 h per condition per cycle for 3 cycles; retest 20% of execution; management overhead 15%. For about 400 conditions:

Table 9.4: Effort for about 400 conditions against the reduced capacity

Item	Calculation	Person-hours
Test design	400 x 0.4	160.0
Test execution	400 x 0.25 x 3 cycles	300.0
Retest	20% of 300.0	60.0
Subtotal	160 + 300 + 60	520.0
Management overhead	15% of 520.0	78.0
Total required	520.0 + 78.0	**598.0**
Capacity	3 testers x 4 weeks x 30 h	360.0
Shortfall	598.0 - 360.0	**238.0 (39.8% of the requirement)**

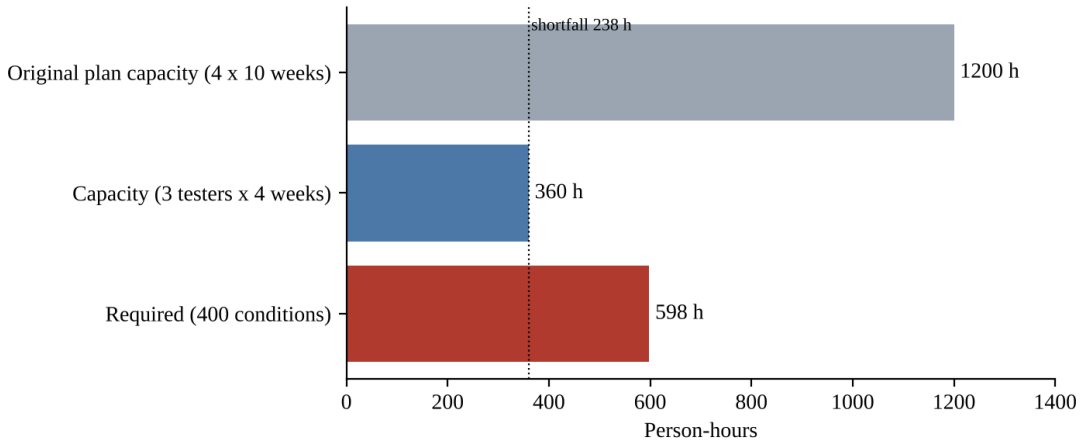


Figure 9.1: Effort required for 400 conditions against the capacity of three testers over four weeks

The required effort is 598 / 360 = 166% of capacity. If every condition were given the same share, only 360 / 598 = 60.2% of the work could be done. The original plan used 22.4% of its capacity; the new plan would need 166%, so the margin of Task 2 has turned into a deficit of 238 hours, and the activities outside the formula (environment, load, triage) are not even counted. The effort per condition is 598 / 400 = 1.495 hours.

9.1.4 Load created by the smart meters

Each meter uploads every 15 minutes, 96 times a day. For 12 lakh (1,200,000) consumers: 1,200,000 x 96 = **115.2 million readings a day**, an average of 115,200,000 / 86,400 = **1,333 readings a second**, and 1,200,000 x 96 x 30 = **3.456 billion readings a month**. If the meters synchronise on the quarter hour and all upload within 60 seconds, the burst is 1,200,000 / 60 = **20,000 readings a second**. These are the figures against which the load test is designed; the burst, not the average, decides the capacity needed. If the SRS-13 target of 2,000 concurrent users for 3 lakh consumers (Assumption A4) scales in proportion, the state requires 8,000 concurrent users (assumption).

9.1.5 Data-protection rule applied to test data and documents

Table 9.5: Items that must be masked or removed, and how

Item	Where it appears	Treatment
Consumer name, address	Test data, bills, screenshots, incident reports	Replace by synthetic values; keyed token in logs
Consumer number	Test data, logs, matrix, reports	Keyed token (HMAC); no reversible mapping in the test environment
Mobile number, e-mail	Test data, SMS stub logs	Partial mask or synthetic; stub accepts only masked numbers
Smart-meter reading series	Upload test data, load data	Generate synthetic profiles from a statistical model; do not copy real profiles, which reveal when people are at home

Item	Where it appears	Treatment
Payment instrument, bank references	Payment tests	Gateway sandbox tokens only; never real instruments
Complaint free text	Complaint tests	Synthetic text; redact in reports
Screenshots and log extracts	Incident reports, summary reports	Capture only masked data; scan before attaching
Units, amounts, tariff version	All	**Not masked**: they are needed to compute the expected bill

The masking utility (Appendix B) replaces the identifying fields by keyed hash tokens (HMAC-SHA256 with a key that is not held in the test environment), keeps the format of the numbers and leaves units and amounts intact so that the oracle still works. Figure 9.2 shows the real output on three synthetic records. The retention of the last four digits of the consumer number is a convenience that the legal adviser must approve; if it is not approved the field is fully tokenised.

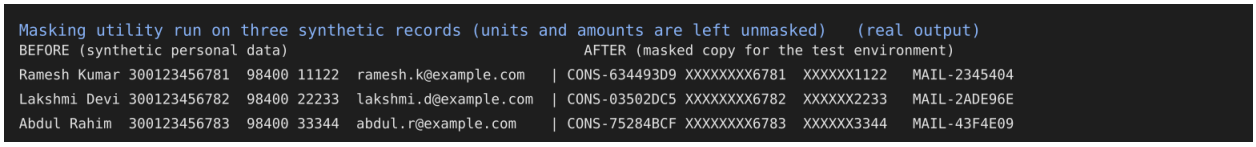


Figure 9.2: Output of the masking utility on synthetic records (real run)

Auditability adds a second requirement. Every test result must be stored with the baseline identifier, the tariff version, the tester, the time and a checksum of the exported result set, in a store that testers cannot edit. This is the extension of policy statement P4 to all results.

9.2 Evaluation of the original process

9.2.1 What remains valid and what becomes unsafe

Table 9.6: Checklist of the original process against the new constraints

Part of the original process	Valid?	Reason
Policy B: principles, definitions, release rule P1 (no open Critical)	Valid	Independent of size and time
Policy statements P4 (baseline identity), P7 (regression test per fix), P8 (no unmasked data)	Valid, and more important	Now legal and audit requirements
Baseline and configuration identification (Task 3, B)	Valid	Cheap, catches a wrong tariff file at deployment without a test
Traceability for billing and payment (Task 3, C)	Valid	Must be extended to smart-meter requirements
Systematic fault localisation and regression test per fix (Task 1)	Valid	Saves rework when time is short
Three-cycle plan on three builds (Task 2)	Unsafe	Four weeks allow weekly builds and one cycle per tier
Effort estimate of 269.1 h and capacity margin	Unsafe	Now 598 h against 360 h

Part of the original process	Valid?	Reason
Documentation set C with full documents for billing and payment	Partly	Full documents for billing and payment cost 60 h of the 360
Module-aligned teams with a floating tester (Task 6)	Partly	Three testers for three modules leaves no floating tester; two offices
Weekly status and summary at end of level (Task 4)	Unsafe	Too slow; replace by daily live view
Realistic test data on the test server	Unsafe	Illegal without masking
Single test server	Unsafe	Cannot hold 12 lakh consumers or the 15-minute stream

9.2.2 The original pass/fail and exit criteria under the new conditions

The criteria of Task 2 were: all High-risk conditions executed in the latest cycle and passing; no Critical or Major defect open in billing or payment; at least 95% of all conditions passing; load target SRS-13 met. Used unchanged they fail in two ways. **They cannot be met:** a *latest cycle* implies three cycles, and 598 hours of work do not fit into 360 hours. A criterion that cannot be met is dropped under pressure without a record, which is worse than changing it openly. **They are weaker than they look at 12 lakh:** a 95% pass rate of conditions says nothing about how many consumers are billed wrongly. At 3 lakh consumers a defect that hits 0.1% of bills affects 300 consumers; at 12 lakh it affects 1,200. The meaning of "acceptable" shifts with the volume although the percentage is unchanged. Finally, the criteria have no statement about the smart-meter stream or about the masking and the audit trail.

9.2.3 The measure of the original reporting trusted least

The measure to trust least is the **pass rate of executed cases**. Under compression the executed set is a chosen subset: if the tester, pressed for time, executes the easy cases first and leaves the hard ones, the pass rate rises while the risk stays. Task 4 showed that the figure counts tests and not defects, that 7 failed tests were 3 defects, and that a suite without a boundary case above 500 units passed on a wrong tariff file. The weekly status report is the second least trusted because of its delay. The measures that remain reliable are those that do not depend on what was chosen: the number of High-risk conditions *not yet executed*, the number of open Critical defects, and the reconciliation result of an independent sample.

9.3 Compare and adapt

9.3.1 The three strategies measured against the new capacity

The conditions are ranked by risk into three tiers (assumption): **High** (120 conditions: bill calculation, payment, duplicate prevention, smart-meter upload and reconciliation, audit and masking), **Medium** (160) and **Low** (120). High-risk conditions are executed in three cycles, the others in one. The cost of a condition is design 0.4 h plus execution time per cycle x number

of cycles x 1.2 (the factor 1.2 includes the 20% retest); the 15% management overhead is taken from the capacity before the allocation. Each strategy has a fixed overhead (documents, tool set-up, masking, coordination between the offices) that is taken from the 360 hours first. Conditions are then added in order of risk until the hours are used.

Table 9.7: Fixed overheads of the three strategies (modelled assumptions)

Item (hours)	A. Retain and cut	B. Modify	C. Redesign
Plans and documents	48 (full set)	16 (short iteration plans)	40 (new policy and process set-up)
Tool and evidence automation	0	16 (shared tool)	30 (automated evidence capture)
Masking	24 (manual)	16 (semi-automated)	16 (automated)
Coordination between offices	16	12 (embedded testers)	24 (central group)
Fixed total	**88**	**60**	**110**
Execution time per condition per cycle	0.25 h	0.25 h	0.20 h (evidence captured automatically)

Table 9.8: Result of the allocation (measured by script from the assumptions)

Measure	A. Retain and cut	B. Modify	C. Redesign
Hours left for testing after fixed overhead and 15% management	(360 - 88) / 1.15 = 236.5	(360 - 60) / 1.15 = 260.9	(360 - 110) / 1.15 = 217.4
High-risk conditions covered (of 120)	120	120	120
Medium-risk conditions covered (of 160)	115	149	129
Low-risk conditions covered (of 120)	0	0	0
Conditions covered (of 400)	235 (58.8%)	269 (67.3%)	249 (62.3%)
Risk exposure covered (weights 5, 3, 1; total 1,200)	945 (78.8%)	1,047 (87.3%)	987 (82.3%)

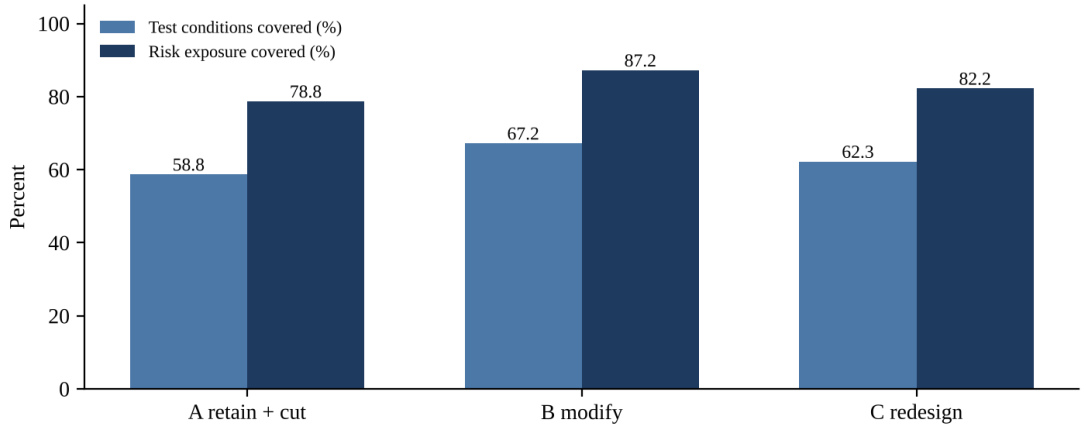


Figure 9.3: Conditions and risk exposure covered by the three strategies

Two points follow from the numbers. First, **no strategy covers all 400 conditions**: even with no overhead, 360 h of capacity cannot do 598 h of work. Every strategy therefore *chooses* what to leave untested, and the plan must say so in writing. Second, **strategy B covers the most, 269 conditions and 87.3% of the risk exposure,** because it spends the fewest hours on overhead (60 against 88 and 110). Strategy C's saving in execution time (0.20 h instead of 0.25 h) does not repay the 110 hours of set-up in a four-week project.

Table 9.9: Comparison on the five criteria (1 = poor, 5 = good)

Criterion	A. Retain and cut	B. Modify	C. Redesign	Basis
Schedule feasibility	2	4	3	A has 88 h of overhead; C needs 110 h of set-up before it pays back
Risk coverage	3	4	4	78.8%, 87.3%, 82.3% of exposure
Audit evidence	3	4	5	C captures evidence automatically; B uses the tool log; A is manual
Cost (cash, against Rs 1,50,000)	4	4	2	A: Rs 1,20,000 (load, second environment, masking); B: Rs 1,40,000 with the shared tool; C: about Rs 1,90,000 (over by Rs 40,000), all assumed
Team effectiveness (two offices, three testers)	2	4	3	Embedded testers in B; A keeps hand-offs; C needs a new central group
Total (of 25)	**14**	**20**	**17**	

9.3.2 Decision: modify

The decision is to **modify** the process: strategy B, taking from C the cheap parts that serve audit (masking automation and evidence captured by the tool) and from the original process the parts that remain valid (release rule, baseline identification, traceability for billing and payment, systematic localisation and regression tests). Retaining the process unchanged (A) leaves 88 hours of overhead in a project that is already 238 hours short; a full redesign (C) spends 110 hours before the first test and exceeds the budget. The change is moderate because the part of the process that matters most, the evidence that a bill is right, is valid and cheap.

9.3.3 The revised test process

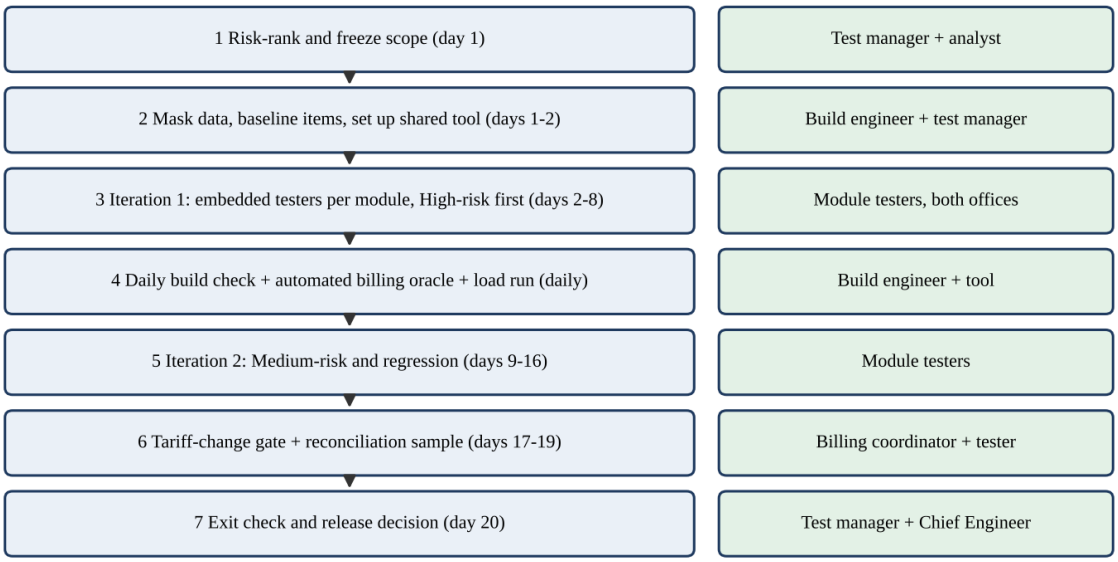


Figure 9.4: The revised four-week test process (20 working days) with owners

Table 9.10: Revised plan outline: stages, what each does, owner and the reason for the order

Stage (days)	What it does	Owner	Why in this position
1 Risk-rank and freeze scope (day 1)	Tier the 400 conditions; agree what is not tested and have the Chief Engineer sign it	Test manager, analyst	Everything else depends on the choice of what to test
2 Mask data, baseline, shared tool (days 1-2)	Create masked test data; tag the baseline; set up the tool in both offices; start the audit log	Build engineer, test manager	Legal precondition; nothing may be run on unmasked data
3 Iteration 1 (days 2-8)	Execute High-risk conditions on build 1 with testers embedded in the module teams; fix and retest	Module testers in both offices	The highest risk gets the longest time and the most cycles
4 Daily checks (every day)	Smoke test of the new build; baseline and tariff check; automated billing oracle on a sample; load run	Build engineer, tool	Cheap, repeated, catches regressions early
5 Iteration 2 (days 9-16)	Medium-risk conditions (149 selected) and regression of High-risk	Module testers	Medium risk only once, after High risk is stable
6 Tariff gate and reconciliation (days 17-19)	Freeze the tariff file; run the reconciliation sample; final load and burst test	Billing coordinator, tester	The tariff is the largest late-change risk; freeze it before the end
7 Exit check and decision (day 20)	Compare with the revised exit criteria; release recommendation	Test manager, Chief Engineer	The decision rests on the criteria and not on the calendar

9.3.4 Revised exit criteria

Table 9.11: Revised exit criteria and how each limits risk

ID	Criterion	Risk it limits
E1	All 120 High-risk conditions executed in the last two iterations	Unexecuted high-risk conditions hidden by a good pass rate
E2	No Critical or Major defect open in billing, payment or smart-meter upload	Wrong bills or lost money released
E3	At least 98% of High-risk conditions pass and every failure is explained in an incident report	Unexplained failures accepted
E4	At least 149 Medium-risk conditions executed with at least 95% passing	Medium-risk neglect
E5	Independent reconciliation of a random sample of 3,000 bills has 0 mismatches, and the 8 boundary cases pass	A critical billing defect that no case tests (see below)
E6	Load test: sustained 1,333 readings a second, burst 20,000 a second for 60 s, and 8,000 concurrent users with 3 s response (targets assumed)	Failure at state scale
E7	100% of results carry baseline, tariff version, tester and time; masking scan shows 0 findings	Audit and legal failure
E8	Untested Low-risk conditions (120) and unselected Medium-risk conditions (11) listed and accepted in writing by the Chief Engineer; staged rollout starting with one district	Silent acceptance of untested areas

Why E5 limits the risk of an unnoticed critical billing defect. A random sample of n bills, recomputed by an independent reference calculation, detects a defect that affects a fraction p of bills with probability 1 - (1 - p)^n. For p = 0.1% and n = 3,000 this is 1 - 0.999^3000 = 95.0%; the sample size needed for 95% is ln(0.05) / ln(0.999) = 2,994. For p = 0.5% and n = 1,000 it is 99.3%. A defect that hits 0.1% of 12 lakh consumers is a defect that hits 1,200 households, so E5 makes it unlikely to pass unnoticed. A random sample is weak against a defect that affects only a boundary value (the Task 1 fault at exactly 500 units), which a random sample rarely picks, and that is why E5 also requires the deterministic boundary cases. The two together cover the two kinds of billing defect: the common and the rare-but-exact. The residual risk, a defect below 0.1% that escapes the sample with up to 5% probability, is handled by the staged rollout of E8.

9.3.5 Daily reporting set for the Chief Engineer

Table 9.12: One-page daily report

No.	Item	Source
1	High, Medium and Low conditions executed against planned, by tier	Tool
2	Passed, failed and blocked, with open defects by severity and age (billing, payment and smart-meter first)	Tool
3	Result of the reconciliation sample: n, mismatches	Oracle script
4	Load status: last run, rate reached, errors	Load tool

No.	Item	Source
5	Baseline and tariff version check: pass or fail	Build engineer
6	Masking scan: findings	Scan
7	Hours used and hours remaining against 360, by office	Tool
8	Three top risks and the decision needed	Test manager

The report is short on purpose. It shows the *unexecuted* High-risk conditions and the reconciliation result, the two measures that are hard to flatter, and it shows hours remaining, so that the Chief Engineer sees the 238-hour shortfall as a daily number rather than a surprise on day 20.

9.4 Evidence-based defence

9.4.1 Defence of the decision

Table 9.13: Evidence for the decision to modify the process

Evidence	Source	What it shows
598 h required against 360 h available; shortfall 238 h	Section 9.1 (Task 2 formula)	No process can test everything; the question is what is left out and who accepts it
Strategy B covers 269 conditions and 87.3% of risk exposure; A 58.8% and 78.8%; C 62.3% and 82.3%	Table 9.8	B spends the fewest hours on overhead
Baseline check detects a wrong tariff file with no test	Task 3	A cheap safeguard that stays valid under time pressure
Systematic localisation leaves a probe and regression test for every fix; 18% reopen rate in the pilot	Task 1	Reduces rework in a short project
Module-aligned teams decide a dispute in 1 hop (4 h) against 4 and 6 hops	Task 6	Short loops matter when there are 20 days
Lean documents cost 12.5 h per feature against 38 h; full documents kept for billing and payment	Task 8	Effort is spent where the risk is
Billing and payment first: the pilot's escapes were wrong bills and duplicate payments	Tasks 1, 4, 5	Risk ranking follows the failures that happened
Masking and an audit trail are legal requirements, not options	Section 9.1	Fixed steps in the plan that cannot be dropped to save time

9.4.2 What the redesigned process gives up, and one condition under which it fails

Completeness. The process leaves 120 Low-risk conditions and 11 Medium-risk conditions untested, that is 131 of 400 (32.7%). Medium-risk conditions are executed only once. **Flexibility.** Scope is frozen on day 1 and the plan has no reserve: the 360 hours are fully allocated, so a change has to be paid for with something. **Documentation.** Narrative is thin for the modules outside billing and payment, which costs readability for a newcomer.

A condition under which the process fails: a late change in the tariff regulation. Suppose that in the third week (day 12) the regulator revises the slabs, as it did in week 6 of the original plan. At the new scale perhaps 60 conditions are tariff dependent (assumption). Redesigning their expected values takes 60 x 0.2 = 12 h and re-executing them in the remaining time 60 x 0.25 x 2 = 30 h, in all 42 h, with no reserve to pay for it. Under the plan as it stands the Medium-risk work (not the High-risk work) would be dropped, about 42 / 1.15 / 0.7 = 52 conditions fewer, to 97 of 160, and the exposure covered would fall to about 74.3% (891 of 1,200), or the release date would slip. The tariff table is in any case under version control and is a data item, which helps the retest but not the arithmetic.

What I would change next. (1) Keep a reserve of 10% (36 h) from the start by reducing the Medium-risk target to about 104 conditions, so that a tariff change can be absorbed without reopening the scope. (2) Parameterise the tariff in the test data so that a regulation change alters data and not test cases. (3) Ask the Chief Engineer for a fifth week as an option, to be exercised only if the tariff changes, since the shortfall of 238 hours is a fact that no clever structure removes. (4) Negotiate a staged rollout by district as the formal way to buy confidence that the four-week test cannot supply.

9.5 Observation against prediction, summary and recommendation

Table 9.14: Strategies as rows and measures as columns

Strategy	Fixed overhead	Conditions covered	Risk exposure covered	Score (of 25)	Prediction confirmed?
A. Retain and cut	88 h	235 (58.8%)	78.8%	14	Yes
B. Modify	60 h	269 (67.3%)	87.3%	20	Yes
C. Redesign	110 h	249 (62.3%)	82.3%	17	Yes; cost over the budget was larger than predicted

The predictions were confirmed in direction. The surprise was the cost of C, which exceeds the budget by an assumed Rs 40,000 on top of its 110 hours of set-up. **Recommendation:** modify the process (strategy B) with the cheap audit and masking automation of C; test High-risk conditions three times, Medium once; keep the exit criteria E1 to E8; present the 238-hour shortfall and the list of untested conditions to the Chief Engineer for signature; and hold the tariff table frozen from day 17.

10. Conclusion

The nine tasks were handled as one argument. The pilot of PowerPay failed because testing was treated as an activity that happens near the end and is judged by feel. The exercises of this report show, one task at a time, what replaces that: a fault is located by a procedure that leaves evidence (Task 1); a plan states what will be tested, why and when testing stops (Task 2); the item under test is identified by content, so that a wrong tariff file is caught before a test is run (Task 3); results are reported in three layers for three readers and never as a single pass rate (Task 4); a short policy with measurable statements gives the process authority (Task 5); a team structure is chosen by how fast it makes decisions and how it survives absence (Task 6); skills are built where the gap is, which for PowerPay is domain knowledge and planning (Task 7); documents are produced where money moves and generated elsewhere (Task 8).

The changed requirement of Task 9 tested the process itself. The effort of 598 hours against 360 shows that no structure can test everything, and the right response is to choose what is left out, say so in writing and make the remaining evidence hard to flatter: unexecuted High-risk conditions, open Critical defects, an independent reconciliation sample, a baseline and tariff check, a masking scan and an audit trail. The pieces of the original process that survived were the cheap ones that serve evidence; the pieces that failed were the ones that assumed time. Three results deserve emphasis because they were found by doing and not by assuming: a probe that does not isolate one fault can make a bisection name the wrong commit; seven failed tests can be three defects; and a suite that stops at the boundary of the pilot failure can pass a wrong tariff for the next slab.

The limits of the report are the limits of its evidence. The portal was simulated; effort, cost and likelihoods are modelled; the classmate reproducibility test, the real AI transcripts and the real skills assessment remain to be done with the templates of Appendix D. Within those limits the recommendations are consistent with each other and with the numbers: they can be checked, and where they turn out to be wrong, the records are there to show why.

References

1. IEEE Std 829-2008. *IEEE Standard for Software and System Test Documentation*. IEEE, 2008.
2. Burnstein, I. *Practical Software Testing: A Process-Oriented Approach*. Springer, 2003.
3. Myers, G. J., Sandler, C. and Badgett, T. *The Art of Software Testing*, 3rd ed. Wiley, 2011.
4. Zeller, A. *Why Programs Fail: A Guide to Systematic Debugging*, 2nd ed. Morgan Kaufmann, 2009.
5. Zeller, A. and Hildebrandt, R. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering*, 28(2), 2002.
6. TMMi Foundation. *Test Maturity Model integration (TMMi) Reference Model*.
7. ISTQB. *Standard Glossary of Terms Used in Software Testing*.
8. ISO/IEC/IEEE 29119. *Software and systems engineering: Software testing* (series).
9. Git documentation. *git-bisect*, https://git-scm.com/docs/git-bisect.
10. CSA37 Software Testing, Assignment 4 (CO4): question document for the PowerPay scenario.
11. CSA37 Software Testing, Assignment 1 (CO1): answer key for the Student Portal scenario (used for style and depth only).

Appendix A. Assumptions Register and SRS Excerpt

The question does not give the text of the SRS, the schedule details or several figures. The assumptions below were made by the author and should be confirmed with the faculty and, in a real project, with the stakeholders named in the last column.

Table A.1: Assumptions register

ID	Assumption	Used in	To be confirmed by
A1	Slab tariff is telescopic: each unit is charged at the rate of the slab in which it lies (bill at 500 units = Rs 2,700)	Tasks 1, 3, 4, 9	Billing coordinator
A2	Each tester gives 30 productive hours per week	Tasks 2, 9	Test manager
A3	Ten weeks with builds at weeks 3, 5 and 8 and three cycles; four weeks with weekly builds in Task 9	Tasks 2, 9	Chief Engineer
A4	SRS-13: 2,000 concurrent users and 3 s response for 3 lakh consumers; 8,000 concurrent users for 12 lakh (proportional)	Tasks 2, 9	Analyst
A5	About 40 of the 180 conditions (60 of 400) depend on the tariff	Tasks 2, 9	Test manager
A6	"20% retest effort" means 20% of the execution effort	Tasks 2, 9	Faculty
A7	Each decision hop costs 4 working hours	Task 6	Test manager
A8	Risk tiers of 120, 160 and 120 conditions with weights 5, 3 and 1	Task 9	Test manager and analyst
A9	Tool budget allocation of Rs 1,50,000 as in Table 2.2; cost of training Rs 22,000 per head	Tasks 2, 7, 9	Chief Engineer, training section
A10	Documentation and reporting hours of Tasks 4 and 8	Tasks 4, 8	Test manager
A11	Skill scores of the four testers	Task 7	Test manager (assessment)
A12	Risk probabilities and impacts; likelihood of policies being followed	Tasks 2, 5	Test manager, Chief Engineer

Table A.2: SRS excerpt used in this report (author's wording)

ID	Requirement
SRS-01	Consumer registration with 12-digit consumer number and mobile OTP verification
SRS-02	Login with password policy and lock-out after 5 failed attempts
SRS-03	Meter-reading upload: integer reading not lower than previous reading
SRS-04	Consumption equals current reading minus previous reading; negative consumption rejected
SRS-05	Slab tariff: 0-100 units Rs 3.00, 101-300 Rs 5.00, 301-500 Rs 7.00, above 500 Rs 9.00 per unit
SRS-06	Bill generation with slab-wise breakup, unique bill number and due date
SRS-07	Online payment by UPI, card or net banking; amount must equal the outstanding bill
SRS-08	A transaction identifier is recorded once; duplicate payments are prevented
SRS-09	Payment receipt with unique receipt number, amount and date
SRS-10	SMS receipt sent within 60 seconds of a successful payment
SRS-11	Complaint registration with category and ticket number
SRS-12	Outage reporting by area or feeder with status updates
SRS-13	Load: 2,000 concurrent users with page response within 3 seconds

ID	Requirement
SRS-14	Audit log of all billing and payment transactions, including failed attempts
SRS-15	Tariff table changes by authorised user only, versioned with effective date

Appendix B. Scripts and Evidence Index

All scripts were run in the container used to prepare this report. They are described here so that the measured results can be reproduced; the student should keep the scripts and their output with the submission.

Table B.1: Scripts and the results they produced

Script	Purpose	Results used in
`exp1_build.py`	Builds a Git repository of ten commits with faults F1 to F4 introduced in c09, c05, c03 and c07	Task 1
`exp1_run.py`, `probes/p1..p4.py`, `probes/n1,n2,n4.py`	Runs the print rounds, the trace comparison, `ddmin`, `git bisect run` with differential and absolute probes, fixes and regression tests	Localisation, bisection and fix tables and console figures of Task 1
`tests/test_bill_regression.py`	Four regression tests and a boundary suite	Task 1
`exp1b.py`	Runs four illustrative candidate repairs against the tests	Task 1
`exp3.py`	Simulates the wrong tariff file with SHA-256 manifest check and test execution	Task 3
`exp4.py`	Executes 23 test cases against a simulated build and writes the log	Task 4
`model.py`	Effort, risk, load, sampling, traceability, keyword baseline and masking calculations	Tasks 2, 3, 5, 8, 9
`figs.py`, `outimgs.py`	Draw the charts and render the console outputs as images	All figures

The core of the differential probe and of the masking function is shown below.

```
# probes/p1.py: differential probe for F1 (the 500th unit must cost Rs 7)
from bill import calculate_bill
exit(0 if calculate_bill(500) - calculate_bill(499) == 7.0 else 1)

# masking: keyed token, deterministic, no reversible map kept in the test
environment
import hmac, hashlib
def token(value, prefix, key):
    return prefix + hmac.new(key, value.encode(),
hashlib.sha256).hexdigest()[:8].upper()
```

Appendix C. Test Cases and Results

Table C.1 lists the 33 test cases of the traceability matrix of Task 3 with the requirement they cover and the status in the exercise of Task 4. TC-UI-01 has no requirement and is the orphan test.

Table C.1: Test cases, requirements and status

Test case	Title	Requirement	Status
TC-REG-01	Register with a valid 12-digit consumer number and OTP	SRS-01	Not executed in this exercise
TC-REG-02	Register with an 11-digit consumer number (rejected)	SRS-01	Not executed in this exercise
TC-LOG-01	Log in with valid credentials	SRS-02	Not executed in this exercise
TC-LOG-02	Account locked after 5 failed log-ins	SRS-02	Not executed in this exercise
TC-MR-01	Upload a reading higher than the previous reading	SRS-03	Not executed in this exercise
TC-MR-02	Upload a reading lower than the previous reading (rejected)	SRS-03	Not executed in this exercise
TC-MR-03	Consumption equals current minus previous reading	SRS-03, SRS-04	Not executed in this exercise
TC-BILL-01	Bill for 0 units	SRS-05	PASS
TC-BILL-02	Bill for 1 unit	SRS-05	PASS
TC-BILL-03	Bill for 100 units	SRS-05	PASS
TC-BILL-04	Bill for 101 units	SRS-05	PASS
TC-BILL-05	Bill for 300 units	SRS-05	PASS
TC-BILL-06	Bill for 301 units	SRS-05	PASS
TC-BILL-07	Bill for 500 units	SRS-05, SRS-06	FAIL
TC-BILL-08	Bill for 501 units	SRS-05, SRS-06	FAIL
TC-BILL-09	Bill for 1,000 units	SRS-05	FAIL
TC-BILL-10	Negative consumption rejected	SRS-04	PASS
TC-BILL-11	Bill for 600 units	SRS-05	FAIL
TC-PAY-01	Pay the exact outstanding amount	SRS-07	PASS
TC-PAY-02	Pay less than the outstanding amount (rejected)	SRS-07	PASS
TC-PAY-03	Same callback twice recorded once	SRS-08	FAIL
TC-PAY-04	Outstanding is zero after payment	SRS-07	PASS
TC-PAY-05	Pay more than outstanding (rejected)	SRS-07	PASS
TC-PAY-06	Unknown consumer rejected	SRS-07	PASS
TC-PAY-07	Total paid equals the bill	SRS-07, SRS-08	FAIL
TC-PAY-08	Payment of zero rejected	SRS-07	PASS
TC-RCP-01	Receipt numbers unique	SRS-09	PASS
TC-RCP-02	Receipt amount equals amount paid	SRS-09	PASS
TC-SMS-01	SMS receipt within 60 s	SRS-10	BLOCKED

Test case	Title	Requirement	Status
TC-CMP-01	Complaint registered with ticket number	SRS-11	Not executed in this exercise
TC-AUD-01	Failed payment attempt is audit-logged	SRS-14	FAIL
TC-TRF-01	Tariff change by authorised user only, versioned	SRS-15	Not executed in this exercise
TC-UI-01	Download bill as PDF in dark theme	**none (orphan)**	Not executed in this exercise

Appendix D. Templates for Evidence to be Inserted

D.1 Record sheet for an AI-assistant session (Tasks 1, 2, 4, 5)

Table D.1: AI-assistant session record

Field	Entry
Tool and version	_____
Date and time	_____
Prompt given (pasted in full)	_____
Output (pasted in full)	_____
Statements checked against the facts: correct / false / unsupported	____ / ____ / ____
Change made by the student before use	_____
Reviewer and date	_____

D.2 Incident report template

Table D.2: Test incident report template

Field	Entry
Identifier, date, reporter	_____
Summary	_____
Item, build, baseline, tariff version, environment	_____
Steps to reproduce (numbered, with exact inputs)	_____
Expected result and requirement ID	_____
Actual result	_____
Impact	_____
Severity and priority, with reason	_____
Evidence (log lines, screenshot, file)	_____
Status and owner	_____

D.3 Skills assessment sheet (Task 7)

Table D.3: Skills assessment record

Tester	Category	Score week 0	Score week 4	Evidence (exercise, review)
_____	Technical	__	__	_____
_____	Domain	__	__	_____
_____	Analytical	__	__	_____
_____	Communication	__	__	_____
_____	Planning	__	__	_____

Tester	Category	Score week 0	Score week 4	Evidence (exercise, review)
_____	Tools	__	__	_____